

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC

ĐỒ ÁN THIẾT KẾ KIẾN TRÚC PHẦN MỀM

**Đề Tài : XÂY DỰNG PHẦN MỀM WEB-BASE ĐỂ TỔ CHỨC ĐĂNG KÝ HỌC
PHỤ ĐẠO CHO LƯU HỌC SINH TẠI TRƯỜNG ĐHXD HN**

Lớp: 67PM1

Nhóm thực hiện: 6

Các thành viên:

- | | | |
|--------------------|---------|---------------|
| 1) Nguyễn Minh Đức | 0309767 | (Nhóm Trưởng) |
| 2) Lê Đức Thành | 0321067 | |
| 3) Hoàng Văn Đức | 0005067 | |
| 4) Cao Việt Linh | 0162167 | |
| 5) Hà Quang Hòa | 0005167 | |

Giáo viên hướng dẫn : Phan Hữu Tùng

Hà Nội, 19/04/2026

LỜI CẢM ƠN

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến khoa công nghệ thông tin của Trường Đại học Xây Dựng Hà Nội, cũng như đến Giảng viên Phan Hữu Tùng, với sự tận tâm và sự hướng dẫn dẫn dắt suốt hành trình của chúng em trong việc hoàn thành đồ án môn học thiết kế kiến trúc phần mềm cho đề tài “xây dựng phần mềm web-based để tổ chức đăng ký học phụ đạo cho lưu học sinh tại trường tại trường ĐHXD HN”.

Trong suốt thời gian học tại Trường Đại học Xây Dựng Hà Nội, chúng em đã được đón nhận một môi trường học tập chất lượng cùng sự quan tâm và hỗ trợ từ các thầy cô giáo. Chúng em không thể không nhớ đến sự kiên nhẫn, kiến thức sâu rộng, và lòng nhiệt huyết mà các thầy cô đã dành cho chúng em. Đặc biệt, chúng em xin gửi lời cảm ơn chân thành tới Giảng viên Phan Hữu Tùng vì những lời khuyên, hướng dẫn chi tiết và sự động viên không ngừng đối với nhóm chúng tôi trong quá trình thực hiện đồ án. Sự hiểu biết của cô đã giúp chúng em vượt qua những khó khăn, thắc mắc để hiểu rõ hơn về chủ đề và phương pháp nghiên cứu.

Dù chúng em đã cố gắng hết sức để hoàn thành đề tài một cách tốt nhất, nhưng chúng em thừa nhận rằng không tránh khỏi những sai sót. Thời gian và trình độ kiến thức còn hạn chế nên chúng em cũng chưa thể hoàn thành đồ án một cách hoàn hảo. Vì vậy, chúng em rất mong nhận được ý kiến đóng góp từ các thầy cô và bạn bè, để có thể hoàn thiện đồ án một cách tốt hơn.

Một lần nữa, chúng em xin chân thành cảm ơn sự hỗ trợ và sự dẫn dắt của Giảng viên Phan Hữu Tùng. Chúng em rất biết ơn và tự hào về việc được học tập và nghiên cứu tại Trường Đại học Xây Dựng Hà Nội, nơi mà kiến thức, kỹ năng và tinh thần học tập đã được truyền đạt một cách xuất sắc.

Em xin chân thành cảm ơn.

MỤC LỤC

CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI	5
1.1. Lý do chọn đề tài	5
1.2. Mục tiêu đề tài.....	6
1.3. Phạm vi và giới hạn.....	6
1.3.1. Yêu cầu chức năng của hệ thống	7
1.3.2. Yêu cầu phi chức năng của hệ thống	8
1.4. Công nghệ sử dụng	8
1.4.1. Giới thiệu ngôn ngữ PHP	8
1.4.2. Giới thiệu Lavarel.....	8
CHƯƠNG II: PHÂN TÍCH HỆ THỐNG	10
2.1. Xác định tác nhân và trường hợp sử dụng hệ thống	10
2.2. Biểu đồ Use Case.....	11
2.3.1. Use Case toàn bộ hệ thống.....	11
2.3.2. Đặc tả Use Case	11
2.3. Biểu đồ Biểu đồ tuần tự chức năng.....	19
2.3.1. Biểu đồ tuần tự chức năng Đăng nhập	19
2.3.2. Biểu đồ tuần tự chức năng Thêm đợt phụ đạo	20
2.3.3. Biểu đồ tuần tự chức năng Sửa đợt phụ đạo.....	21
2.3.4. Biểu đồ tuần tự chức năng Xóa đợt phụ đạo	22
2.3.5. Biểu đồ tuần tự chức năng Thêm người dùng.....	22
2.3.6. Biểu đồ tuần tự chức năng Sửa người dùng	22
2.3.7. Biểu đồ tuần tự chức năng Xóa người dùng	23
2.3.8. Biểu đồ tuần tự chức năng Cài đặt hệ thống.....	24
2.3.9. Biểu đồ tuần tự chức năng Thanh toán tiền phụ đạo.....	24
2.3.10. Biểu đồ tuần tự chức năng Thống kê đợt phụ đạo	25
2.3.11. Biểu đồ tuần tự chức năng Gửi email về bộ môn	25
2.3.12. Biểu đồ tuần tự chức năng Phân công giảng viên phụ đạo.....	26
2.3.13. Biểu đồ tuần tự chức năng Sửa bộ môn	26
2.3.14. Biểu đồ tuần tự chức năng Đăng ký học phụ đạo	27
2.3.15. Biểu đồ tuần tự chức năng Hủy đăng ký phụ đạo.....	27
2.4. Biểu đồ Biểu đồ hoạt động chức năng.....	28
2.4.1. Biểu đồ hoạt động chức năng Đăng nhập.....	28
2.4.2. Biểu đồ hoạt động chức năng Thêm đợt phụ đạo	29
2.3.3. Biểu đồ hoạt động chức năng Sửa đợt phụ đạo.....	30
2.3.4. Biểu đồ hoạt động chức năng Xóa đợt phụ đạo	31
2.3.5. Biểu đồ hoạt động chức năng Thêm người dùng.....	32

2.3.6.	Biểu đồ hoạt động chức năng Sửa người dùng	33
2.3.7.	Biểu đồ hoạt động chức năng Xóa người dùng.....	34
2.3.8.	Biểu đồ hoạt động chức năng Cài đặt hệ thống.....	35
2.3.9.	Biểu đồ hoạt động chức năng Thanh toán tiền phụ đạo.....	36
2.3.10.	Biểu đồ hoạt động chức năng Thống kê đợt phụ đạo	36
2.3.11.	Biểu đồ hoạt động chức năng Gửi email về bộ môn	36
2.3.12.	Biểu đồ hoạt động chức năng Phân công giảng viên phụ đạo	38
2.3.13.	Biểu đồ hoạt động chức năng Sửa bộ môn	39
2.3.14.	Biểu đồ hoạt động chức năng Đăng ký học phụ đạo.....	40
2.3.15.	Biểu đồ hoạt động chức năng Hủy đăng ký học phụ đạo.....	41
CHƯƠNG III: THIẾT KẾ HỆ THỐNG		42
3.1.	Sơ đồ kiến trúc tổng thể hệ thống:.....	42
3.1.1.	Khái niệm và lý do áp dụng.....	42
3.1.2.	Ánh xạ Hexagonal ↔ cấu trúc mã nguồn.....	42
3.1.3.	Sơ đồ kiến trúc tổng thể.....	43
3.1.4.	Luồng xử lý một request (chuỗi Hexagonal).....	43
3.1.5.	Phân tích chi tiết từng tầng	44
3.1.6.	So sánh với kiến trúc Laravel monolith truyền thống.....	46
3.1.7.	Ưu điểm đạt được trong thực tế dự án	47
3.2.	Thiết kế pattern	47
3.2.1.	State Pattern.....	47
3.2.2.	Adapter Pattern	52
3.2.3.	Decorator Pattern	55
3.2.4.	Observer Pattern.....	58
3.2.5.	Repository Pattern	63
3.3.	Thiết kế cơ sở dữ liệu.....	69
	Thiết kế các bảng	69
	Quan hệ giữa các thực thể	73
3.4.	Giao diện người dùng.....	75
Chương IV. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG.....		81
4.1.	Môi trường triển khai	81
4.1.1.	Kiến trúc tổng thể.....	81
4.1.2.	Môi trường phát triển (Development)	82
4.1.3.	Môi trường triển khai thực tế (Production — định hướng).....	82
4.2.	Cài đặt hệ thống	83
4.2.1.	Yêu cầu hệ thống.....	83
4.2.2.	Các bước cài đặt chương trình	85
Chương V. Kết quả và đánh giá.....		88
5.1.	Kết quả thử nghiệm hệ thống	88

5.1.1. Kiểm thử tự động — PHPUnit (Remedial System)	88
5.1.2. Kiểm thử hồi quy thủ công — ba vai trò	89
5.1.3. Kiểm thử tích hợp	90
5.2. Đánh giá hiệu quả hệ thống.....	91
5.2.1. Mức độ đáp ứng yêu cầu nghiệp vụ.....	91
5.2.2. Đánh giá kỹ thuật.....	92
5.2.3. Đánh giá hiệu quả vận hành	93
5.2.4. Kết luận chương 5	93

CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Nhằm đáp ứng mục tiêu phát triển và hội nhập, trong những năm qua trường Đại học Xây dựng đã đẩy mạnh hoạt động hợp tác quốc tế trong đó có các chương trình hợp tác, liên kết đào tạo. Hoạt động hợp tác, liên kết đào tạo mở ra cơ hội tiếp cận, học tập kinh nghiệm của các nền giáo dục tiên tiến, nhờ đó số lượng du học sinh tại trường Đại học Xây dựng rất đông đảo. Tuy nhiên, các bạn du học sinh thường gặp khó khăn trong học tập bởi vì rào cản ngôn ngữ, lượng bài tập lớn.... Do đó để đảm bảo chất lượng đào tạo, nhà trường đã giao cho phòng Đào tạo tổ chức phụ đạo các môn học lý thuyết (không phụ đạo các mô thực hành, thí nghiệm, thực tập) cho du học sinh trước mỗi kỳ thi.

Quy trình tổ chức phụ đạo cho du học sinh hiện tại được thực hiện như sau:

Bước 1: Du học sinh nộp đơn học phụ đạo tại khoa Đào tạo quốc tế trước đợt thi 02 tuần.

Bước 2: Khoa Đào tạo quốc tế lập danh sách các môn, các lớp sinh viên có nguyện vọng học phụ đạo và chuyển về phòng Đào tạo.

Bước 3: Phòng Đào tạo gửi thông báo về các Bộ môn để phân công giảng viên phụ đạo.

Bước 4: Khoa Đào tạo quốc tế thông báo thông tin giảng viên phụ đạo để sinh viên liên hệ với giảng viên để thống nhất lịch phụ đạo.

Bước 5: Khoa Đào tạo quốc tế lập bảng thanh toán khối lượng và chuyển phòng Tài vụ để thanh toán cho giảng viên.

Với quy trình như trên gây tốn kém thời gian cho Khoa Đào tạo quốc tế, phòng Đào tạo và không thực sự thuận lợi cho sinh viên.

Những phân tích trên đây cho thấy sự cần thiết phải xây dựng một hệ thống hỗ trợ công tác tổ chức phụ đạo môn học cho du học sinh tại trường Đại học Xây dựng.

1.2. Mục tiêu đề tài

Xây dựng một giải pháp phần mềm dựa trên nền web. Phần mềm bao gồm các chức năng đăng ký học phụ đạo, phân công giảng viên phụ đạo, gửi email thông báo tới bộ môn phụ đạo, thanh toán tiền phụ đạo cho giảng viên, thống kê báo cáo.

Giải pháp trên sẽ góp phần giảm tải khối lượng công việc của khoa Đào tạo quốc tế, phòng Đào tạo trong công tác tổ chức phụ đạo môn học dành cho du học sinh nước ngoài trước các kỳ thi.

Đem lại sự thuận lợi cho du học sinh khi đăng ký học phụ đạo.

1.3. Phạm vi và giới hạn

Để giải quyết bài toán nêu trên, sinh viên thực hiện đồ án tốt nghiệp đề xuất xây dựng giải pháp phần mềm dựa trên nền web sử dụng ngôn ngữ PHP với framework laravel để xây dựng hệ thống, sử dụng hệ quản trị cơ sở dữ liệu SQL Server để lưu trữ

dữ liệu. Đây đều là giải pháp công nghệ có tốc độ thực thi nhanh, dễ cài đặt triển khai.

1.3.1. Yêu cầu chức năng của hệ thống

STT	Phân hệ	Chức năng
	Quản lý người dùng	Thêm mới người dùng
		Cập nhật thông tin người dùng
		Xoá người dùng
2	Quản lý đợt phụ đạo	Thêm mới đợt phụ đạo
		Cập nhật thông tin đợt phụ đạo
		Xoá đợt phụ đạo
		Xem chi tiết danh sách sinh viên đăng ký học phụ đạo
3	Đăng ký phụ đạo	Sinh viên đăng ký phụ đạo môn học
		Sinh viên hủy môn học đăng ký phụ đạo
4	Phân công giảng viên phụ đạo	Gửi email danh sách các môn, các lớp, danh sách sinh viên đăng ký phụ đạo tới các Bộ môn.
		Bộ môn phân công giảng viên phụ đạo cho từng môn, từng lớp.
5	Quản lý thông tin bộ môn	Cập nhật thông tin Bộ môn để tiện liên hệ khi tổ chức học phụ đạo.
6	Thanh toán tiền phụ đạo cho giảng viên	Lập bảng kê thanh toán tiền phụ đạo
		Thống kê, báo cáo về các đợt phụ đạo

		Gửi email thông báo tiền phụ đạo về cho các GV
7	Cài đặt hệ thống	Cài đặt thiết lập các thông tin phục vụ cho hệ thống

Bảng 1: Bảng yêu cầu chức năng

1.3.2. Yêu cầu phi chức năng của hệ thống

Yêu cầu giao diện

Giao diện cần phải đảm bảo khả năng truy nhập các trang một cách nhanh chóng, tuy nhiên cần phải đơn giản, dễ sử dụng. Giao diện cũng cần có tính thẩm mỹ cao, sử dụng màu sắc hợp lý, bắt mắt. Bố trí hàng cột thẳng hàng, phân chia thành các vùng chức năng rõ rệt, sử dụng ngôn ngữ tiếng Việt.

Yêu cầu khả năng mở rộng

Trang web có khả năng mở rộng tính năng trong tương lai mà không làm ảnh hưởng tới các tính năng hiện tại, có thể nâng cấp dễ dàng.

1.4. Công nghệ sử dụng

1.4.1. Giới thiệu ngôn ngữ PHP

PHP (Hypertext Preprocessor) là một ngôn ngữ lập trình kịch bản phía máy chủ (server-side) được sử dụng rộng rãi trong phát triển ứng dụng web. PHP cho phép nhúng trực tiếp vào HTML, giúp xây dựng các trang web động một cách dễ dàng. Ngôn ngữ này có cú pháp đơn giản, dễ tiếp cận, đồng thời hỗ trợ tốt nhiều hệ quản trị cơ sở dữ liệu như MySQL, PostgreSQL.

Ngoài ra, PHP có cộng đồng phát triển lớn, tài liệu phong phú và khả năng chạy trên nhiều nền tảng khác nhau. Nhờ đó, PHP trở thành một trong những công nghệ phổ biến để xây dựng các hệ thống web từ cơ bản đến nâng cao.

1.4.2. Giới thiệu Laravel

Laravel là một framework mã nguồn mở được xây dựng trên nền tảng PHP, hỗ trợ phát triển ứng dụng web một cách nhanh chóng và hiệu quả. Laravel áp dụng mô hình kiến trúc MVC (Model – View – Controller), giúp tổ chức mã nguồn rõ ràng, dễ bảo trì và mở rộng.

Bên cạnh việc xây dựng web truyền thống, Laravel còn hỗ trợ mạnh mẽ trong việc phát triển RESTful API. Framework cung cấp hệ thống routing riêng cho API (thông qua file api.php), giúp định nghĩa các endpoint một cách rõ ràng và dễ quản lý. Laravel

hỗ trợ trả dữ liệu dưới dạng JSON, phù hợp cho việc kết nối với các ứng dụng frontend như ReactJS, mobile app hoặc các hệ thống khác.

Ngoài ra, Laravel còn tích hợp nhiều tính năng phục vụ phát triển API như:

- Eloquent ORM: thao tác cơ sở dữ liệu dễ dàng, trả dữ liệu dạng JSON
- Middleware: xử lý xác thực và phân quyền (authentication, authorization)
- Laravel Sanctum / Passport: hỗ trợ xác thực API bằng token
- Validation: kiểm tra dữ liệu đầu vào
- Resource & Collection: chuẩn hóa dữ liệu trả về API

Nhờ những tính năng này, Laravel không chỉ phù hợp để xây dựng ứng dụng web mà còn là một lựa chọn mạnh mẽ để phát triển các hệ thống backend API hiện đại.

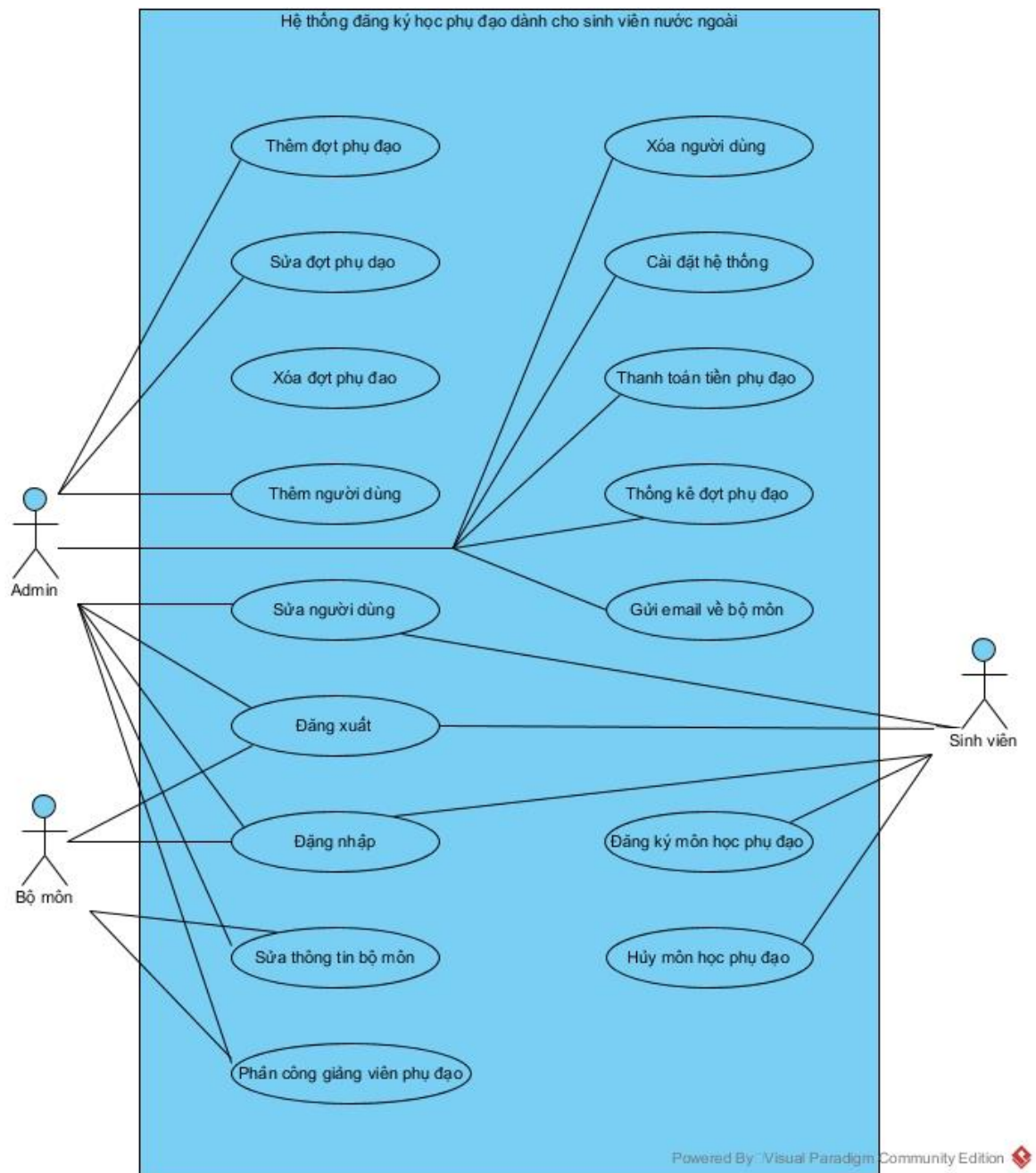
CHƯƠNG II: PHÂN TÍCH HỆ THỐNG

2.1. Xác định tác nhân và trường hợp sử dụng hệ thống

- Tác nhân: Admin, Bộ môn, Sinh viên
- Nhu cầu Admin
 - Quản lý đợt phụ đạo
 - Quản lý người dùng
 - Phân công Bộ môn giảng dạy
 - Thanh toán tiền phụ đạo
 - Thống kê, báo cáo
- Nhu cầu Bộ môn
 - Phân công giảng viên phụ đạo
- Nhu cầu Sinh viên
 - Đăng ký phụ đạo
 - Xem thông tin giảng viên phụ đạo

2.2. Biểu đồ Use Case

2.3.1. Use Case toàn bộ hệ thống



2.3.2. Đặc tả Use Case

Use Case Đăng nhập

Name	Đăng nhập
Actor	Admin, Bộ môn, Sinh viên
Descriptio	Cho phép người dùng đăng nhập vào hệ thống

Normal Flow	1. Người dùng nhập tài khoản gồm tên đăng nhập và mật khẩu 2. Hệ thống kiểm tra tính hợp lệ của thông tin tài khoản 3. Hệ thống kiểm tra tài khoản có tồn tại trong cơ sở dữ liệu 4. Chuyển đến trang chủ (phụ thuộc vào Actor)
Alternative Flow	1. Nếu thông tin tài khoản không hợp lệ, hệ thống đưa ra thông báo và yêu cầu nhập lại 2. Nếu tài khoản không tồn tại trong cơ sở dữ liệu, hệ thống đưa ra thông báo và yêu cầu nhập lại

Bảng 2: Bảng đặc tả Use case Đăng nhập

Use Case Thêm đợt phụ đạo

Name	Thêm đợt phụ đạo
Actor	Admin
Descriptio	Cho phép người dùng thêm mới đợt phụ đạo
Normal Flow	1. Người dùng chọn chức năng thêm đợt phụ đạo 2. Người dùng nhập thông tin đợt phụ đạo 3. Hệ thống kiểm tra thời gian đợt phụ đạo 4. Hệ thống thêm mới đợt phụ đạo vào cơ sở dữ liệu 5. Hệ thống hiển thị đợt phụ đạo được thêm mới
Alternative Flow	1. Nếu thời gian đợt phụ đạo không hợp lệ, hệ thống thông báo thời gian không hợp lệ 2. Nếu thêm thất bại, hệ thống đưa ra thông báo 3. Người dùng hủy thao tác thêm mới, hệ thống quay lại giao diện chính

Bảng 3: Bảng đặc tả Use case Thêm đợt phụ đạo

Use Case Sửa đợt phụ đạo

Name	Sửa đợt phụ đạo
Actor	Admin
Descriptio	Cho phép người dùng chỉnh sửa đợt phụ đạo

Normal Flow	1. Người dùng chọn đợt phụ đạo muốn sửa trong danh sách đợt phụ đạo 2. Người dùng nhập thông tin đợt phụ đạo 3. Hệ thống kiểm tra thời gian đợt phụ đạo 4. Hệ thống chỉnh sửa đợt phụ đạo trong cơ sở dữ liệu 5. Hệ thống hiển thị thông tin đợt phụ đạo đã chỉnh sửa
Alternative Flow	1. Nếu thông thời gian phụ đạo không hợp lệ, hệ thống thông báo thời gian không hợp lệ 2. Nếu chỉnh sửa thất bại, hệ thống đưa ra thông báo 3. Người dùng hủy thao tác chỉnh sửa, hệ thống quay lại giao diện chính

Bảng 4: Bảng đặc tả Use case Sửa đợt phụ đạo

Use Case Xóa đợt phụ đạo

Name	Xóa đợt phụ đạo
Actor	Admin
Descriptio	Cho phép người dùng xóa đợt phụ đạo
Normal Flow	1. Người dùng chọn đợt phụ đạo muốn xóa trong danh sách đợt phụ đạo 2. Xác nhận xóa 3. Hệ thống kiểm tra có sinh viên đăng ký phụ đạo 4. Hệ thống xóa đợt phụ đạo trong cơ sở dữ liệu 5. Loại bỏ đợt phụ đạo trên danh sách hiển thị
Alternative Flow	1. Nếu có sinh viên đăng ký, thông báo không thể xóa 2. Nếu xóa thất bại, hệ thống đưa ra thông báo 3. Người dùng hủy thao tác chỉnh xóa, hệ thống quay lại giao diện chính

Bảng 5: Bảng đặc tả Use case Xóa đợt phụ đạo

Use Case Thêm người dùng

Name	Thêm người dùng
Actor	Admin
Descriptio	Cho phép người dùng thêm tài khoản người dùng hệ thống

Normal Flow	1. Người dùng chọn chức năng thêm người dùng 2. Nhập thông tin người dùng 3. Hệ thống kiểm tra tính hợp lệ của thông tin 4. Hệ thống thêm mới người dùng vào cơ sở dữ liệu 5. Hệ thống hiển thị người dùng được thêm mới
Alternative Flow	1. Nếu thông tin người dùng không hợp lệ, hệ thống thông báo thông tin không hợp lệ và yêu cầu nhập lại 2. Nếu thêm người dùng thất bại, hệ thống đưa ra thông báo 3. Người dùng hủy thao tác chỉnh xóa, hệ thống quay lại giao diện chính

Bảng 6: Bảng đặc tả Use case Thêm người dùng

Use Case Sửa người dùng

Name	Sửa người dùng
Actor	Admin
Descriptio	Cho phép người dùng chỉnh sửa tài khoản người dùng hệ thống
Normal Flow	1. Người dùng chọn người dùng muốn sửa trong danh sách người dùng 2. Người dùng nhập thông tin người dùng 3. Hệ thống kiểm tra tính hợp lệ của thông tin 4. Hệ thống chỉnh sửa người dùng trong cơ sở dữ liệu 5. Hệ thống hiển thị thông tin người dùng đã chỉnh sửa
Alternative Flow	1. Nếu thông tin người không hợp lệ, hệ thống thông báo thông tin không hợp lệ và yêu cầu nhập lại 2. Nếu chỉnh sửa thất bại, hệ thống đưa ra thông báo 3. Người dùng hủy thao tác chỉnh sửa, hệ thống quay lại giao diện chính

Bảng 7: Bảng đặc tả Use case Sửa người dùng

Use Case Xóa người dùng

Name	Xóa Người dùng
Actor	Admin
Descriptio	Cho phép người dùng xóa tài khoản người dùng hệ thống

Normal Flow	1. Người dùng chọn người dùng muốn xóa trong danh sách người dùng 2. Xác nhận xóa 3. Hệ thống xóa người dùng trong cơ sở dữ liệu 4. Loại bỏ người dùng trên danh sách hiển thị
Alternative Flow	1. Nếu xóa thất bại, hệ thống đưa ra thông báo 2. Người dùng hủy thao tác chỉnh sửa, hệ thống quay lại giao diện chính

Bảng 8: Bảng đặc tả Use case Xóa người dùng

Use Case Cài đặt hệ thống

Name	Cài đặt hệ thống
Actor	Admin
Descriptio	Cho phép người dùng cài đặt các thông tin cơ bản để hệ thống hoạt động được
Normal Flow	1. Người dùng chọn chức năng cài đặt hệ thống 2. Người dùng nhập thông tin cấu hình hệ thống 3. Hệ thống kiểm tra tính hợp lệ của thông tin 4. Hệ thống lưu lại thông tin cấu hình hệ thống vào cơ sở dữ liệu
Alternative Flow	1. Nếu thất bại, hệ thống đưa ra thông báo

Bảng 9: Bảng đặc tả Use case Cài đặt hệ thống

Use Case Thanh toán tiền phụ đạo

Name	Thanh toán tiền phụ đạo
Actor	Admin
Descriptio	Cho phép người dùng thanh toán tiền phụ đạo cho giảng viên
Normal Flow	1. Người dùng chọn chức năng thanh toán tiền phụ đạo 2. Hệ thống tổng hợp, tính tiền phụ đạo cho giảng viên theo từng môn học và hiển thị danh sách thanh toán tiền phụ đạo cho người dùng 4. Người dùng chọn xuất danh sách ra excel 5. Hệ thống xuất thông tin thành file excel

Alternative Flow	
------------------	--

Bảng 10: Bảng đặc tả Use case Thanh toán tiền phụ đạo

Use Case Thống kê đợt phụ đạo

Name	Thống kê đợt phụ đạo
Actor	Admin
Descriptio	Cho phép người dùng tổng hợp thông tin về tổng số sinh viên, tổng số lớp, tổng số môn học, tổng tiền phụ đạo trong từng đợt phụ đạo
Normal Flow	1. Người dùng chọn chức năng thống kê đợt phụ đạo 2. Hệ thống tổng hợp thông tin theo từng đợt phụ đạo và hiển thị danh sách thống kê được
Alternative Flow	

Bảng 11: Bảng đặc tả Use case Thống kê đợt phụ đạo

Use Case Gửi email về bộ môn

Name	Gửi email về bộ môn
Actor	Admin
Descriptio	Cho phép người dùng gửi email danh sách môn học phụ đạo kèm theo danh sách sinh viên theo từng môn về bộ môn được phân phụ đạo môn học
Normal Flow	1. Người dùng chọn chức năng gửi email về bộ môn 2. Người dùng nhập thông tin của email gửi về bộ môn 3. Hệ thống kiểm tra tính hợp lệ của thông tin email 4. Hệ thống gửi email về bộ môn và thông báo gửi mail thành công
Alternative Flow	1. Nếu thông tin không hợp lệ, hệ thống thông báo và yêu cầu nhập lại thông tin 2. Nếu gửi email thất bại, hệ thống thông báo lỗi không gửi được email

Bảng 12: Bảng đặc tả Use case Gửi email về bộ môn

Use Case Phân công giảng viên phụ đạo

Name	Phân công bộ môn phụ đạo
Actor	Admin, Bộ môn
Descriptio	Cho phép người dùng phân công giảng viên phụ đạo cho từng môn học mà sinh viên đăng ký phụ đạo
Normal Flow	1. Người dùng chọn môn học muốn phân công giảng viên 2. Người dùng chọn giảng viên , nhập thông tin giảng viên và chọn lưu 3. Hệ thống lưu lại thông tin phân công giảng viên 4. Hệ thống hiển thị thông tin giảng viên được phân công
Alternative Flow	1. Nếu thất bại, hệ thống đưa ra thông báo 2. Người dùng hủy thao tác chỉnh sửa, hệ thống quay lại giao diện chính

Bảng 13: Bảng đặc tả Use case Phân công giảng viên phụ đạo

Use Case Sửa bộ môn

Name	Sửa thông tin bộ môn
Actor	Admin, Bộ môn
Descriptio	Cho phép người dùng chỉnh sửa thông tin email bộ môn và số điện thoại trưởng bộ môn của bộ môn
Normal Flow	1. Người dùng chọn bộ môn muốn sửa trong danh sách bộ môn 2. Người dùng chọn chức năng sửa bộ môn 3. Hệ thống chỉnh sửa bộ môn trong cơ sở dữ liệu 4. Hệ thống hiển thị thông tin bộ môn đã chỉnh sửa
Alternative Flow	1. Nếu chỉnh sửa thất bại, hệ thống đưa ra thông báo 2. Người dùng hủy thao tác chỉnh sửa, hệ thống quay lại giao diện chính

Bảng 14: Bảng đặc tả Use case Sửa bộ môn

Use Case Đăng ký học phụ đạo

Name	Đăng ký môn học phụ đạo
Actor	Sinh viên
Descriptio	Cho phép sinh viên đăng ký môn học phụ đạo khi có đợt phụ đạo được mở

Normal Flow	1. Sau khi sinh viên đăng nhập, hệ thống chuyển tới giao diện đăng ký môn học phụ đạo 2. Sinh viên chọn các môn học muốn đăng ký phụ đạo và chọn lưu đăng ký 3. Hệ thống lưu lại và hiển thị các môn học mà sinh viên đăng ký phụ đạo
Alternative Flow	1. Nếu đăng ký thất bại, quy trở lại giao diện chính

Bảng 15: Bảng đặc tả Use case Đăng ký môn học phụ đạo

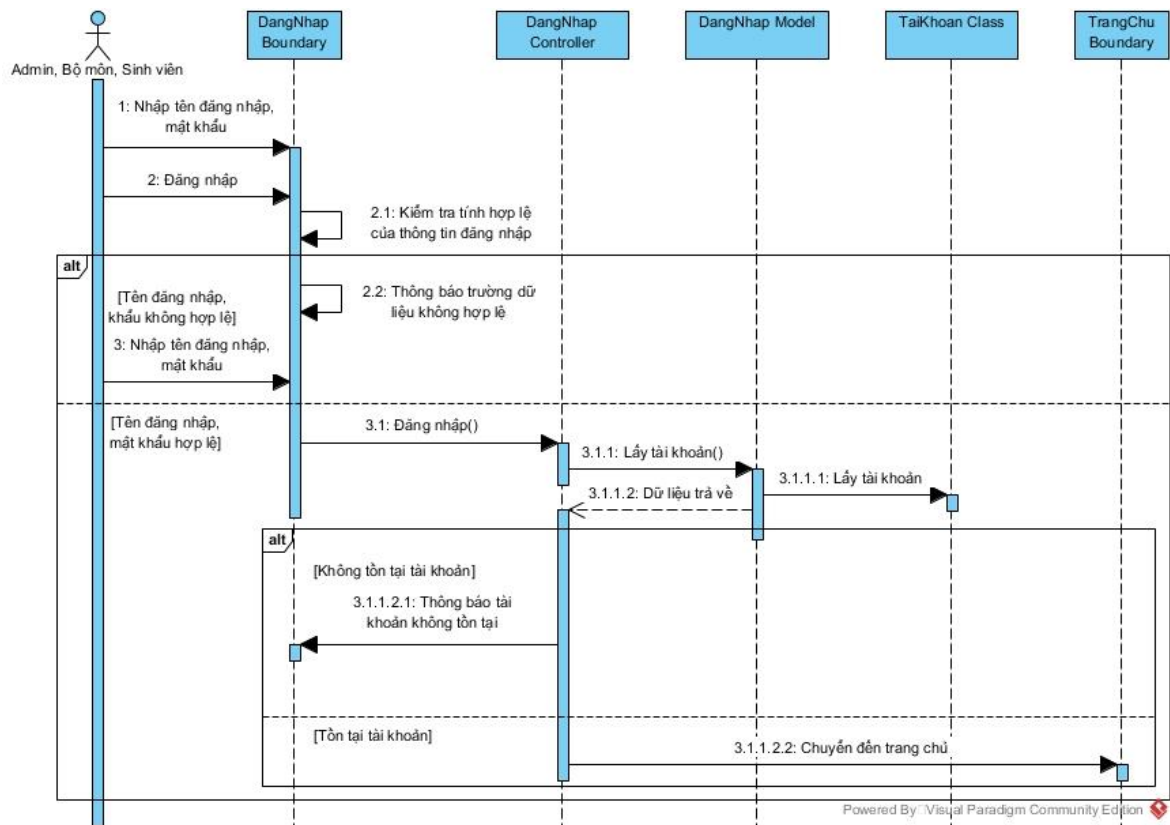
Use Case Hủy đăng ký phụ đạo

Name	Hủy đăng ký môn học phụ đạo
Actor	Sinh viên
Descriptio	Cho phép sinh viên hủy các môn học đã đăng ký phụ đạo
Normal Flow	1. Sinh viên chọn các môn học muốn hủy đăng ký phụ đạo và chọn xóa đăng ký 2. Hệ thống xóa và hiển thị các môn học mà sinh viên đăng ký phụ đạo
Alternative Flow	1. Nếu hủy đăng ký thất bại, quay lại giao diện chính

Bảng 16: Bảng đặc tả Use case Hủy đăng ký môn học phụ đạo

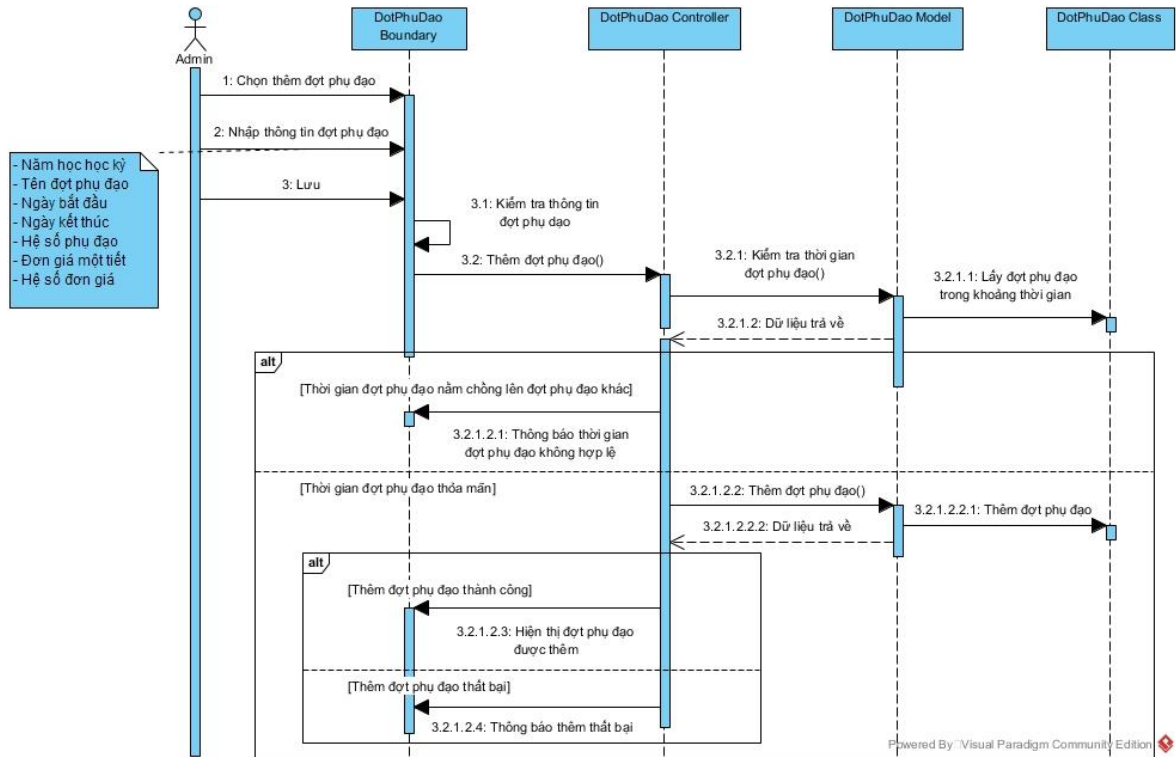
2.3. Biểu đồ Biểu đồ tuần tự chức năng

2.3.1. Biểu đồ tuần tự chức năng Đăng nhập



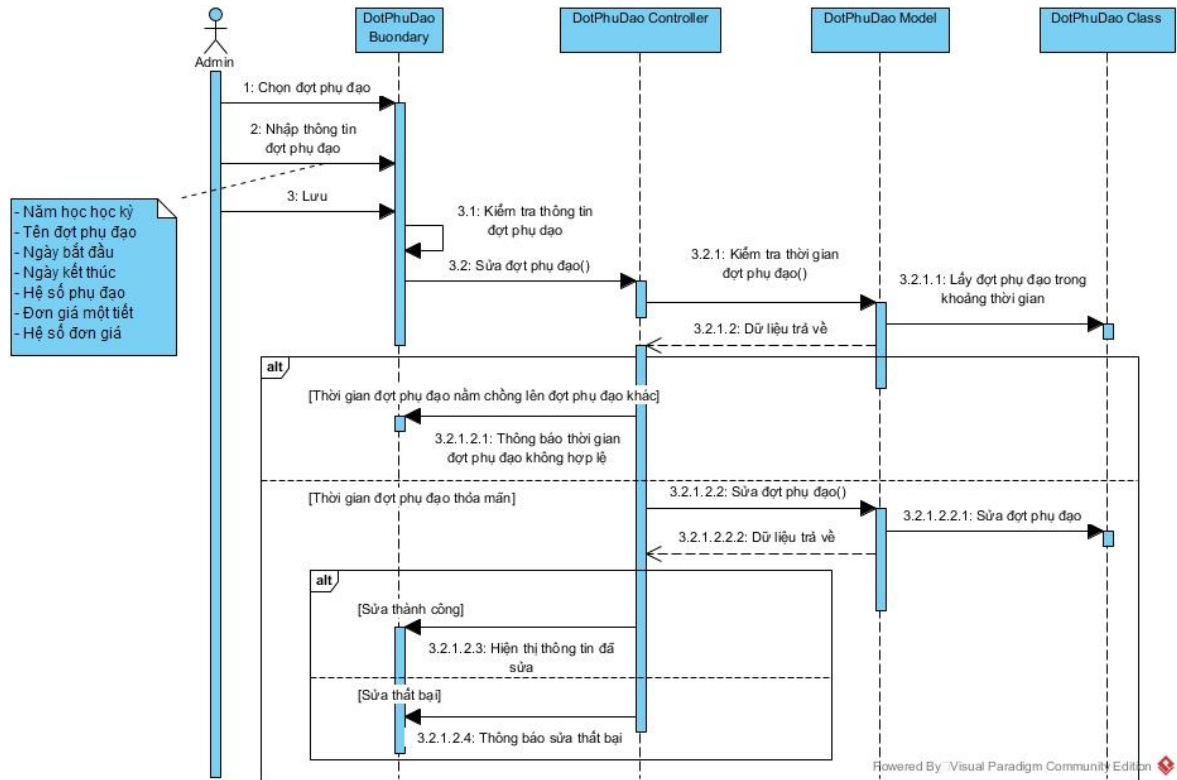
Hình 1: Biểu đồ Biểu đồ tuần tự chức năng Đăng nhập

2.3.2. Biểu đồ tuần tự chức năng Thêm đợt phụ đạo



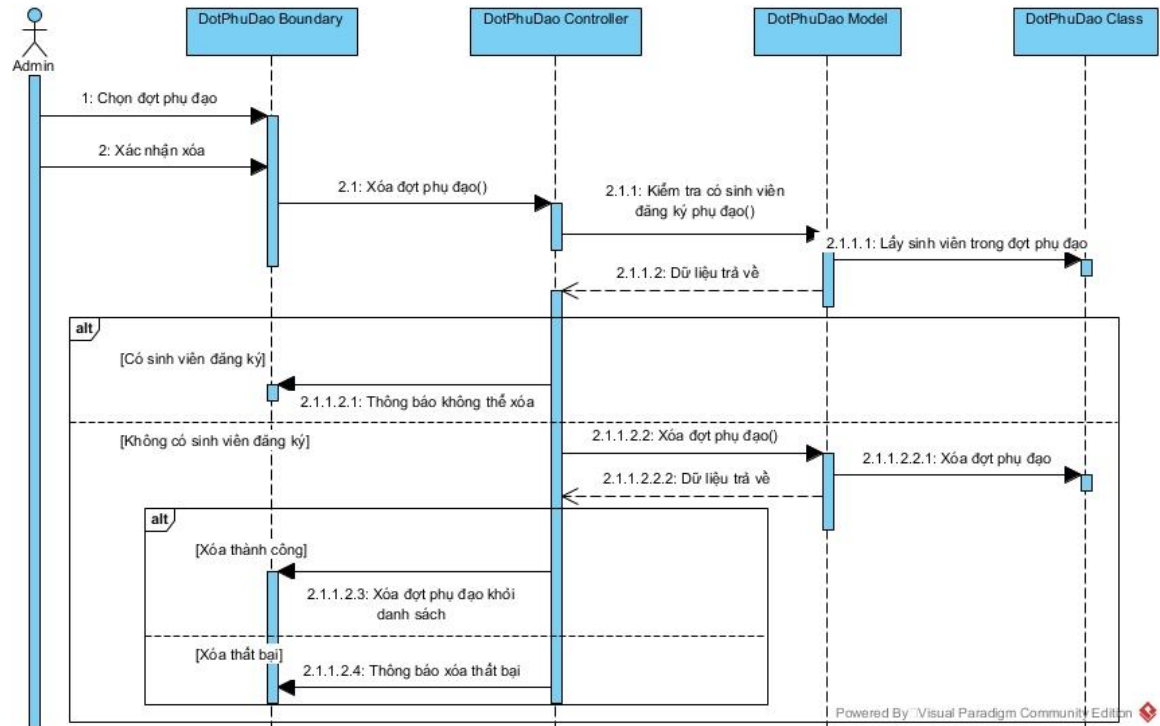
Hình 2: Biểu đồ tuần tự chức năng Thêm đợt phụ đạo

2.3.3. Biểu đồ tuần tự chức năng Sửa đợt phụ đạo



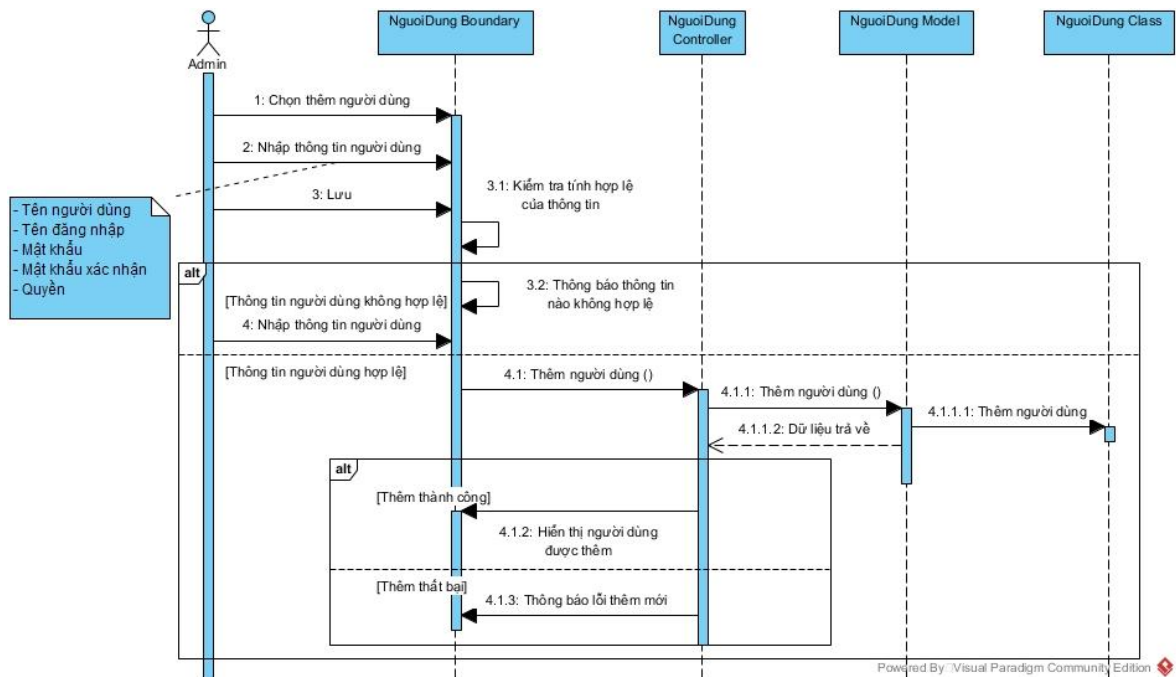
Hình 3: Biểu đồ tuần tự chức năng Sửa đợt phụ đạo

2.3.4. Biểu đồ tuần tự chức năng Xóa đợt phụ đạo



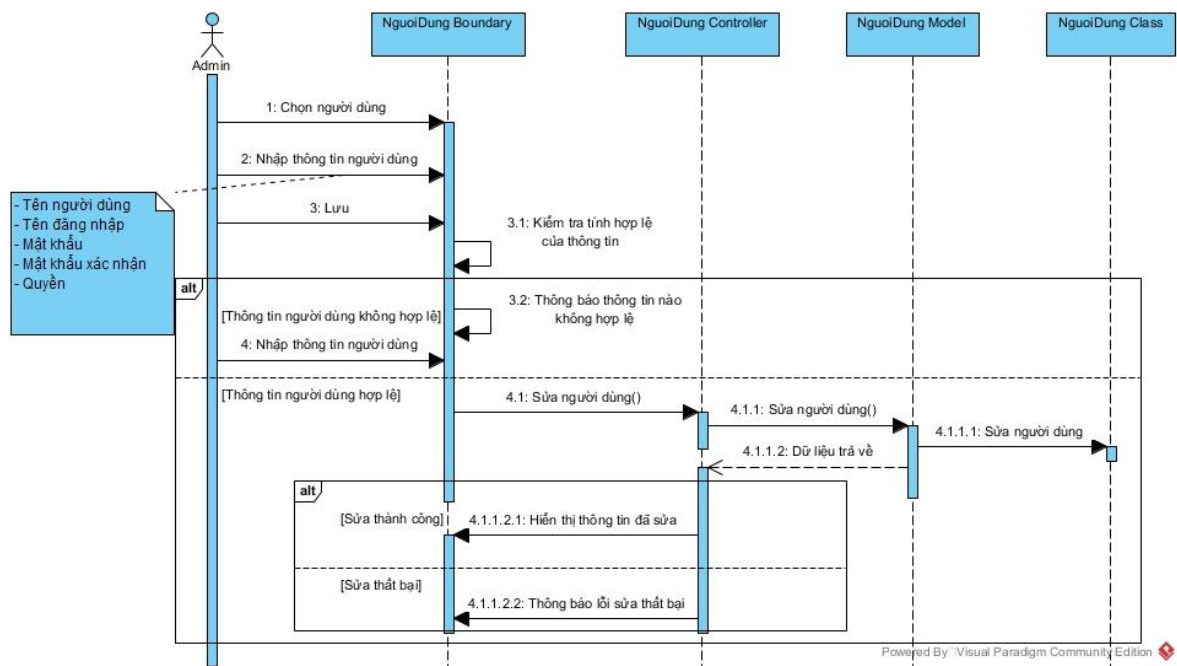
Hình 4: Biểu đồ tuần tự chức năng Xóa đợt phụ đạo

2.3.5. Biểu đồ tuần tự chức năng Thêm người dùng



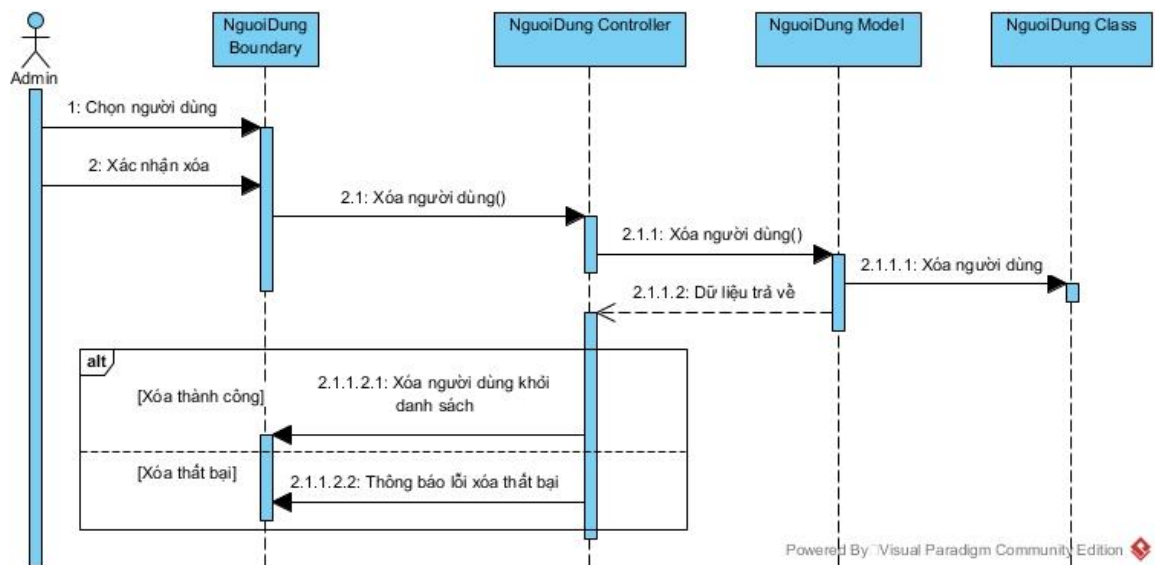
Hình 5: Biểu đồ tuần tự chức năng Thêm người dùng

2.3.6. Biểu đồ tuần tự chức năng Sửa người dùng



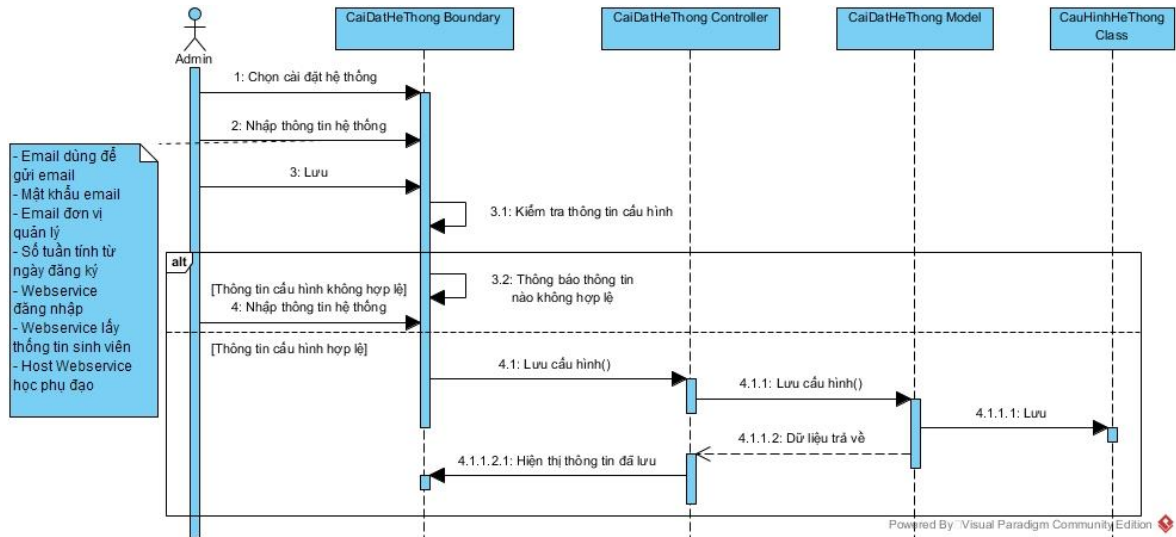
Hình 6: Biểu đồ tuần tự chức năng Sửa người dùng

2.3.7. Biểu đồ tuần tự chức năng Xóa người dùng



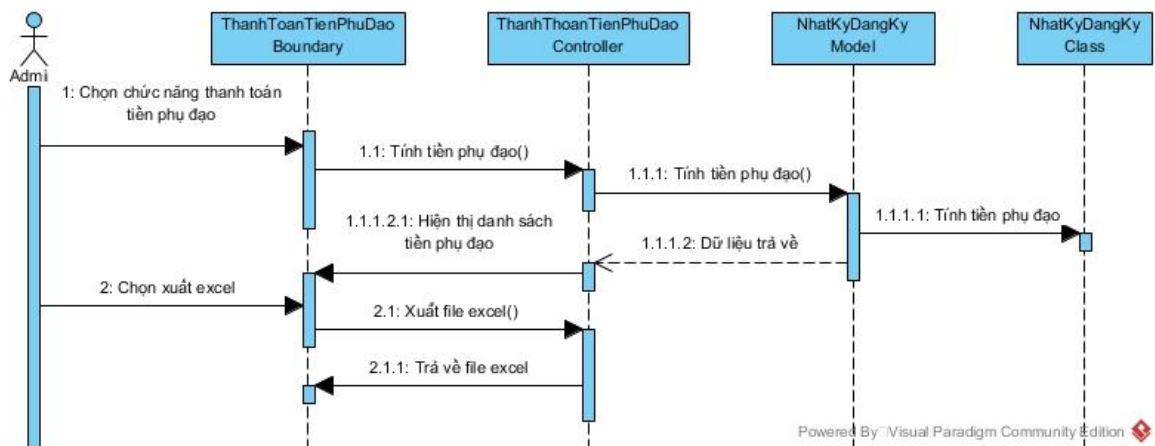
Hình 7: Biểu đồ tuần tự chức năng Xóa người dùng

2.3.8. Biểu đồ tuần tự chức năng Cài đặt hệ thống



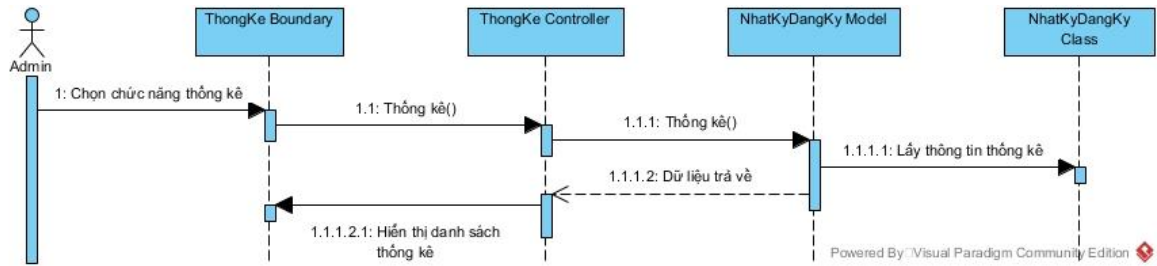
Hình 8: Biểu đồ tuần tự chức năng Cài đặt hệ thống

2.3.9. Biểu đồ tuần tự chức năng Thanh toán tiền phụ đạo



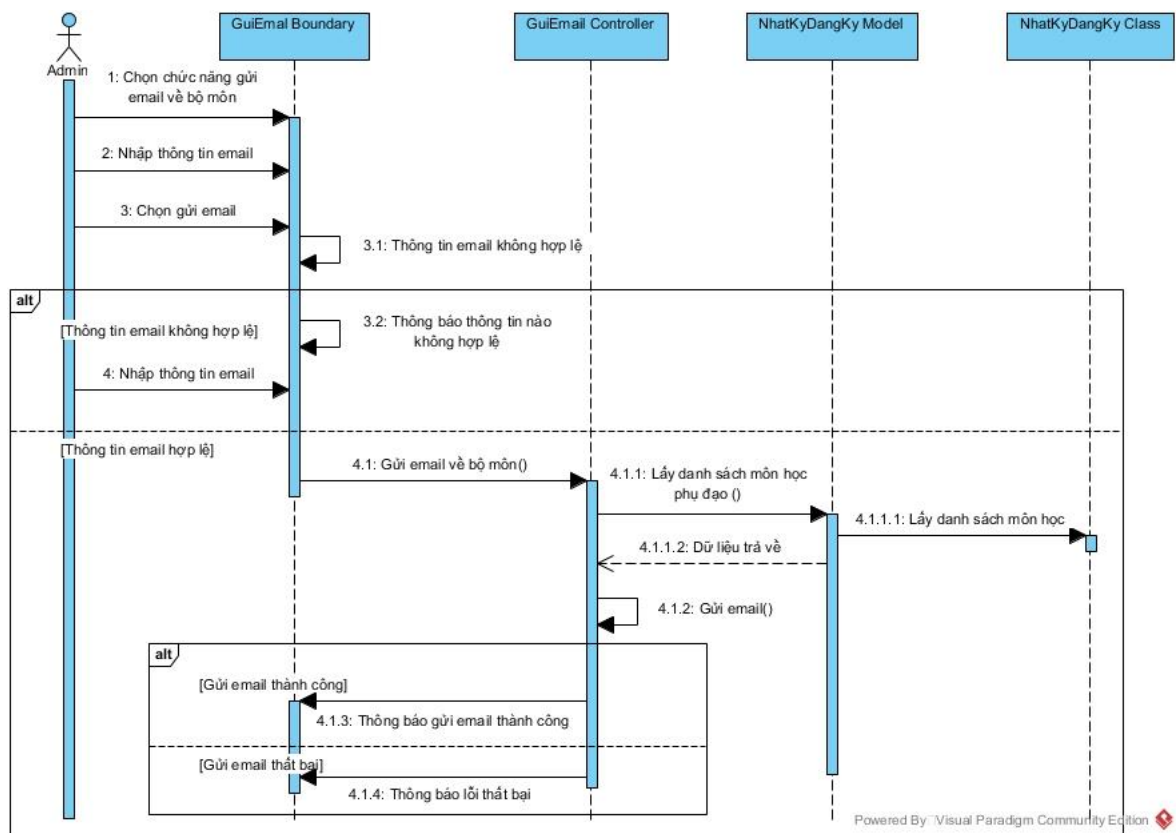
Hình 9: Biểu đồ tuần tự chức năng Thanh toán tiền phụ đạo

2.3.10. Biểu đồ tuần tự chức năng Thống kê đợt phụ đạo



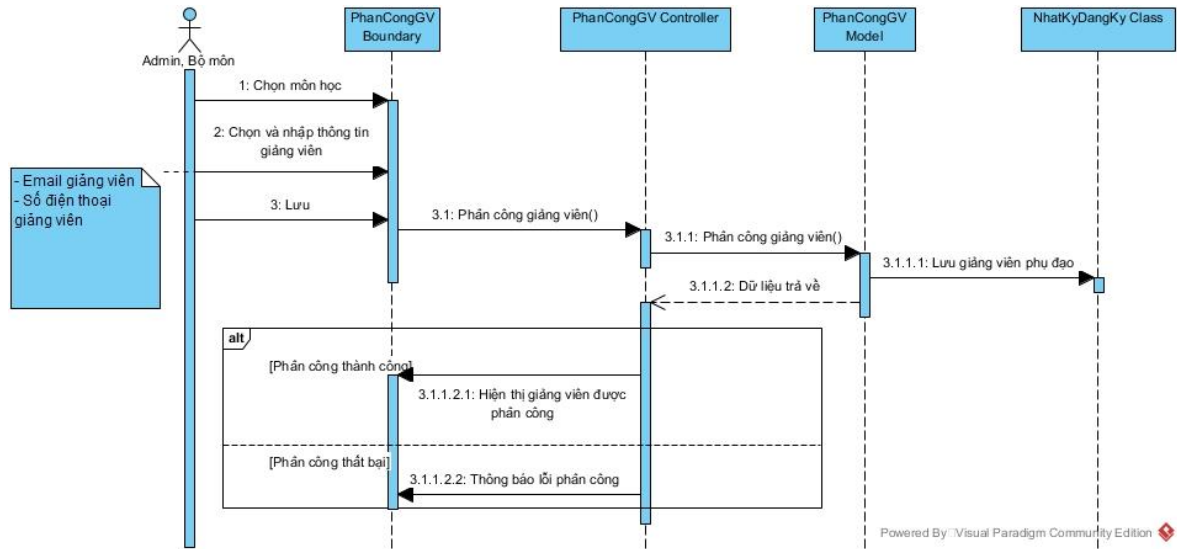
Hình 10: Biểu đồ Biểu đồ tuần tự chức năng Thống kê tiền phụ đạo

2.3.11. Biểu đồ tuần tự chức năng Gửi email về bộ môn



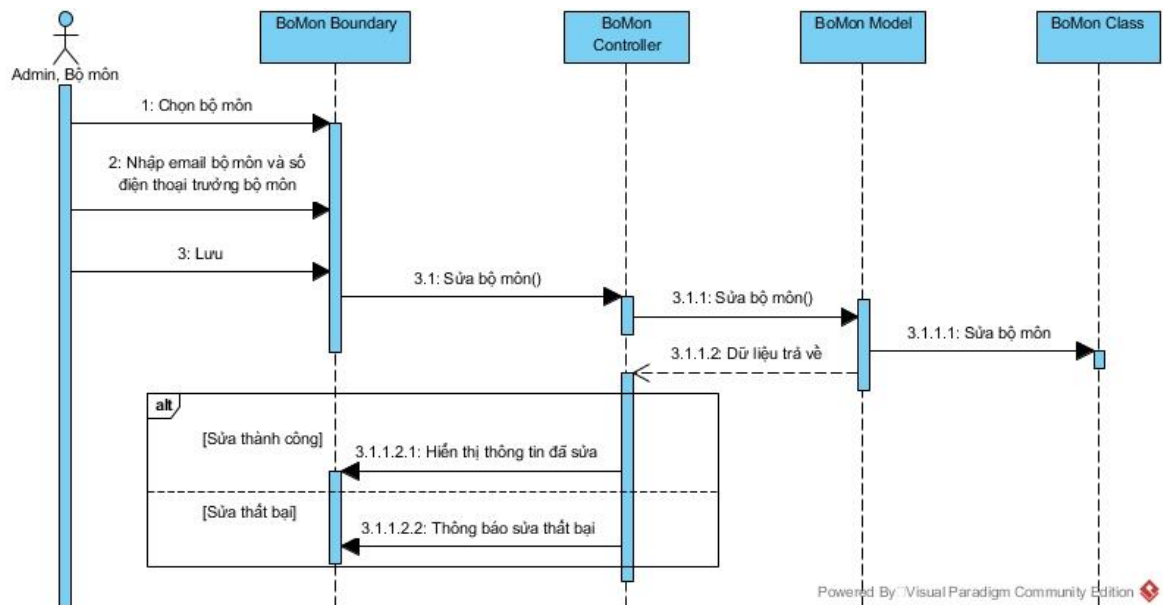
Hình 11: Biểu đồ Biểu đồ tuần tự chức năng Gửi email về bộ môn

2.3.12. Biểu đồ tuần tự chức năng Phân công giảng viên phụ đạo



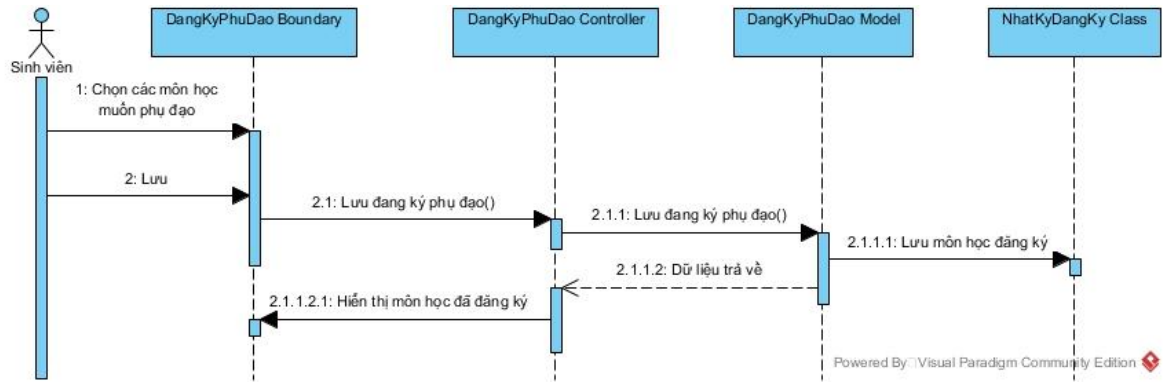
Hình 12: Biểu đồ Biểu đồ tuần tự chức năng Phân công giảng viên phụ đạo

2.3.13. Biểu đồ tuần tự chức năng Sửa bộ môn



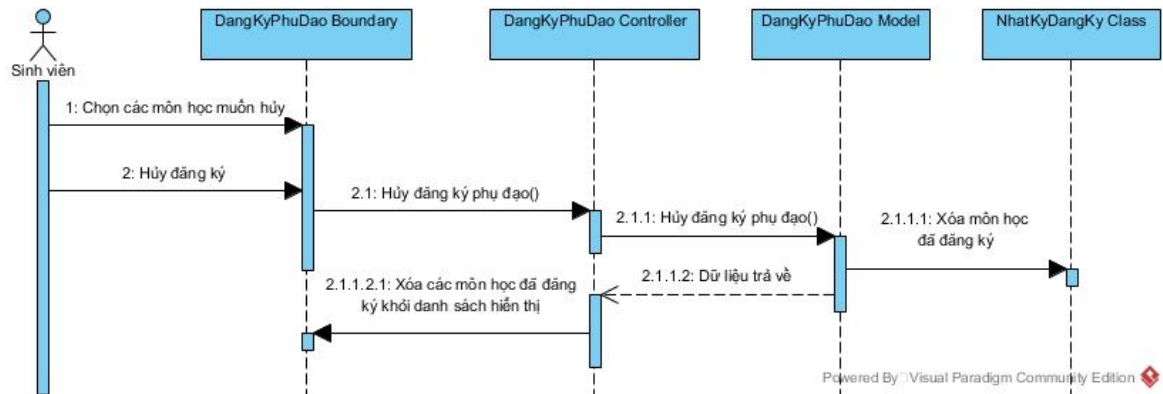
Hình 13: Biểu đồ Biểu đồ tuần tự chức năng Sửa bộ môn

2.3.14. Biểu đồ tuần tự chức năng Đăng ký học phụ đạo



Hình 14: Biểu đồ tuần tự chức năng Đăng ký học phụ đạo

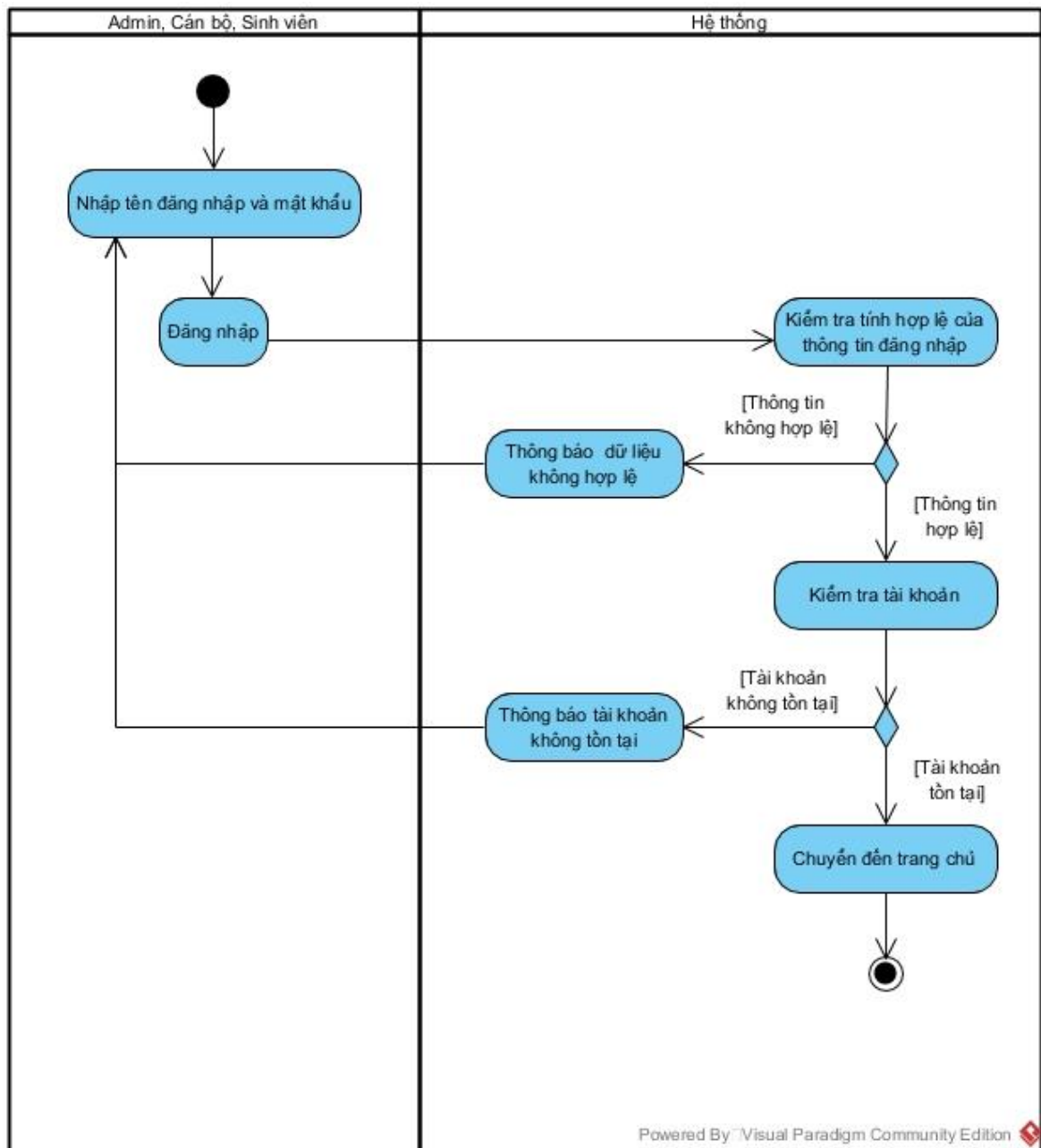
2.3.15. Biểu đồ tuần tự chức năng Hủy đăng ký học phụ đạo



Hình 15: Biểu đồ tuần tự chức năng Hủy đăng ký học phụ đạo

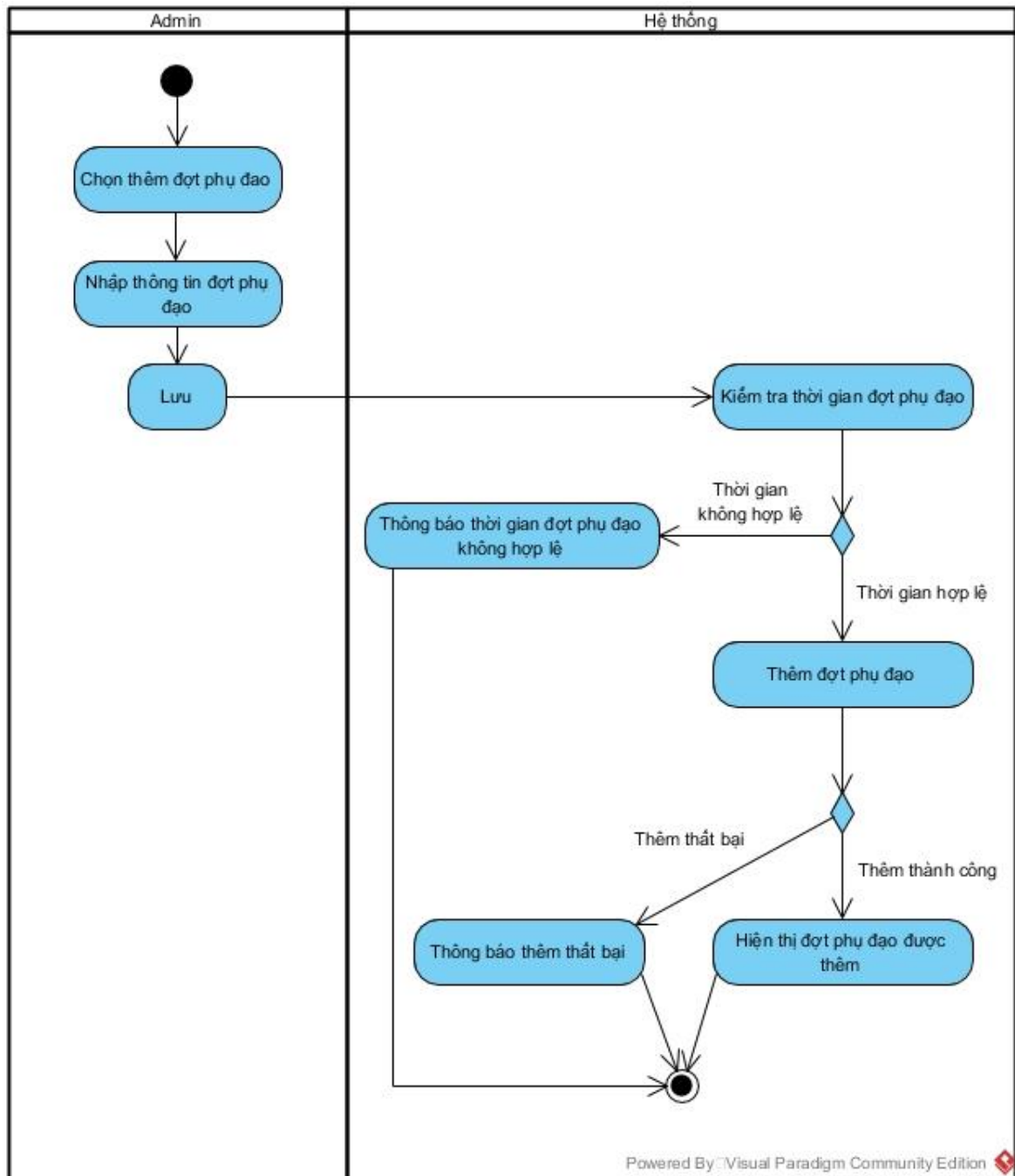
2.4. Biểu đồ Biểu đồ hoạt động chức năng

2.4.1. Biểu đồ hoạt động chức năng Đăng nhập



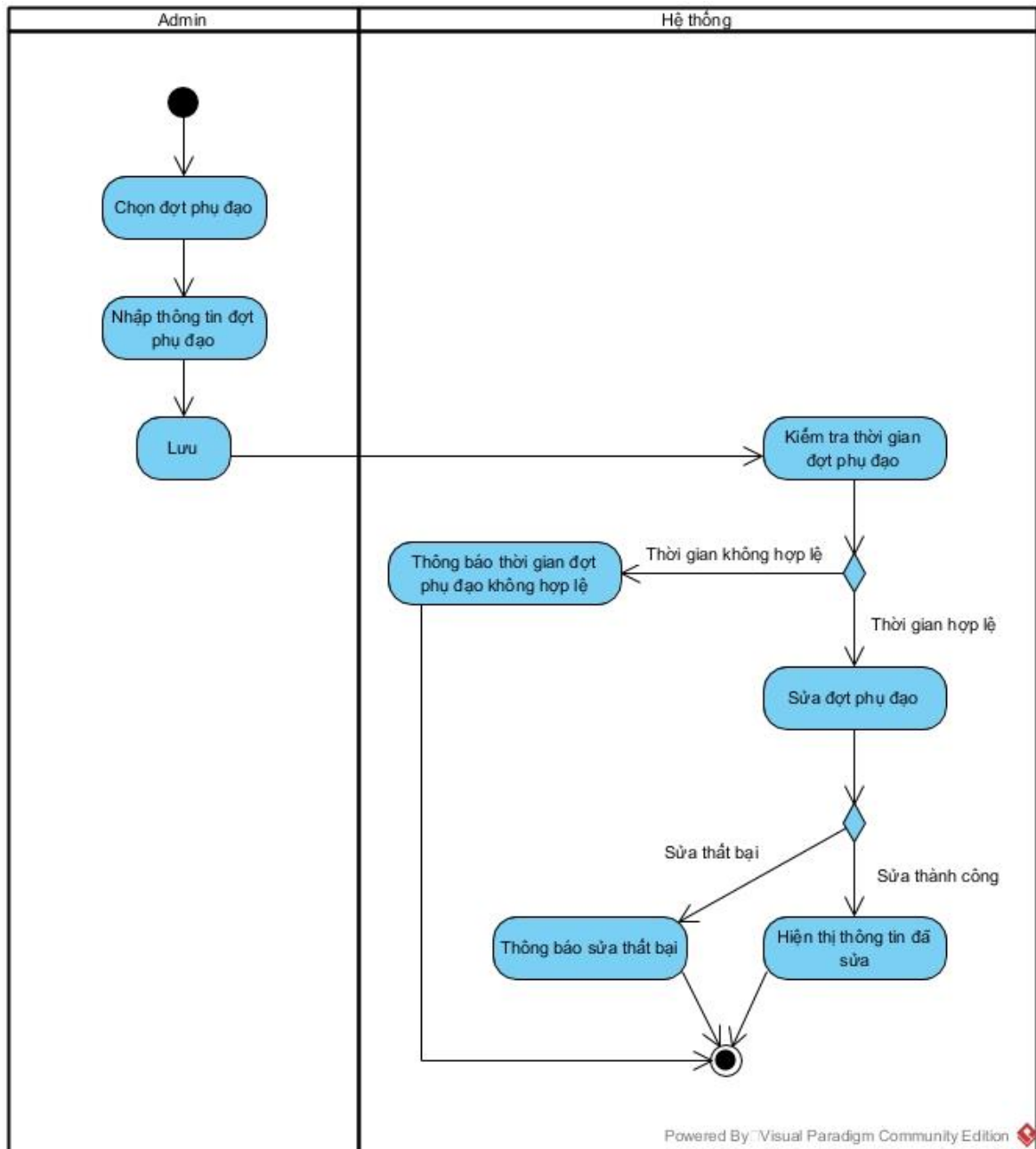
Hình 16: Biểu đồ Biểu đồ hoạt động chức năng Đăng nhập

2.4.2. Biểu đồ hoạt động chức năng Thêm đợt phụ đạo



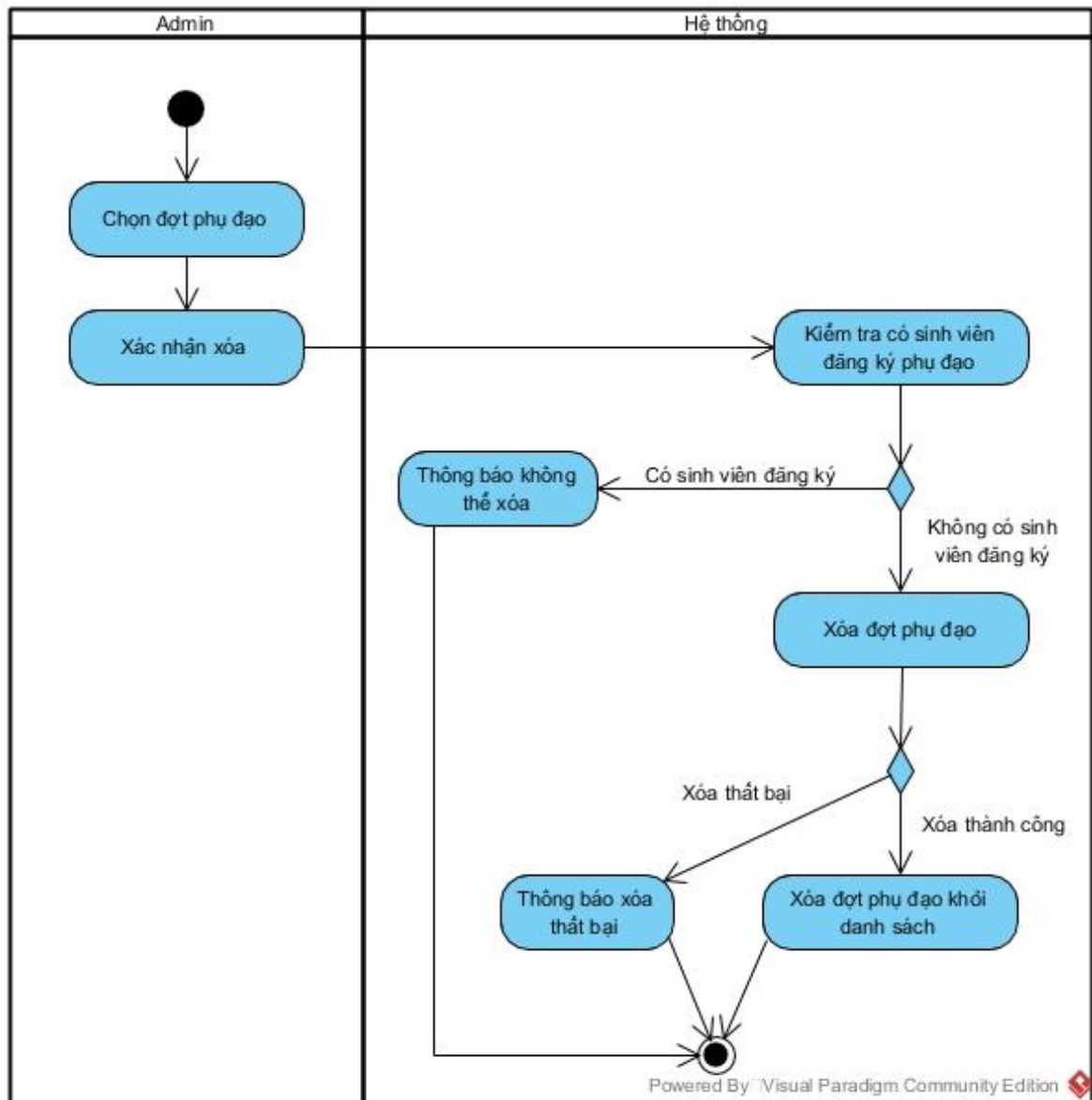
Hình 17: Biểu đồ Biểu đồ hoạt động chức năng Thêm đợt phụ đạo

2.3.3. Biểu đồ hoạt động chức năng Sửa đợt phụ đạo



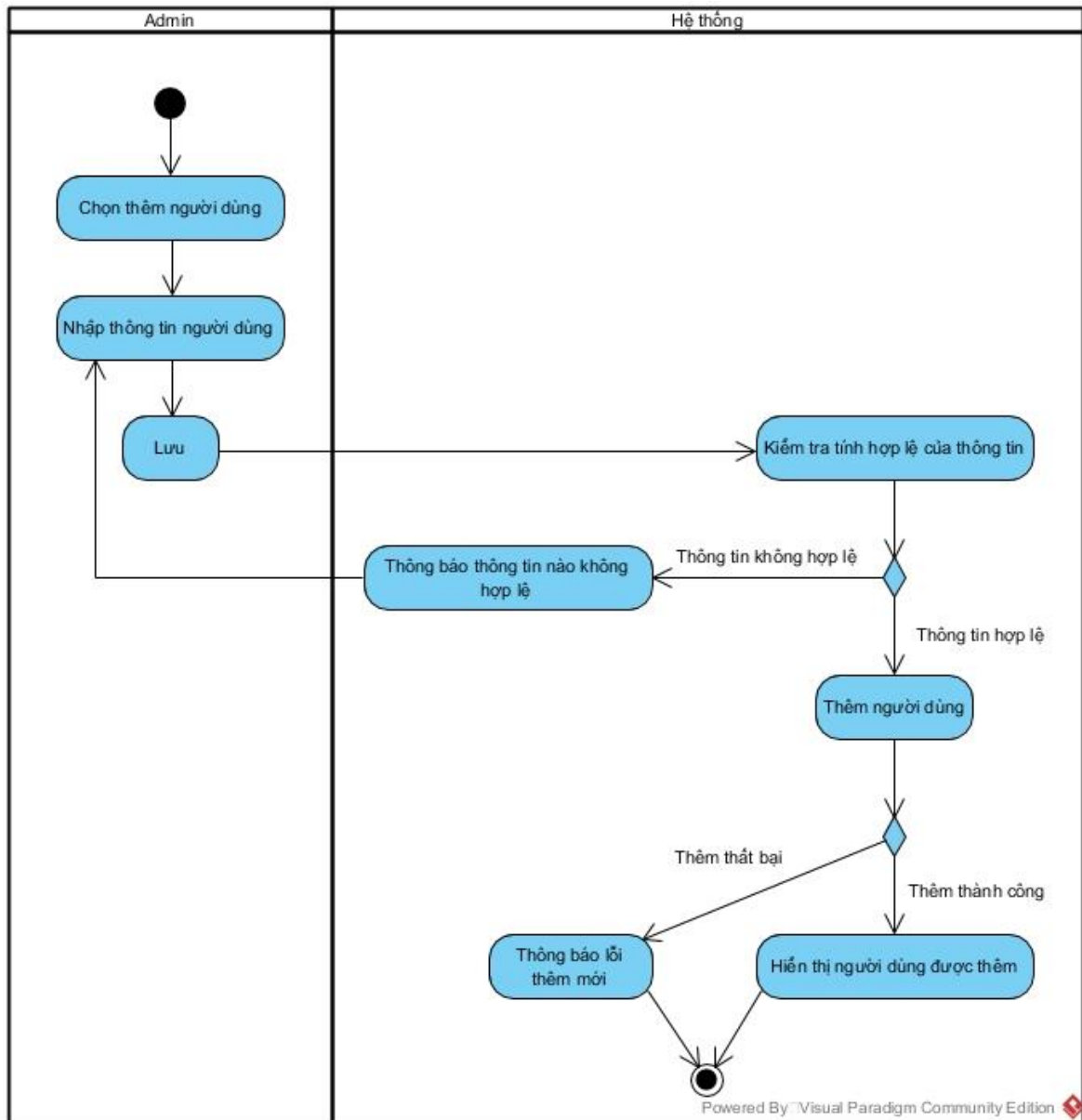
Hình 18: Biểu đồ Biểu đồ hoạt động chức năng Sửa đợt phụ đạo

2.3.4. Biểu đồ hoạt động chức năng Xóa đợt phụ đạo



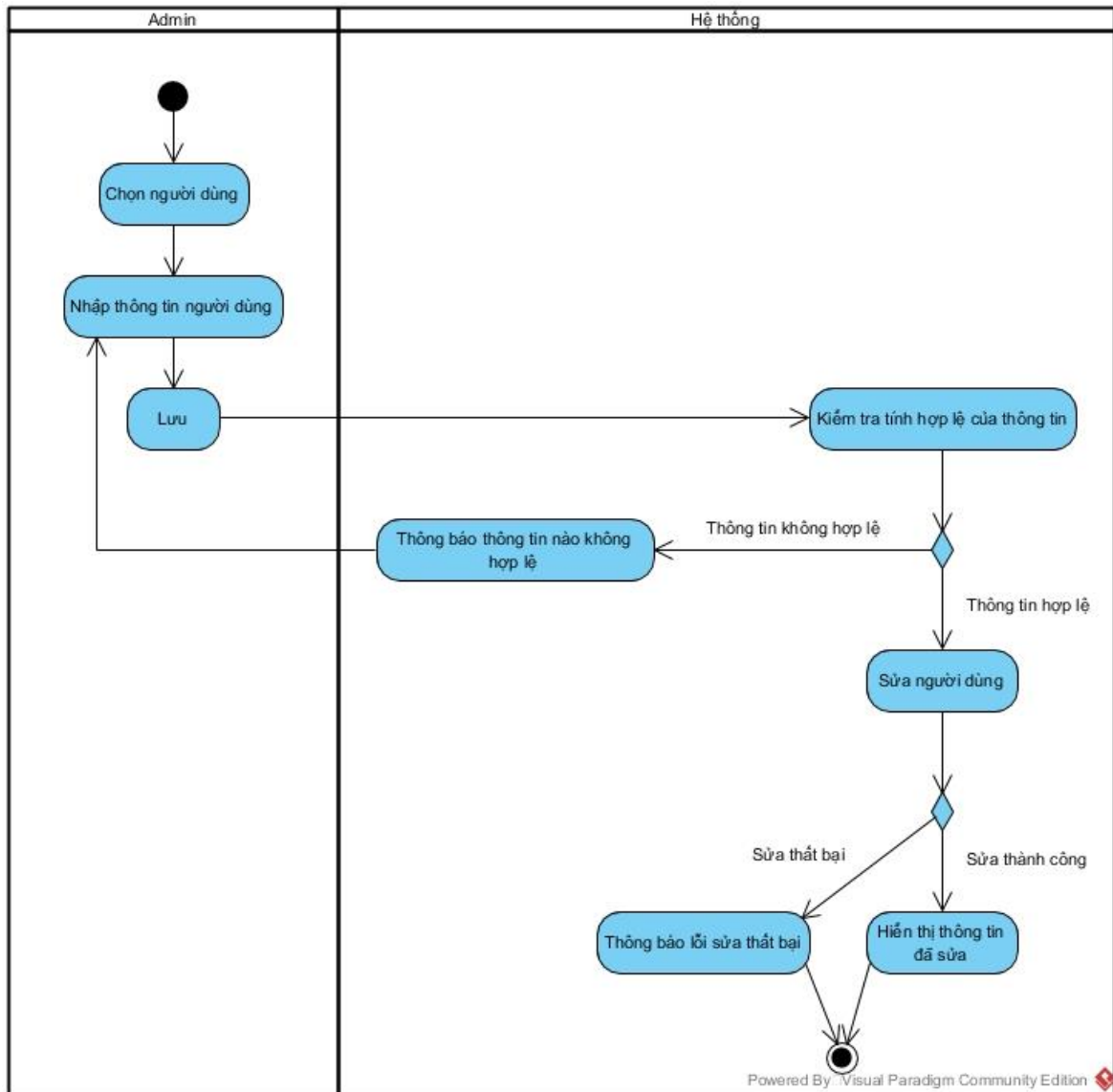
Hình 19: Biểu đồ Biểu đồ hoạt động chức năng Xóa đợt phụ đạo

2.3.5. Biểu đồ hoạt động chức năng Thêm người dùng



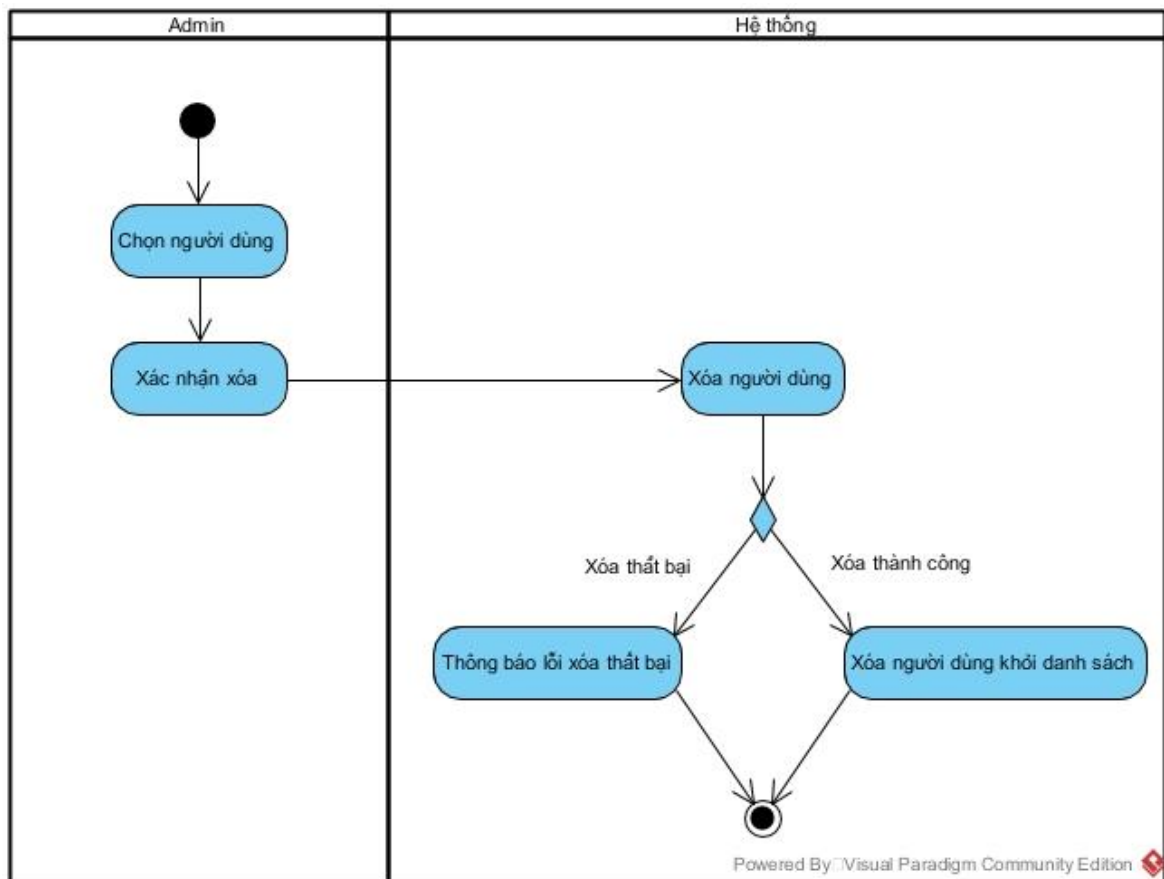
Hình 20: Biểu đồ Biểu đồ hoạt động chức năng Thêm người dùng

2.3.6. Biểu đồ hoạt động chức năng Sửa người dùng



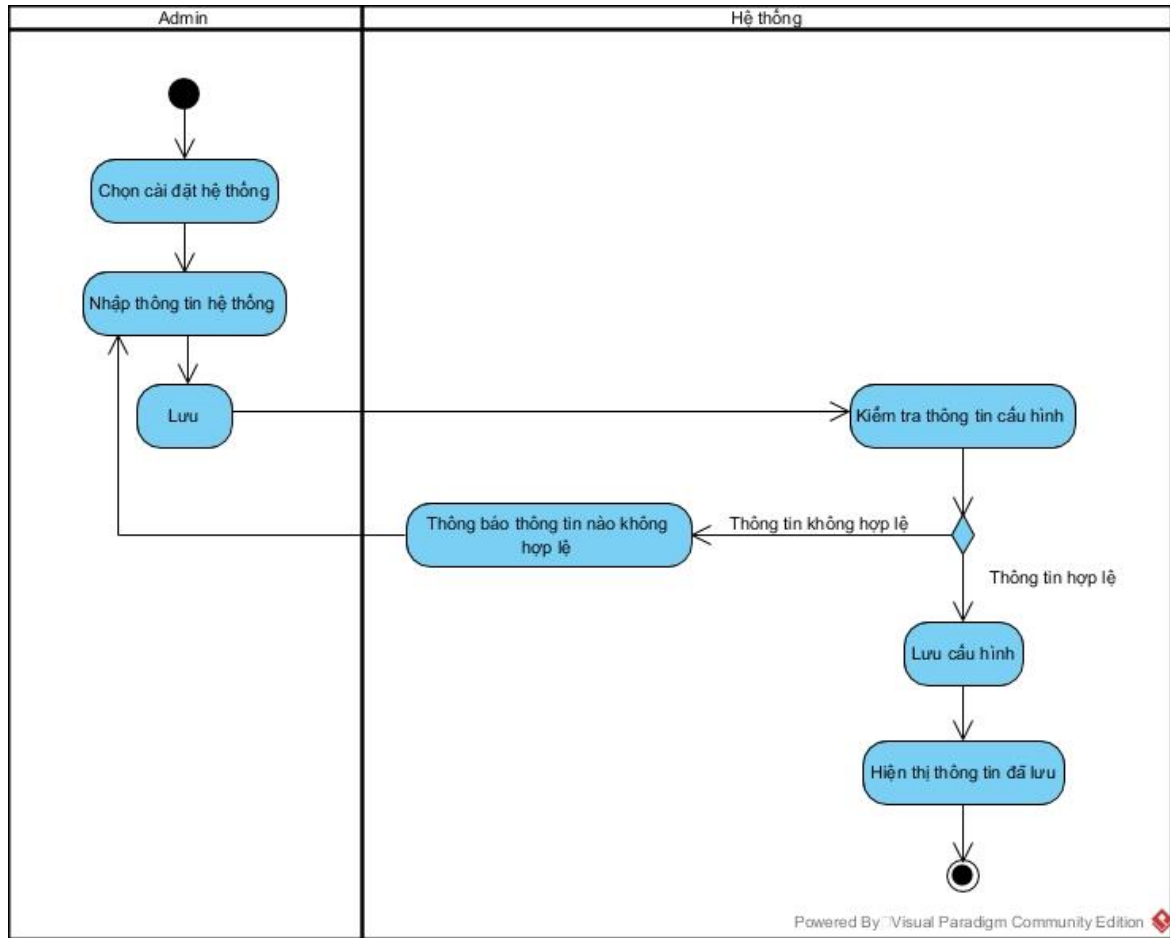
Hình 21: Biểu đồ Biểu đồ hoạt động chức năng Sửa người dùng

2.3.7. Biểu đồ hoạt động chức năng Xóa người dùng



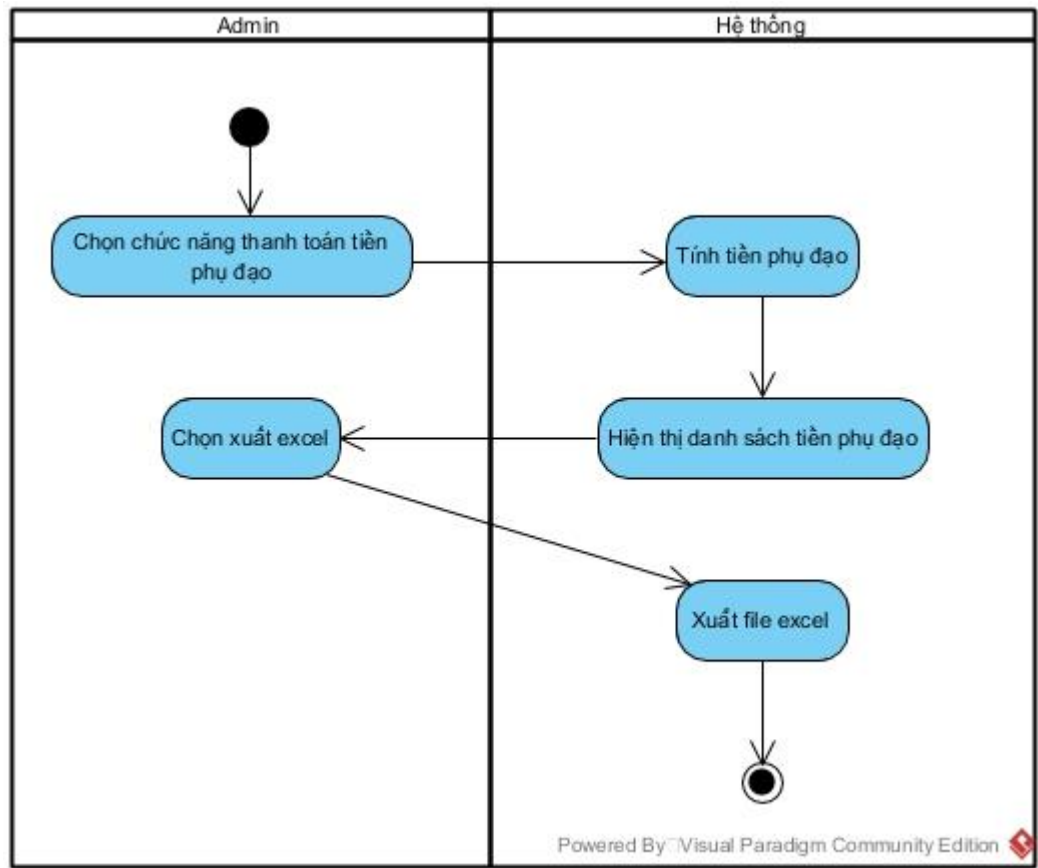
Hình 22: Biểu đồ Biểu đồ hoạt động chức năng Xóa người dùng

2.3.8. Biểu đồ hoạt động chức năng Cài đặt hệ thống



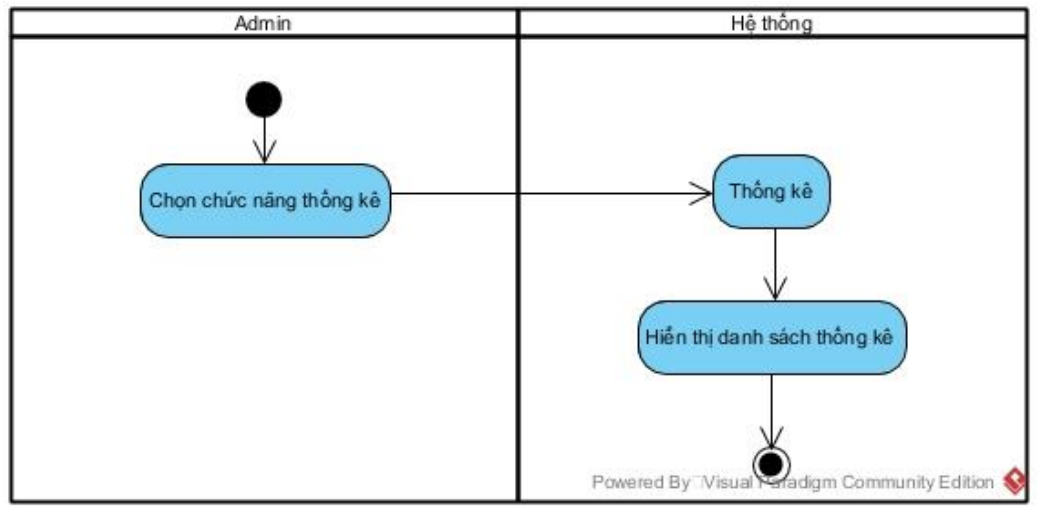
Hình 23: Biểu đồ Biểu đồ hoạt động chức năng Cài đặt hệ thống

2.3.9. Biểu đồ hoạt động chức năng Thanh toán tiền phụ đạo



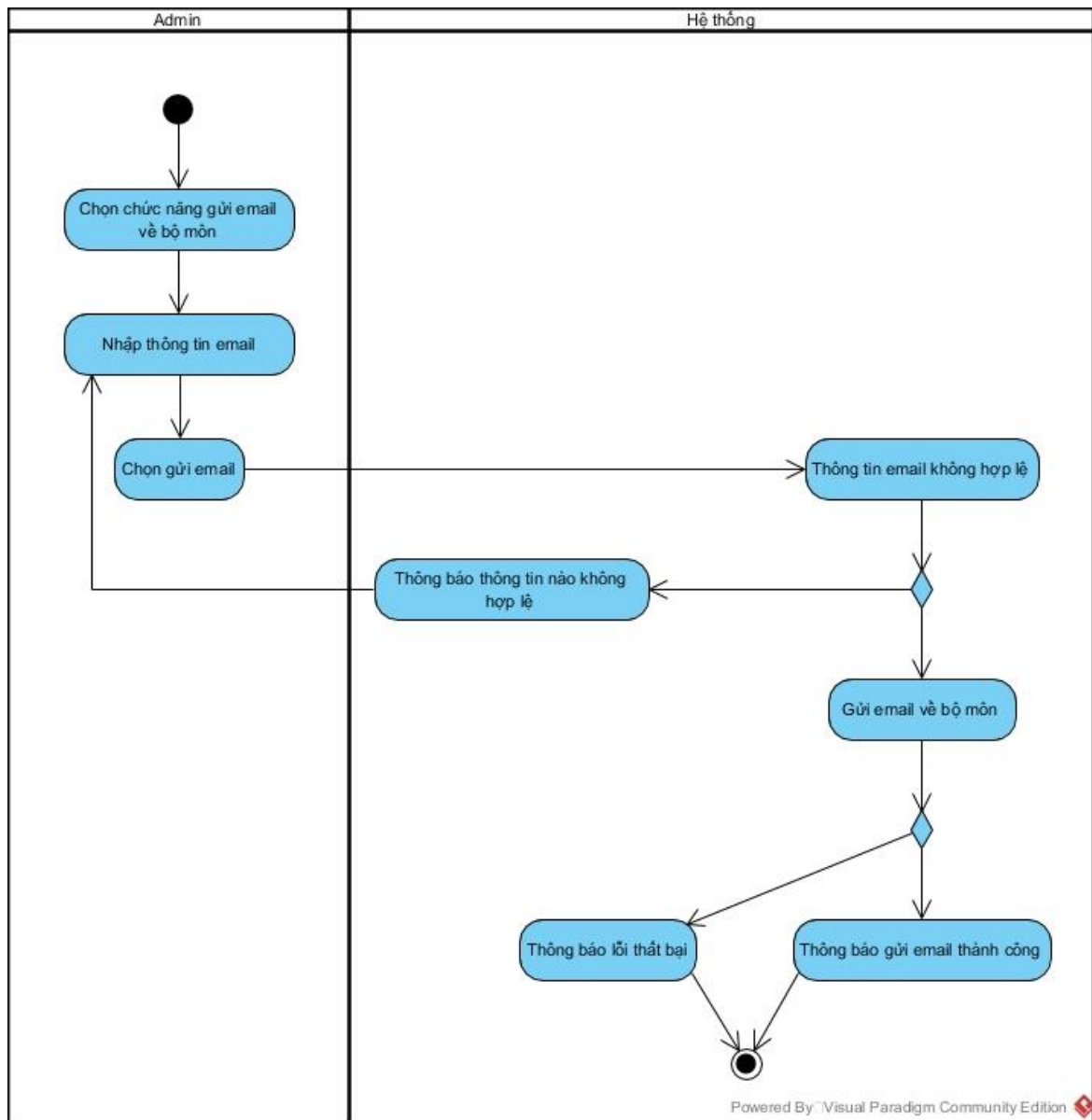
Hình 24: Biểu đồ Biểu đồ hoạt động chức năng Thanh toán tiền phụ đạo

2.3.10. Biểu đồ hoạt động chức năng Thống kê đợt phụ đạo



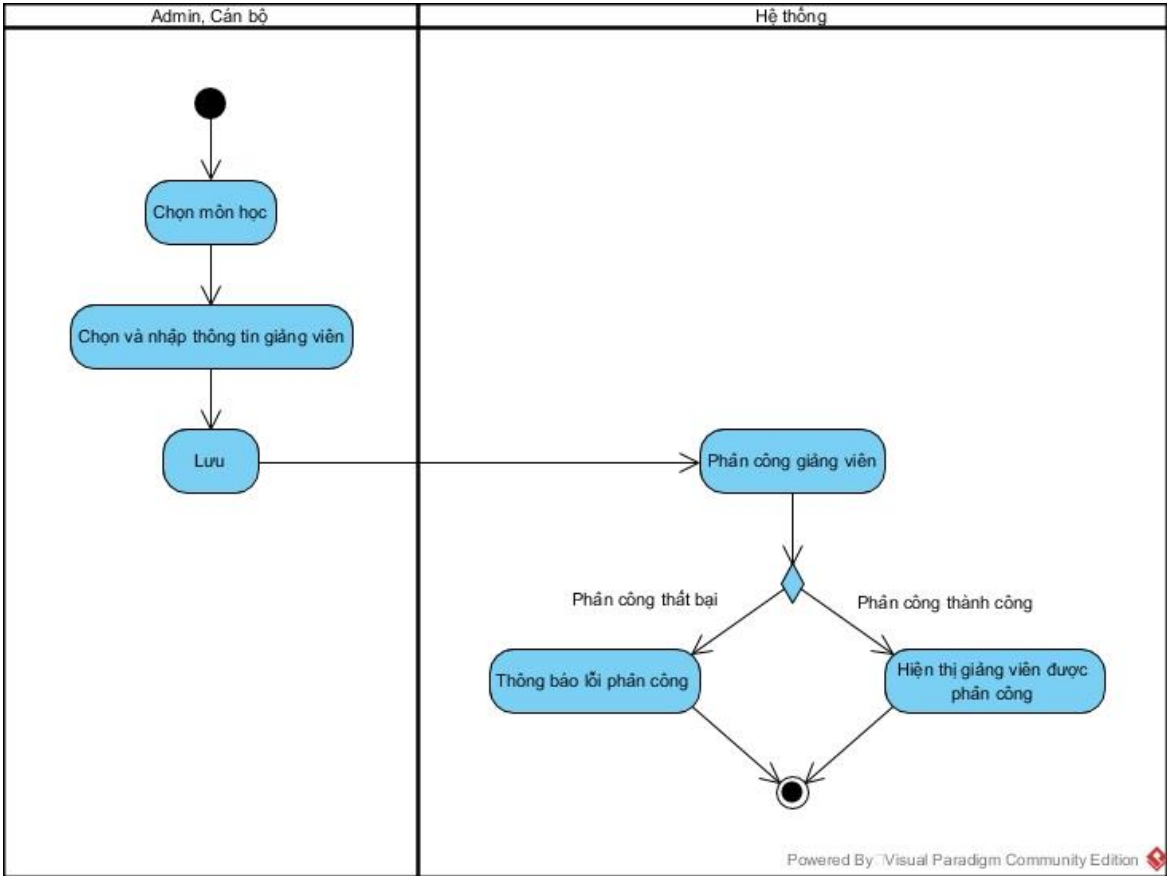
Hình 25: Biểu đồ Biểu đồ hoạt động chức năng Thống kê đợt phụ đạo

2.3.11. Biểu đồ hoạt động chức năng Gửi email về bộ môn



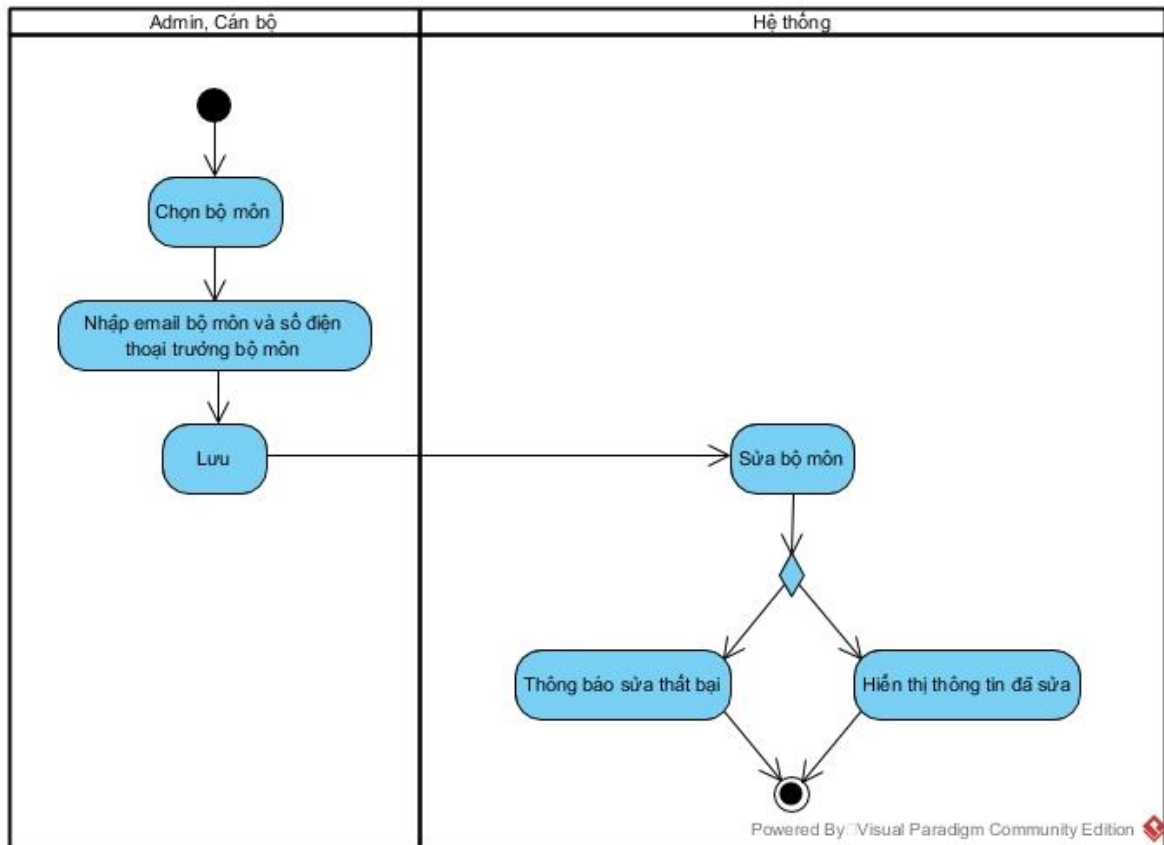
Hình 26: Biểu đồ Biểu đồ hoạt động chức năng Gửi email về bộ môn

2.3.12. Biểu đồ hoạt động chức năng Phân công giảng viên phụ đạo



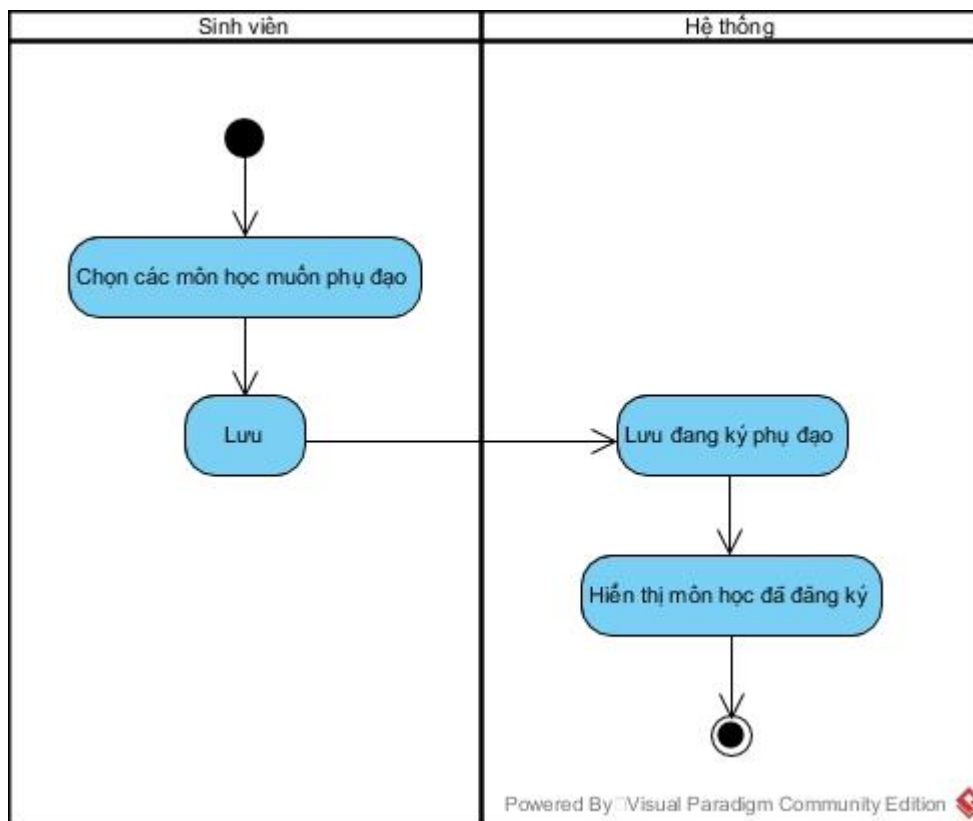
Hình 27: Biểu đồ Biểu đồ hoạt động chức năng Phân công giảng viên phụ đạo

2.3.13. Biểu đồ hoạt động chức năng Sửa bộ môn



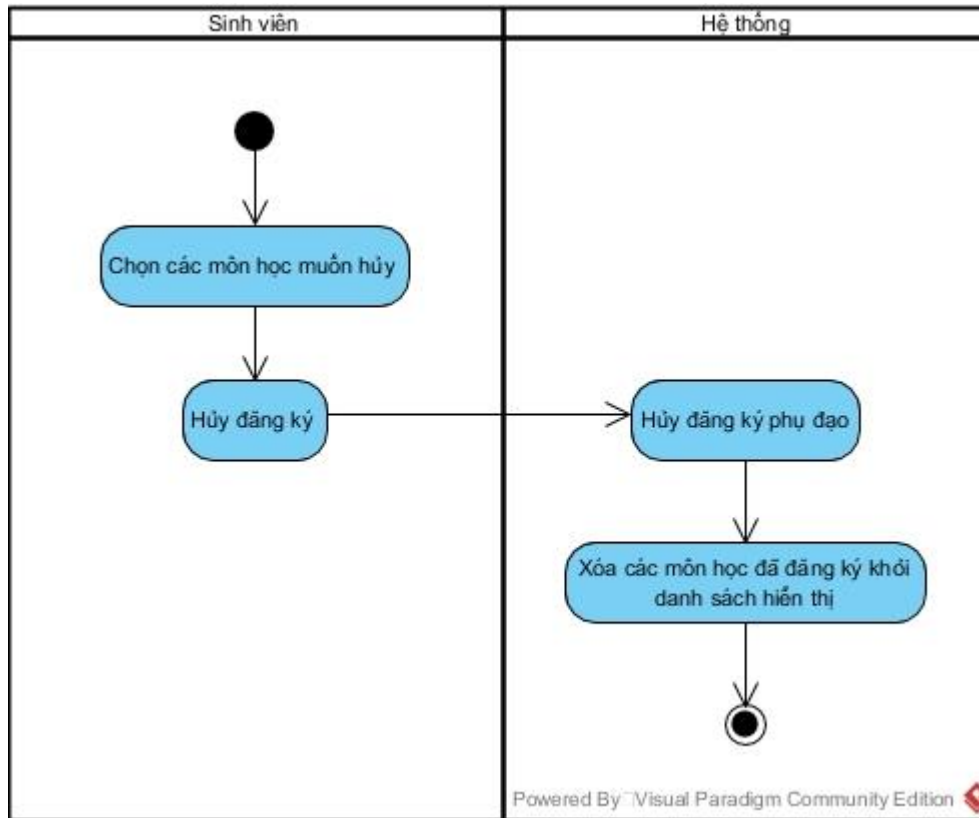
Hình 28: Biểu đồ Biểu đồ hoạt động chức năng Sửa bộ môn

2.3.14. Biểu đồ hoạt động chức năng Đăng ký học phụ đạo



Hình 29: Biểu đồ Biểu đồ hoạt động chức năng Đăng ký phụ đạo

2.3.15. Biểu đồ hoạt động chức năng Hủy đăng ký học phụ đạo



Hình 30: Biểu đồ Biểu đồ hoạt động chức năng Hủy đăng ký phụ đạo

CHƯƠNG III: THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ kiến trúc tổng thể hệ thống:

Phân tích kiến trúc Hexagonal (Ports & Adapters) trong HUCE-ISRS

3.1.1. Khái niệm và lý do áp dụng

Kiến trúc Hexagonal (còn gọi **Ports & Adapters**, Alistair Cockburn) coi **logic nghiệp vụ là lõi** (Domain), đặt ở trung tâm, cô lập khỏi các cơ chế bên ngoài: HTTP, database, email, API hệ thống trường.

Trong HUCE-ISRS (remedial_system), lõi xử lý nghiệp vụ **đăng ký học phần phụ đạo** phải:

- Lưu trữ dữ liệu nội bộ (MySQL qua Eloquent).
- Gọi **University System** để xác thực sinh viên, lấy điểm, môn đăng ký chính quy.
- Phục vụ nhiều **điểm vào** (REST API cho Admin, Bộ môn, Sinh viên).
- Dễ kiểm thử mà không phụ thuộc DB thật hay API trường đang chạy.

Nếu gộp toàn bộ vào Controller + Eloquent (kiểu Laravel truyền thống), mỗi thay đổi ở DB, API trường hay format HTTP đều lan vào logic nghiệp vụ. Hexagonal giải quyết bằng cách:

1. Định nghĩa **Port** (interface) tại Domain — hợp đồng “cần gì”, không quan tâm “làm bằng gì”.
2. Viết **Adapter** tại Infrastructure — implement Port bằng Eloquent, HTTP client, Mail, v.v.
3. **Application Service** (use case) chỉ gọi Port và Entity, không import chi tiết kỹ thuật.

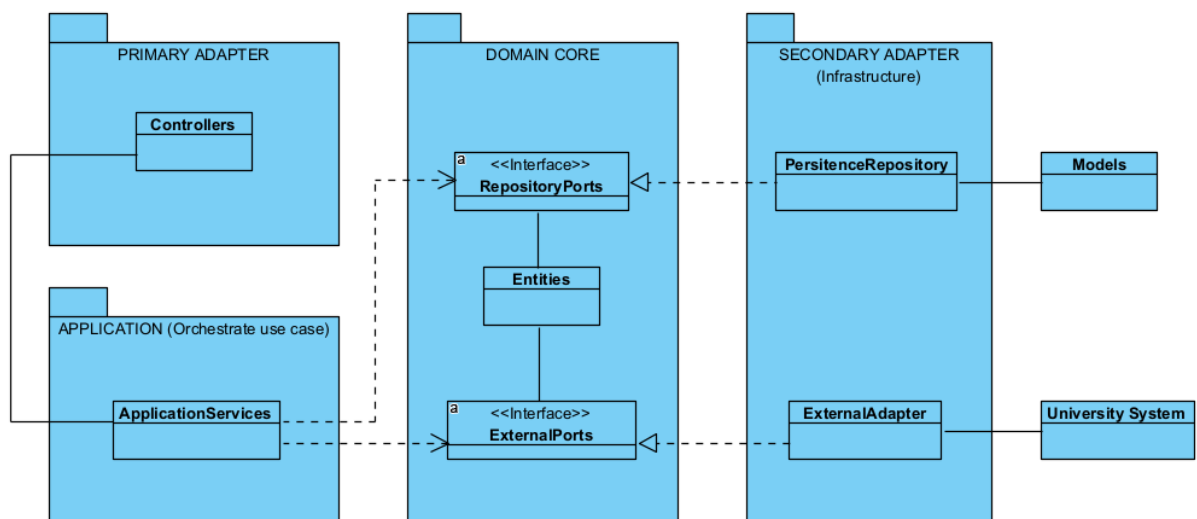
3.1.2. Ánh xạ Hexagonal ↔ cấu trúc mã nguồn

Khái niệm Hexagonal	Thư mục trong remedial_system	Vai trò
Domain (Lõi)	app/Domain/	Entity, Enum, Specification, Exception, Port (interface)

Khái niệm Hexagonal	Thư mục trong remedial_system	Vai trò
Application (Use case)	app/Application/Services/	Điều phối luồng nghiệp vụ, gọi Port, dùng Entity
Primary Adapter (vào)	app/Http/	Controller, FormRequest, Middleware, Resource JSON
Secondary Adapter (ra)	app/Infrastructure/	Repository Eloquent, University API, Mail, CircuitBreaker
Composition Root	app/Providers/RemedialServiceProvider.php	Bind Port → Adapter, đăng ký Service
Chi tiết DB	app/Models/	Eloquent Model — chỉ Infrastructure được dùng

Nguyên tắc phụ thuộc: mũi tên luôn hướng **vào trong** — Http và Infrastructure phụ thuộc Application/Domain; Domain **không** phụ thuộc Laravel, Eloquent hay HTTP.

3.1.3. Sơ đồ kiến trúc tổng thể



3.1.4. Luồng xử lý một request (chuỗi Hexagonal)

Client (React)

↓ HTTP

Route → Middleware (auth:sanctum, role)



Controller ← Primary Adapter

↓ validated()

FormRequest



Application Service ← Use Case



Domain Entity + quy tắc ← Lỗi nghiệp vụ



Port (interface)



Repository / External Adapter ← Secondary Adapter



MySQL / University API



Resource / Presenter → JSON envelope { success, message, data }

Controller **không** chứa logic nghiệp vụ dày; chỉ validate input, gọi Service, map exception → HTTP status, trả BaseController::success().

3.1.5. Phân tích chi tiết từng tầng

3.1.5.1. Domain — Lỗi thuần nghiệp vụ

Entities (app/Domain/Entities/): đối tượng nghiệp vụ, tách khỏi bảng DB.

- RemedialTerm: trạng thái đợt phụ đạo, isRegistrationOpen(), transitionTo() qua State pattern (TermStates/).
- RemedialRegistration: đơn đăng ký phụ đạo.
- StudentInfo, SubjectResult, TermRegisteredCourse: **Anti-Corruption Layer** — biểu diễn dữ liệu từ University theo ngôn ngữ domain, không leak cấu trúc API/SQL Server trường.

Specifications (app/Domain/Specifications/): quy tắc tái sử dụng, ví dụ SubjectEligibleForRemedial kiểm tra môn đủ điều kiện phụ đạo (≥ 2 tín chỉ).

Ports (app/Domain/Ports/): chỉ **interface**, chia hai nhóm:

Nhóm	Ví dụ	Mục đích
Persistence/	RemedialTermRepositoryPort, RemedialRegistrationQueryPort	Hợp đồng đọc/ghi DB
External/	StudentInfoPort	Hợp đồng gọi hệ thống trường

Tách **ghi** (RemedialRegistrationRepositoryPort) và **đọc/báo cáo** (RemedialRegistrationQueryPort) theo hướng CQRS nhẹ — query phức tạp (group theo môn, thống kê) không làm ô nhiễm use case ghi.

Exceptions (app/Domain/Exceptions/): lỗi có ý nghĩa nghiệp vụ (InvalidCredentialsException, ExternalSystemException); Controller map sang 401/404/502.

Domain **không** use Illuminate* hay Eloquent — giữ lỗi portable và testable.

3.1.5.2. Application — Use case điều phối

Mỗi Service trong app/Application/Services/ tương ứng một (hoặc vài) use case:

Service	Use case
ManageRemedialTermService	Admin CRUD đợt phụ đạo, chuyển trạng thái
RemedialRegistrationService	Sinh viên đăng ký/hủy môn phụ đạo
AuthenticateUserService	Đăng nhập staff / sinh viên
StudentProvisioningService	Tạo user SV lần đầu sau xác thực trường
StudentSyncService	Đồng bộ catalog subjects từ University
DepartmentManageRegistrationService	Bộ môn gán giảng viên sau hết hạn đăng ký

Application **orchestrate**: lấy dữ liệu qua Port, gọi method Entity, ném DomainException khi vi phạm quy tắc — **không** viết SQL/Eloquent trực tiếp (trừ vài chỗ còn rò rỉ cần refactor).

3.1.5.3. *Http — Primary Adapter (driving)*

- Controllers/{Admin,Department,Student,Api}/: entry REST, inject Application Service.
- Requests/: validation + authorize theo role.
- Resources/: serialize Entity/array → JSON snake_case.
- Middleware/EnsureUserHasRole: admin | bo_mon | sinh_vien.

Ví dụ RemedialTermController chỉ gọi ManageRemedialTermService::list(), create(), transitionTo() — không dùng RemedialTermModel.

3.1.5.4. *Infrastructure — Secondary Adapter (driven)*

Persistence: Eloquent*Repository implement *RepositoryPort; *Mapper chuyển Eloquent Model ↔ Domain Entity — tránh leak Active Record vào Application.

External/University:

StudentInfoPort

← CachedStudentInfoAdapter (decorator cache TTL ~1h)

← StudentInfoApiAdapter (HTTP)

← UniversityAuthClient (OAuth client credentials)

Khi University lỗi, CircuitBreaker hạn chế cascade; test dùng FakeStudentInfoPort thay adapter thật.

Mail: DepartmentRemedialSummary — adapter gửi email tổng hợp cho bộ môn.

3.1.6. So sánh với kiến trúc Laravel monolith truyền thống

Tiêu chí	Laravel thuần (Controller + Model)	Hexagonal (HUCE-ISRS)
Vị trí logic nghiệp vụ	Model / Controller lẫn lộn	Domain Entity + Application Service
Truy cập DB	Eloquent trực tiếp mọi nơi	Qua Port → Repository → Mapper
Tích hợp API trường	HTTP client rải rác	Một Port StudentInfoPort
Unit test	Khó, cần DB	Mock Port; FakeStudentInfoPort

Tiêu chí	Laravel thuần (Controller + Model)	Hexagonal (HUCE-ISRS)
Đổi framework/D B	Ảnh hưởng rộng	Chủ yếu đổi Infrastructure
Độ phức tạp ban đầu	Thấp	Cao hơn (nhiều lớp, file)

3.1.7. Ưu điểm đạt được trong thực tế dự án

1. **Tách biệt nghiệp vụ phụ đạo khỏi hạ tầng:** Quy tắc cửa đăng ký, chuyển trạng thái đợt, điều kiện môn nằm trong Domain — PHPUnit test được (RemedialTermRegistrationWindowTest, StudentRegistrationTest).
2. **Tích hợp University linh hoạt:** Mock university_system (:8001) hoặc SQL Server EDU_NUCE thật — Application không đổi.
3. **Phân quyền rõ theo bounded context API:** Admin / Department / Student mỗi namespace Controller gọi use case riêng.
4. **Dễ mở rộng:** Thêm export Excel, email, thống kê = adapter hoặc query port mới; lỗi giữ nguyên.

3.2. Thiết kế pattern

3.2.1. State Pattern

Ngữ cảnh

Thực thể RemedialTerm (Đợt phụ đạo) có nhiều trạng thái vòng đời: Draft, Registration Open, Active, Completed, Cancelled. Mỗi trạng thái sẽ quy định các hành vi khác nhau (ví dụ: chỉ có thể hủy đợt nếu chưa hoàn thành).

Mã nguồn TRƯỚC khi áp dụng Pattern (Giả định)

Trước đây, logic được xử lý bằng các câu lệnh if/else hoặc switch/case không lồ bên trong class RemedialTerm.

```
class RemedialTerm
{
    public RemedialTermStatus $status;

    public function openRegistration(): void
    {
        switch ($this->status) {
            case RemedialTermStatus::DRAFT:
```

```

        $this->status = RemedialTermStatus::REGISTRATION_OPEN;
        break;
    case RemedialTermStatus::REGISTRATION_OPEN:
        throw new \Exception("Đã mở đăng ký rồi.");
    case RemedialTermStatus::ACTIVE:
    case RemedialTermStatus::COMPLETED:
    case RemedialTermStatus::CANCELLED:
        throw new \Exception("Không thể mở đăng ký ở trạng thái này.");
    }
}

public function cancel(): void
{
    if ($this->status === RemedialTermStatus::COMPLETED) {
        throw new \Exception("Đợt đã hoàn thành, không thể hủy.");
    }
    $this->status = RemedialTermStatus::CANCELLED;
}

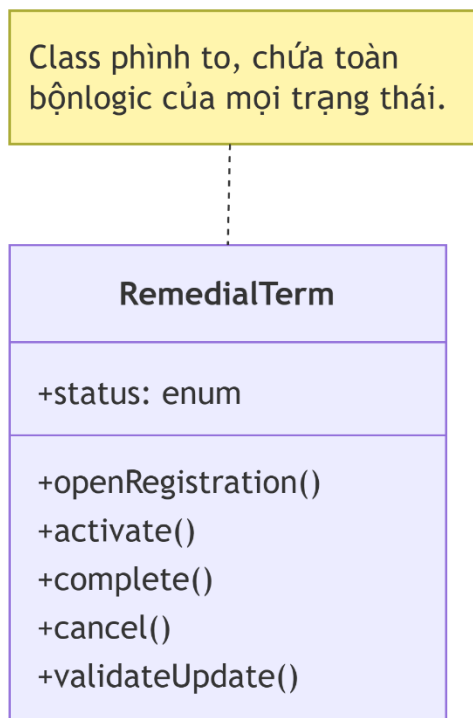
// Các hàm khác tương tự chứa rất nhiều if/switch...
}

```




```
1 class RemedialTerm
2 {
3     public RemedialTermStatus $status;
4
5     public function openRegistration(): void
6     {
7         switch ($this->status) {
8             case RemedialTermStatus::DRAFT:
9                 $this->status = RemedialTermStatus::REGISTRATION_OPEN;
10                break;
11             case RemedialTermStatus::REGISTRATION_OPEN:
12                 throw new \Exception("Đã mở đăng ký rồi.");
13             case RemedialTermStatus::ACTIVE:
14             case RemedialTermStatus::COMPLETED:
15             case RemedialTermStatus::CANCELLED:
16                 throw new \Exception("Không thể mở đăng ký ở trạng thái này.");
17         }
18     }
19
20     public function cancel(): void
21     {
22         if ($this->status === RemedialTermStatus::COMPLETED) {
23             throw new \Exception("Đợt đã hoàn thành, không thể hủy.");
24         }
25         $this->status = RemedialTermStatus::CANCELLED;
26     }
27
28     // Các hàm khác tương tự chứa rất nhiều if/switch...
29 }
30
```

Class Diagram TRƯỚC khi áp dụng



Mã nguồn SAU khi áp dụng Pattern

Được triển khai thực tế tại `app\Domain\Entities\RemedialTerm.php` và thư mục `app\Domain\Entities\TermStates`.

```
// Context Class
class RemedialTerm
{
    public function getState(): \App\Domain\Entities\TermStates\TermState
    {
        return match ($this->status) {
            RemedialTermStatus::DRAFT => new DraftState($this),
            RemedialTermStatus::REGISTRATION_OPEN => new
RegistrationOpenState($this),
            RemedialTermStatus::ACTIVE => new ActiveState($this),
            RemedialTermStatus::COMPLETED => new CompletedState($this),
            RemedialTermStatus::CANCELLED => new CancelledState($this),
        };
    }

    public function openRegistration(): void
    {
        $this->getState()->openRegistration();
    }
}
```

```

    public function cancel(): void
    {
        $this->getState()->cancel();
    }
}

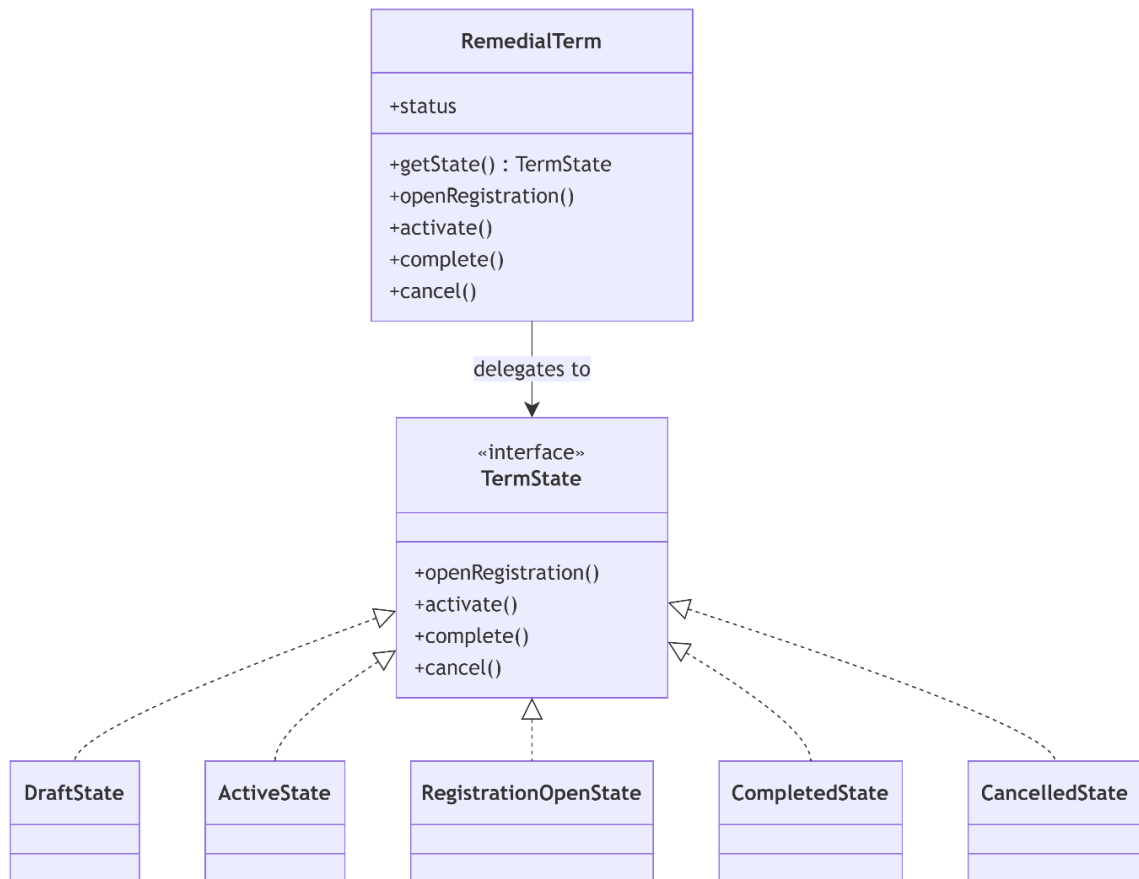
// State Interface
interface TermState
{
    public function openRegistration(): void;
    public function cancel(): void;
    // ...
}

// Concrete State (Ví dụ trạng thái Draft)
class DraftState extends BaseTermState
{
    public function openRegistration(): void
    {
        // Xử lý chuyển đổi hợp lệ từ Draft -> Registration Open
        $this->term->transitionTo(RemedialTermStatus::REGISTRATION_OPEN);
    }

    // ...
}

```

Class Diagram SAU khi áp dụng



So sánh (Đánh giá)

- **Ưu điểm:** Loại bỏ các khối switch/case phức tạp. Tuân thủ nguyên lý OCP (Open/Closed Principle) - khi thêm một trạng thái mới chỉ cần tạo class mới mà không cần sửa đổi class **RemedialTerm**. Logic của từng trạng thái được cô lập hoàn toàn (Single Responsibility Principle).
- **Nhược điểm:** Tăng số lượng class trong project.

3.2.2. Adapter Pattern

Ngữ cảnh

Hệ thống cần lấy dữ liệu sinh viên từ API nội bộ của nhà trường. Để tránh hệ thống phụ thuộc chặt chẽ (tight-coupling) vào API trường, ta sử dụng Adapter pattern bọc API bên ngoài để map sang Model của Domain.

Mã nguồn TRƯỚC khi áp dụng Pattern (Giả định)

Service gọi trực tiếp HTTP Client và gắn chặt với định dạng JSON của API trường.

```
class RemedialRegistrationService
{
    public function registerStudent(string $studentCode)
```

```

{
    // Phụ thuộc chặt vào Http facade và URL của trường
    $response = Http::get("https://university.edu.vn/api/students/{ $studentCode }");

    if ($response->failed()) {
        throw new \Exception("Lỗi API trường");
    }

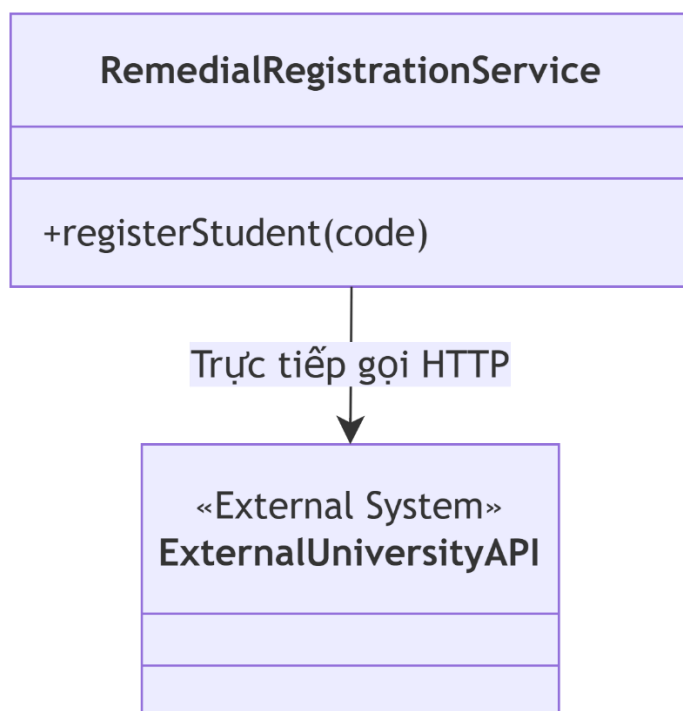
    $data = $response->json();

    // Cấu trúc JSON API trường có thể bị thay đổi làm vỡ code ở đây
    $studentName = $data['full_name_api_format'];

    // Thực hiện logic nghiệp vụ...
}
}

```

Class Diagram TRƯỚC khi áp dụng



Mã nguồn SAU khi áp dụng Pattern

Được triển khai qua StudentInfoPort (Interface/Port) và StudentInfoApiAdapter (Adapter).

```

// Port (Target Interface trong Domain)
interface StudentInfoPort
{
    public function getStudent(string $studentCode): StudentInfo;
}

```

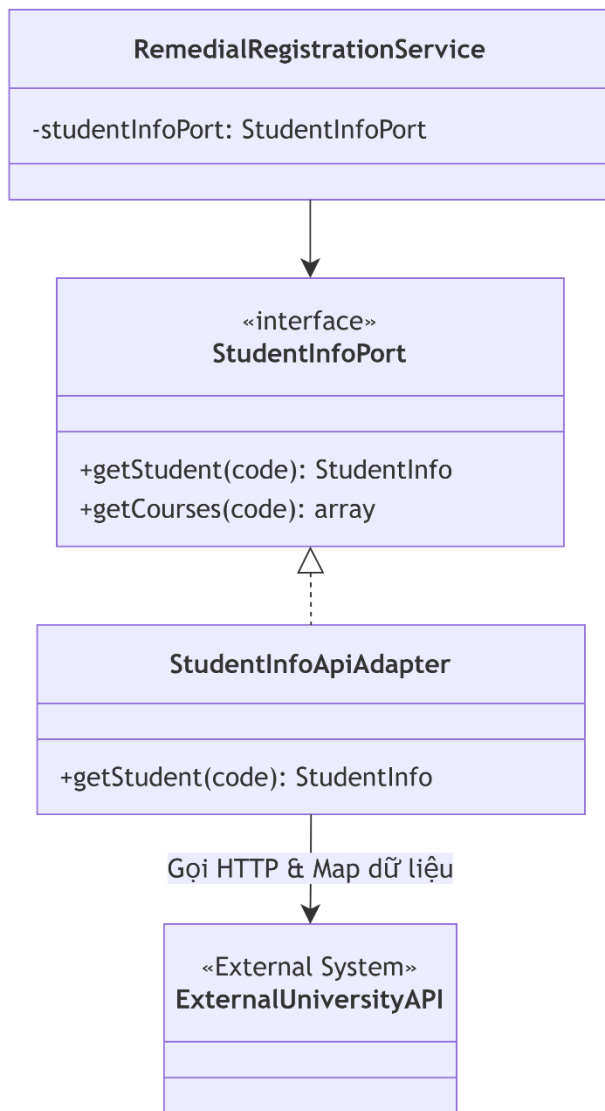
```
// Adapter
class StudentInfoApiAdapter implements StudentInfoPort
{
    public function getStudent(string $studentCode): StudentInfo
    {
        $response = $this->callApi("GET", "/api/students/{$studentCode}");
        $data = $response['data'];

        // Map data trả về từ trường sang Domain Entity
        return new StudentInfo(
            id: $data['id'],
            fullName: $data['fullName'],
            // ...
        );
    }
}

// Sử dụng tại Service (Chỉ phụ thuộc vào Port)
class RemedialRegistrationService
{
    public function __construct(private StudentInfoPort $studentInfoPort) {}

    public function registerStudent(string $studentCode)
    {
        $student = $this->studentInfoPort->getStudent($studentCode);
        // Logic nghiệp vụ an toàn với $student Entity
    }
}
```

Class Diagram SAU khi áp dụng



So sánh (Đánh giá)

- **Ưu điểm:** Core domain (Business Logic) không bị phụ thuộc vào External Services. Khi API của trường thay đổi URL hoặc định dạng JSON, ta chỉ cần sửa ở StudentInfoApiAdapter mà không ảnh hưởng tới các Service nghiệp vụ. Hệ thống dễ dàng mock để viết Unit Test.
- **Nhược điểm:** Tạo thêm một tầng trừu tượng (Interface & Adapter class).

3.2.3. Decorator Pattern

Ngữ cảnh

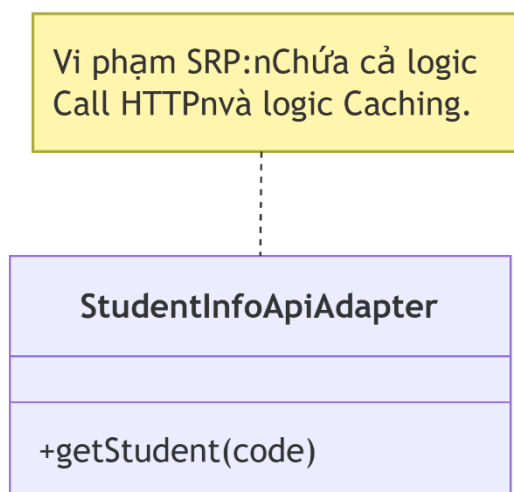
Việc lấy thông tin sinh viên từ API trường (thông qua Adapter) khá chậm. Ta muốn thêm tính năng bộ nhớ đệm (Cache) cho quá trình này nhưng KHÔNG MUỐN sửa đổi class StudentInfoApiAdapter hiện có, để đảm bảo Single Responsibility.

Mã nguồn TRƯỚC khi áp dụng Pattern (Giả định)

Logic gọi Cache và gọi API bị nhồi nhét chung trong cùng một class Adapter.

```
class StudentInfoApiAdapter implements StudentInfoPort
{
    public function getStudent(string $studentCode): StudentInfo
    {
        // Chứa cả logic Cache và logic gọi API
        return Cache::remember("student_info:{$studentCode}", 3600, function() use
($studentCode) {
            $response = $this->callApi("GET", "/api/students/{$studentCode}");
            return new StudentInfo($response['data']['fullName'], /* ... */);
        });
    }
}
```

Class Diagram TRƯỚC khi áp dụng



Mã nguồn SAU khi áp dụng Pattern

Được triển khai qua CachedStudentInfoAdapter bao bọc bên ngoài StudentInfoApiAdapter.

```
// Decorator
class CachedStudentInfoAdapter implements StudentInfoPort
{
    public function __construct(
        private readonly StudentInfoPort $innerAdapter,
        private readonly int $ttlSeconds = 3600
    ) {}

    public function getStudent(string $studentCode): StudentInfo
    {
```



```

return Cache::remember(
    "student_info:{$studentCode}",
    $this->ttlSeconds,
    fn() => $this->innerAdapter->getStudent($studentCode)
);
}

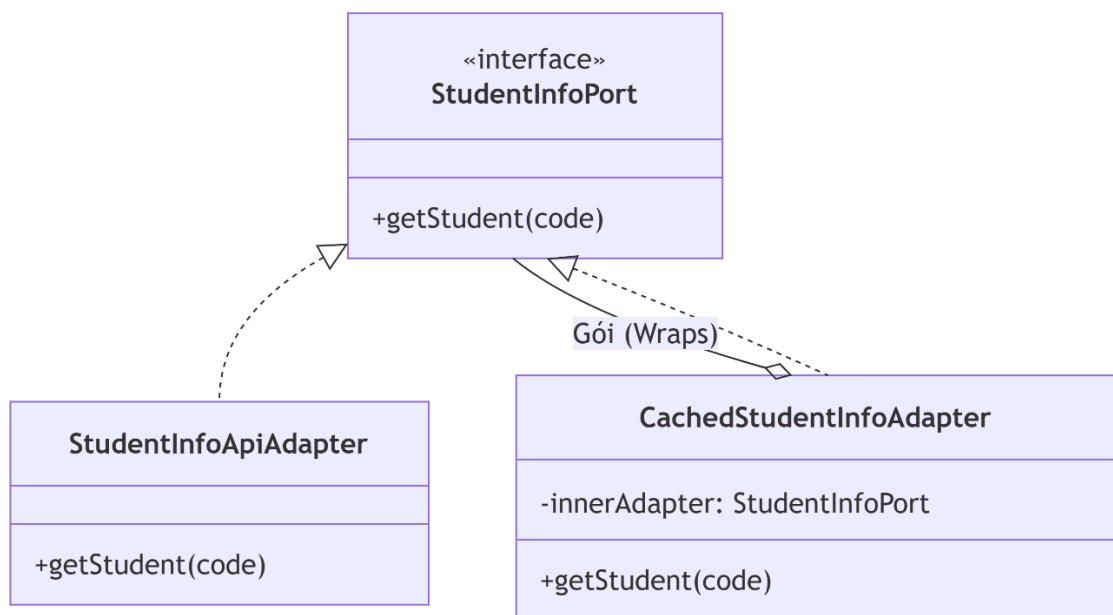
// Các hàm khác ủy quyền tương tự...
}

// Cách khởi tạo (Trong Service Provider)
$baseAdapter = new StudentInfoApiAdapter(/* cấu hình */);
$cachedAdapter = new CachedStudentInfoAdapter($baseAdapter);

$this->app->bind(StudentInfoPort::class, fn() => $cachedAdapter);

```

Class Diagram SAU khi áp dụng



So sánh (Đánh giá)

- **Ưu điểm:** Tuân thủ Single Responsibility Principle. `StudentInfoApiAdapter` chỉ chuyên gọi API. `CachedStudentInfoAdapter` chỉ chuyên xử lý Cache. Ta có thể dễ dàng bật/tắt bộ đệm hoặc thêm các lớp decorator khác (ví dụ: Logging, Retry) bằng cách lồng các class vào nhau.
- **Nhược điểm:** Sinh ra nhiều đối tượng nhỏ lồng nhau, đôi khi gây khó khăn khi debug tracing.

- Trong hệ thống quản lý lớp học phụ đạo, khi giảng viên được phân công giảng dạy cho một lớp hoặc môn học, hệ thống cần gửi thông báo đến giảng viên đó thông qua nhiều kênh như Email, SMS.
- Ban đầu, việc gửi thông báo thường được xử lý trực tiếp trong lớp service nghiệp vụ. Tuy nhiên, cách làm này gây ra nhiều vấn đề về maintain và mở rộng.
- Để giải quyết, ta áp dụng **Observer Pattern** nhằm tách biệt logic nghiệp vụ và logic gửi thông báo.

3.2.4. Observer Pattern

Ngữ cảnh

Trong hệ thống quản lý lớp học phụ đạo, khi giảng viên được phân công giảng dạy cho một lớp hoặc môn học, hệ thống cần gửi thông báo đến giảng viên đó thông qua nhiều kênh như Email, SMS.

Ban đầu, việc gửi thông báo thường được xử lý trực tiếp trong lớp service nghiệp vụ. Tuy nhiên, cách làm này gây ra nhiều vấn đề về maintain và mở rộng.

Để giải quyết, ta áp dụng **Observer Pattern** nhằm tách biệt logic nghiệp vụ và logic gửi thông báo.

Mã nguồn TRƯỚC khi áp dụng Pattern (Giả định)

Service làm quá nhiều việc, Hard-code notification, không mở rộng được.

```
class LecturerAssignmentService {
    public function assignLecturer(array $data):

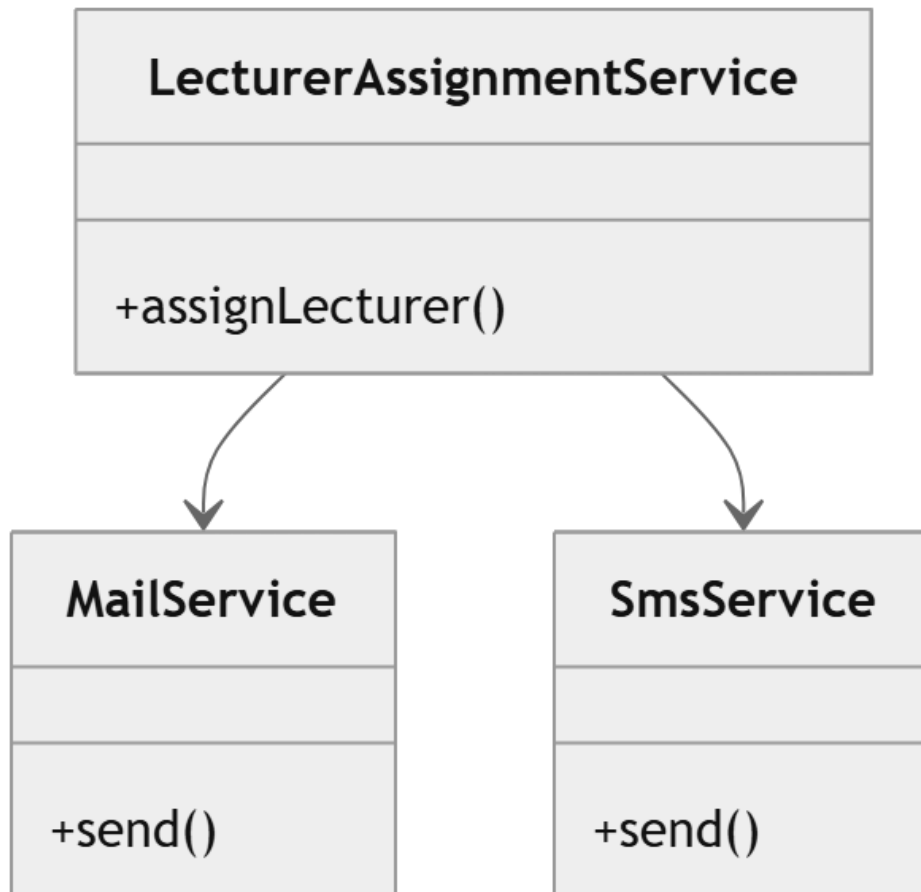
    void {
        // 1. Xử lý nghiệp vụ
        $lecturerEmail = $data['email'];

        // 2. Gửi Email
        Mail::to ($lecturerEmail) -> send(new LecturerAssignmentNotification(...));

        // 3. Gửi SMS
        Sms::send ($data['phone'], "Bạn đã được phân công");

        // 4. Logging
        Log::info ("Assigned lecturer");
    }
}
```

Class Diagram TRƯỚC khi áp dụng



Mã nguồn SAU khi áp dụng Pattern

-- Event

```
class LecturerAssignedToSubject
{
    public function __construct(
        public int $subjectId,
        public int $departmentId,
        public string $lecturerEmail,
        public ?string $lecturerName,
        public ?string $lecturerPhoneNumber,
        public int $updatedCount,
        public ?string $assignedBy,
    ) {}
}
```

```

-- Observer Interface
interface LecturerAssignmentObserver
{
    public function handle(LecturerAssignedToSubject $event): void;
}

-- Dispatcher (Publisher)
class LecturerAssignmentNotificationDispatcher
{
    public function __construct(
        private readonly iterable $observers,
    ) {}

    public function handle(LecturerAssignedToSubject $event): void
    {
        foreach ($this->observers as $observer) {
            try {
                $observer->handle($event);
            } catch (\Throwable $e) {
                Log::error('Observer failed', [
                    'observer' => $observer::class,
                    'error' => $e->getMessage(),
                ]);
            }
        }
    }
}

-- Concrete Observer - Email

```

```

class SendLecturerAssignmentEmail implements LecturerAssignmentObserver
{
    public function handle(LecturerAssignedToSubject $event): void
    {
        Mail::to($event->lecturerEmail)
            ->send(new LecturerAssignmentNotification(...));
    }
}

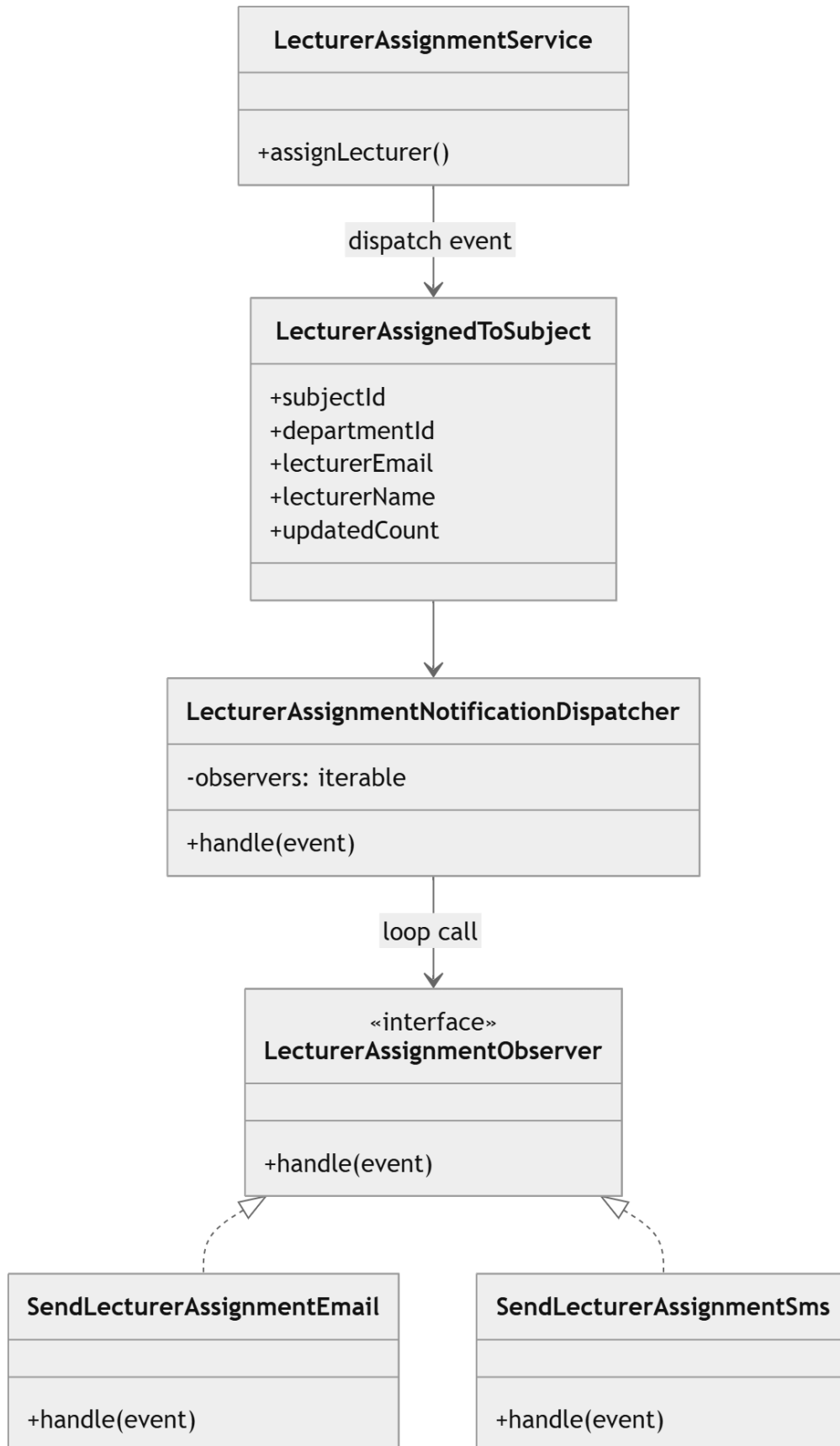
-- Concrete Observer - SMS
class SendLecturerAssignmentSms implements LecturerAssignmentObserver
{
    public function handle(LecturerAssignedToSubject $event): void
    {
        Sms::send($event->lecturerPhoneNumber, "Bạn đã được phân công");
    }
}

class LecturerAssignmentService
{
    public function assignLecturer(...): void
    {
        // business logic

        event(new LecturerAssignedToSubject(...));
    }
}

```

Class Diagram SAU khi áp dụng



So sánh (Đánh giá)

- **Ưu điểm:**

- Tách biệt business và notification
- Dễ mở rộng (thêm Zalo, Push...)
- Không sửa code cũ (OCP)
- Observer chạy độc lập (fail không ảnh hưởng nhau)
- **Nhược điểm:**
 - · Nhiều class hơn
 - · Debug flow khó hơn (không đi thẳng)
 - · Khó trace nếu không log tốt

3.2.5. Repository Pattern

Ngữ cảnh

Trong hệ thống Remedial, nhiều Service cần truy cập dữ liệu của cùng một thực thể (RemedialTerm, RemedialRegistration ...).

Nếu Service gọi trực tiếp Eloquent Model hoặc Query Builder thì:

- Business Logic phụ thuộc vào ORM.
- Query bị lặp ở nhiều nơi.
- Khó mock khi Unit Test.
- Khi thay đổi nguồn dữ liệu (Database/API/Cache) phải sửa nhiều Service.

Để giải quyết, hệ thống sử dụng **Repository Pattern** nhằm tách riêng tầng truy cập dữ liệu khỏi tầng nghiệp vụ.

Mã nguồn TRƯỚC khi áp dụng Pattern (Giả định)

Ví dụ: Service gọi trực tiếp Eloquent

```
class ManageRemedialTermService
{
    public function list(): array
    {
        return RemedialTerm::query()
            ->orderByDesc('year')
            ->orderByDesc('semester')
```

```

        ->get()

        ->all();
    }

    public function findById(int $id): ?RemedialTerm
    {
        return RemedialTerm::find($id);
    }

    public function create(array $data): RemedialTerm
    {
        return RemedialTerm::create([
            'year'          => $data['year'],
            'semester'      => $data['semester'],
            'name'          => $data['name'],
            'start_date'    => $data['start_date'] ?? null,
            'end_date'      => $data['end_date'] ?? null,
            'registration_start' => isset($data['registration_start'])
                ? Carbon::parse($data['registration_start'])->startOfDay() : null,
            'registration_end' => isset($data['registration_end'])
                ? Carbon::parse($data['registration_end'])->endOfDay() : null,
            'remedial_coefficient' => $data['remedial_coefficient'] ?? 1,
            'price_per_period' => $data['price_per_period'] ?? 150000,
            'status'        => 0, // DRAFT
        ]);
    }

    public function delete(int $id): void
    {
        $term = RemedialTerm::findOrFail($id);

        $hasRegistrations = RemedialRegistration::where('remedial_term_id', $id)
            ->where('is_deleted', false)

```



```

        ->exists();
    if ($hasRegistrations) {
        throw new \DomainException('Không thể xóa đợt phụ đạo vì đã có đăng ký.');
```

```

    }

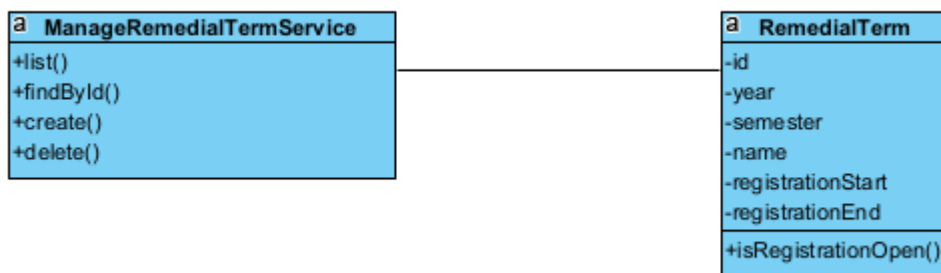
    $term->update(['is_deleted' => true]);
}
}

```

Ở đây:

- Service gọi trực tiếp Eloquent
- Query nằm trong Service
- Service biết quá nhiều về Database

Class Diagram TRƯỚC:



Mã nguồn SAU khi áp dụng Pattern:

Port (interface trong Domain)

```

interface RemedialTermRepositoryPort
{
    public function findById(int $id): ?RemedialTerm;
    public function findCurrent(): ?RemedialTerm;
    /** @return RemedialTerm[] */
    public function findAll(): array;
    public function save(RemedialTerm $term): RemedialTerm;
    public function softDelete(int $id): void;
}

```

```

    public function hasActiveRegistrations(int $id): bool;
}

```

Application Service (chỉ phụ thuộc Port + Entity)

```

class ManageRemedialTermService
{
    public function __construct(
        private readonly RemedialTermRepositoryPort $termRepository,
    ) {}

    public function list(): array
    {
        return $this->termRepository->findAll();
    }

    public function create(array $data): RemedialTerm
    {
        return $this->termRepository->save($this->buildEntity(null, $data));
    }

    public function delete(int $id): void
    {
        $this->requireById($id);
        if ($this->termRepository->hasActiveRegistrations($id)) {
            throw new \DomainException('Không thể xóa đợt phụ đạo vì đã có đăng ký.');
```

```

        // ... kiểm tra nghiệp vụ ...

        return $this->termRepository->save($term->transitionTo($status));
    }
}

```

Repository Adapter (Infrastructure)

class EloquentRemedialTermRepository implements RemedialTermRepositoryPort

```

{
    public function findAll(): array
    {
        return RemedialTermModel::query()
            ->orderByDesc('year')->orderByDesc('semester')->orderByDesc('id')
            ->get()
            ->map(fn ($m) => RemedialTermMapper::toDomain($m))
            ->all();
    }

    public function save(RemedialTerm $term): RemedialTerm
    {
        $model = $term->id
            ? RemedialTermModel::findOrFail($term->id)
            : new RemedialTermModel();

        $model->fill(RemedialTermMapper::toModelAttributes($term));
        $model->save();

        return RemedialTermMapper::toDomain($model->fresh());
    }

    public function softDelete(int $id): void
    {
        RemedialTermModel::whereKey($id)->update(['is_deleted' => true]);
    }
}

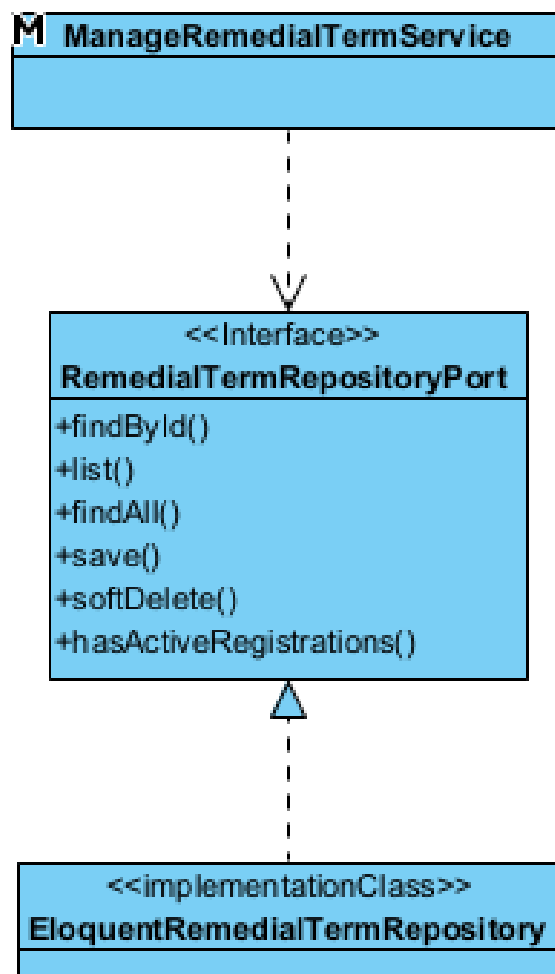
```

```

public function hasActiveRegistrations(int $id): bool
{
    return RemedialRegistrationModel::where('remedial_term_id', $id)
        ->where('is_deleted', false)->exists();
}
}

```

Class Diagram SAU:



So sánh (Đánh giá)

Ưu điểm

- Tách biệt Business Logic và Data Access.
- Tuân thủ **Dependency Inversion Principle (DIP)** khi Service chỉ phụ thuộc vào Repository Interface.

- Dễ thay đổi nguồn dữ liệu (Database, API, Cache...) mà không ảnh hưởng Service.
- Dễ viết Unit Test bằng cách Mock Repository.
- Giảm lặp query giữa các Service.

Nhược điểm

- Tăng số lượng Interface và Class.

Đối với CRUD đơn giản có thể tạo thêm một tầng trừu tượng không thật sự cần thiết.

3.3. Thiết kế cơ sở dữ liệu

Thiết kế các bảng

1. Bảng Departments

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK, Auto Increment	Mã khoa
department_code	varchar(255)	NOT NULL	Mã khoa
faculty_code	varchar(255)	NOT NULL	Mã khoa quản lý
faculty_name	varchar(255)	NULL	Tên khoa
name	varchar(255)	NOT NULL	Tên bộ môn
email	varchar(255)	NULL	Email liên hệ
phone_number	varchar(255)	NULL	Số điện thoại
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint(1)	Default 0	Đánh dấu xóa mềm

2. Bảng Users

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã người dùng
name	varchar(255)	NOT NULL	Họ tên
email	varchar(255)	UNIQUE	Email đăng nhập
email_verified_at	timestamp	NULL	Thời gian xác thực email
password	varchar(255)	NOT NULL	Mật khẩu

role	varchar(255)	NOT NULL	Vai trò người dùng
student_code	varchar(255)	NULL	Mã sinh viên
department_id	bigint unsigned	FK → departments(id)	Khoa quản lý
is_deleted	tinyint(1)	Default 0	Xóa mềm
remember_token	varchar(100)	NULL	Token đăng nhập
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật

3. Bảng Subjects

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã môn
subject_code	varchar(255)	NOT NULL	Mã môn học
name	varchar(255)	NOT NULL	Tên môn học
credits	int	NOT NULL	Số tín chỉ
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint(1)	Default 0	Xóa mềm
department_id	bigint unsigned	FK → departments(id)	Khoa quản lý

4. Bảng Remedial_Terms

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã đợt học
name	varchar(255)	NOT NULL	Tên đợt
year	int	NOT NULL	Năm học
semester	int	NOT NULL	Học kỳ
start_date	datetime	NOT NULL	Ngày bắt đầu
end_date	datetime	NOT NULL	Ngày kết thúc
registration_start	datetime	NOT NULL	Mở đăng ký
registration_end	datetime	NOT NULL	Đóng đăng ký

remedial_coefficient	int	NOT NULL	Hệ số học cải thiện
price_per_period	int	NOT NULL	Giá mỗi tiết
price_coefficient	int	NOT NULL	Hệ số học phí
is_current_term	tinyint	Default 0	Đợt hiện hành
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint	Default 0	Xóa mềm
status	tinyint unsigned	NOT NULL	Trạng thái

5. Bảng Remedial_Registrations

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã đăng ký
remedial_periods	int	NOT NULL	Số tiết đăng ký
registration_date	datetime	NOT NULL	Ngày đăng ký
lecture_name	varchar(255)	NOT NULL	Tên giảng viên
lecturer_phone_number	varchar(255)	NULL	SĐT giảng viên
lecturer_email	varchar(255)	NULL	Email giảng viên
student_id	bigint unsigned	FK → users(id)	Sinh viên đăng ký
remedial_term_id	bigint unsigned	FK → remedial_terms(id)	Đợt học
subject_id	bigint unsigned	FK → subjects(id)	Môn học
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint	Default 0	Xóa mềm

6. Bảng Teachers

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã giảng viên
department_id	varchar(255)	NOT NULL	Mã bộ môn
first_name	varchar(255)	NOT NULL	Họ
last_name	varchar(255)	NOT NULL	Tên
email	varchar(255)	NOT NULL	Email
phone	varchar(255)	NULL	Số điện thoại
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật

7. Bảng Students

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã sinh viên
student_code	varchar(255)	NOT NULL	Mã sinh viên
full_name	varchar(255)	NOT NULL	Họ tên
email	varchar(255)	NOT NULL	Email
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint	Default 0	Xóa mềm

8. Bảng System_Configurations

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	bigint unsigned	PK	Mã cấu hình
key	varchar(255)	NOT NULL	Khóa cấu hình
value	varchar(255)	NOT NULL	Giá trị cấu hình
description	varchar(255)	NULL	Mô tả
created_at	timestamp	NULL	Ngày tạo
updated_at	timestamp	NULL	Ngày cập nhật
is_deleted	tinyint	Default 0	Xóa mềm

Quan hệ giữa các thực thể

*1. Mối quan hệ giữa bảng **Departments** và bảng **Users***

Bảng **Departments** lưu thông tin về các khoa hoặc bộ môn trong trường. Mỗi khoa được định danh duy nhất bởi trường **id**, đóng vai trò là khóa chính của bảng. Trong khi đó, bảng **Users** lưu thông tin các tài khoản người dùng của hệ thống, bao gồm quản trị viên, cán bộ quản lý và sinh viên. Mỗi người dùng sẽ thuộc về một khoa thông qua trường **department_id**, đây là khóa ngoại tham chiếu đến trường **id** của bảng **Departments**.

Mối quan hệ giữa hai bảng là **một - nhiều (One-to-Many)**, nghĩa là một khoa có thể quản lý nhiều người dùng khác nhau, nhưng mỗi người dùng chỉ thuộc về một khoa duy nhất.

Quan hệ này giúp hệ thống dễ dàng quản lý người dùng theo từng đơn vị đào tạo, hỗ trợ thống kê số lượng sinh viên hoặc cán bộ của từng khoa và phục vụ công tác phân quyền trong hệ thống.

*2. Mối quan hệ giữa bảng **Departments** và bảng **Subjects***

Bảng **Subjects** lưu danh mục các môn học của nhà trường. Mỗi môn học sẽ thuộc về một khoa quản lý thông qua trường **department_id**, là khóa ngoại tham chiếu đến trường **id** của bảng **Departments**.

Mối quan hệ giữa hai bảng cũng là **một - nhiều (One-to-Many)**. Một khoa có thể quản lý nhiều môn học khác nhau, trong khi mỗi môn học chỉ được quản lý bởi một khoa duy nhất.

Quan hệ này giúp nhà trường phân chia môn học theo từng khoa chuyên môn, thuận tiện cho việc quản lý chương trình đào tạo cũng như phân công giảng dạy.

*3. Mối quan hệ giữa bảng **Users** và bảng **Remedial_Registrations***

Bảng **Users** lưu thông tin tài khoản của sinh viên và các đối tượng sử dụng hệ thống. Trong khi đó, bảng **Remedial_Registrations** lưu toàn bộ thông tin về các lần đăng ký học cải thiện của sinh viên.

Trường **student_id** trong bảng **Remedial_Registrations** là khóa ngoại tham chiếu đến trường **id** của bảng **Users**.

Mối quan hệ giữa hai bảng là **một - nhiều (One-to-Many)**. Một sinh viên có thể đăng ký học cải thiện nhiều môn học khác nhau hoặc đăng ký nhiều lần ở các học kỳ khác nhau. Tuy nhiên, mỗi bản ghi đăng ký chỉ thuộc về một sinh viên duy nhất.

Đây là mối quan hệ quan trọng nhất trong hệ thống vì nó phản ánh toàn bộ lịch sử đăng ký học cải thiện của từng sinh viên.

4. Mối quan hệ giữa bảng **Subjects** và bảng **Remedial_Registrations**

Bảng **Subjects** lưu thông tin về các môn học, còn bảng **Remedial_Registrations** lưu các lượt đăng ký học cải thiện.

Trường **subject_id** trong bảng **Remedial_Registrations** là khóa ngoại tham chiếu đến trường **id** của bảng **Subjects**.

Mối quan hệ giữa hai bảng là **một - nhiều (One-to-Many)**. Một môn học có thể được nhiều sinh viên đăng ký học cải thiện, nhưng mỗi bản ghi đăng ký chỉ tương ứng với một môn học cụ thể.

Quan hệ này cho phép hệ thống thống kê số lượng sinh viên đăng ký theo từng môn học, đồng thời hỗ trợ việc lập danh sách lớp học cải thiện.

5. Mối quan hệ giữa bảng **Remedial_Terms** và bảng **Remedial_Registrations**

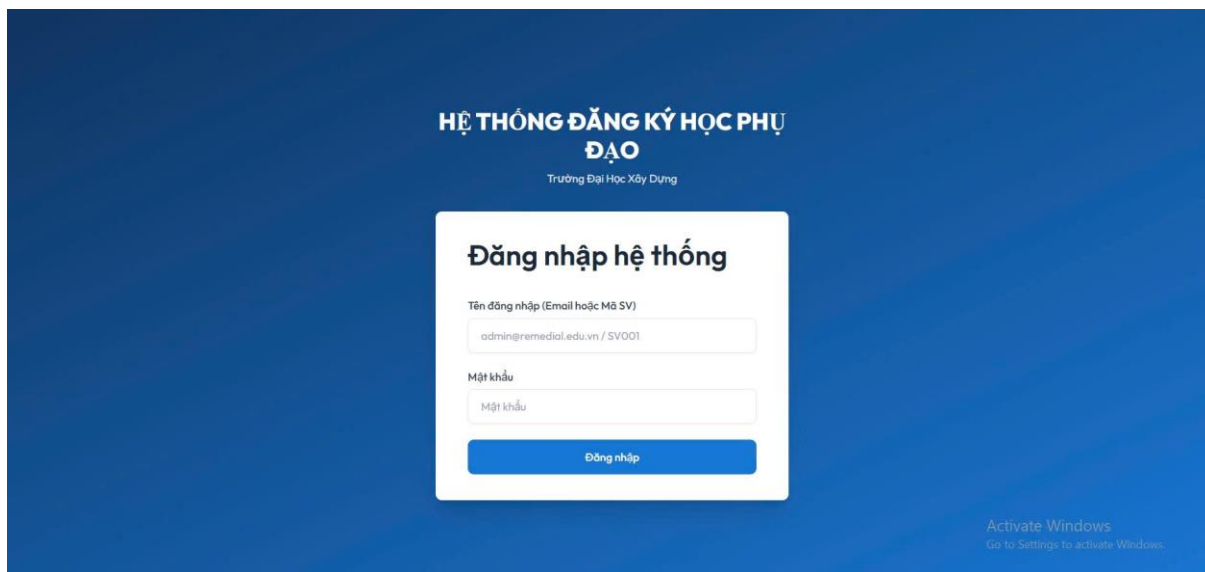
Bảng **Remedial_Terms** lưu thông tin về các đợt hoặc học kỳ mở đăng ký học cải thiện, bao gồm thời gian đăng ký, thời gian học, mức học phí và trạng thái của đợt đăng ký.

Bảng **Remedial_Registrations** sử dụng trường **remedial_term_id** làm khóa ngoại để tham chiếu đến trường **id** của bảng **Remedial_Terms**.

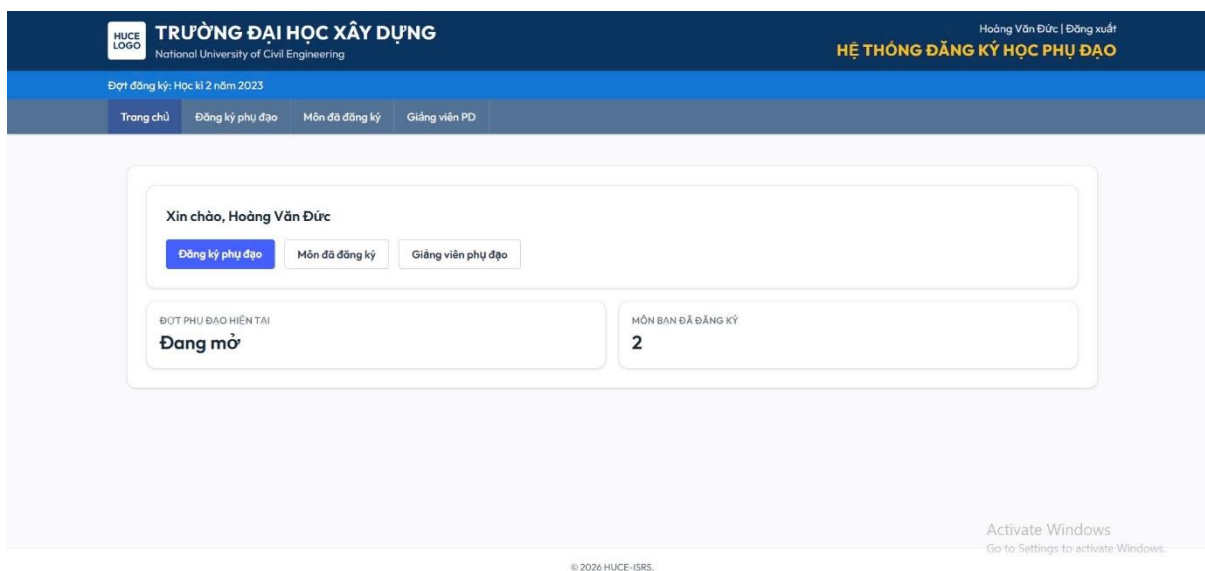
Mối quan hệ giữa hai bảng là **một - nhiều (One-to-Many)**. Một đợt học cải thiện có thể có rất nhiều sinh viên đăng ký, trong khi mỗi bản ghi đăng ký chỉ thuộc về một đợt học cải thiện duy nhất.

Quan hệ này giúp hệ thống quản lý các đợt đăng ký theo từng học kỳ, thống kê số lượng sinh viên đăng ký và kiểm soát thời gian mở hoặc đóng đăng ký.

3.4. Giao diện người dùng



Hình 3.4.1. Giao diện đăng nhập



Hình 3.4.2. Giao diện trang chủ sinh viên

HUCE LOGO

TRƯỜNG ĐẠI HỌC XÂY DỰNG

National University of Civil Engineering

Hoàng Văn Đức | Đồng xuất

HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt đăng ký: Học kì 2 năm 2023

Trang chủ

Đăng ký phụ đạo

Môn đã đăng ký

Giảng viên PD

Danh sách môn học đã đăng ký

Xóa đăng ký

Chọn	Mã MH	Tên môn học	Ngày đăng ký
☐	461730	Phương pháp lập trình hướng đối tượng	25/6/2026
☐	471732	Công nghệ phần mềm	25/6/2026

Danh sách môn học chính quy (kỳ đợt phụ đạo hiện tại)

Lưu đăng ký

Chọn	Mã MH	Tên môn học	Lớp HP	STC
☐	401712	Toán rời rạc	67IT4	3
☐	428803	Chủ nghĩa xã hội khoa học	67IT4	2
☐	430115	Giáo dục Thể chất 3 - Bóng rổ	LOPTC07	1
☐	440215	Tiếng Anh TOEIC 2	67IT4	
☐	461706	Mạng máy tính	67IT4	3

Hình 3.4.3. Giao diện đăng ký phụ đạo

 **TRƯỜNG ĐẠI HỌC XÂY DỰNG**
National University of Civil Engineering

Hoàng Văn Đức | Đăng xuất

HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đại đồng ký: Học kì 2 năm 2023

Trang chủ | Đăng ký phụ đạo | Môn đã đăng ký | Giảng viên PD

Hình 3.4.4. Giao diện xem môn đã đăng ký

 **TRƯỜNG ĐẠI HỌC XÂY DỰNG**
National University of Civil Engineering

Hoàng Văn Đức | Đăng xuất

HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt đăng ký: Học kì 2 năm 2023

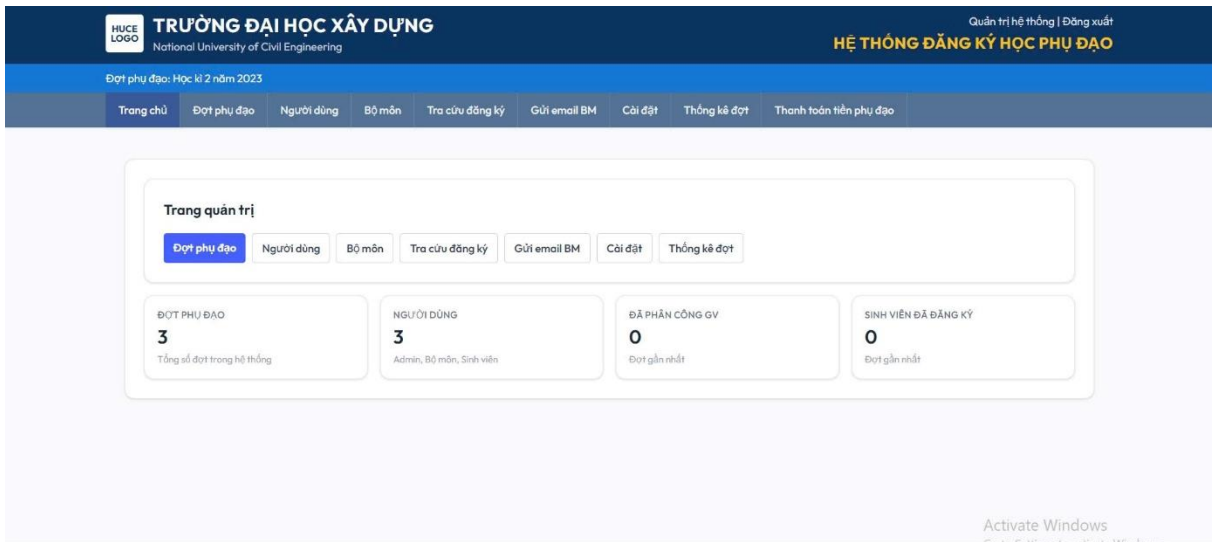
Trang chủ | Đăng ký phụ đạo | Môn đã đăng ký | Giảng viên PD

← Trang chủ sinh viên

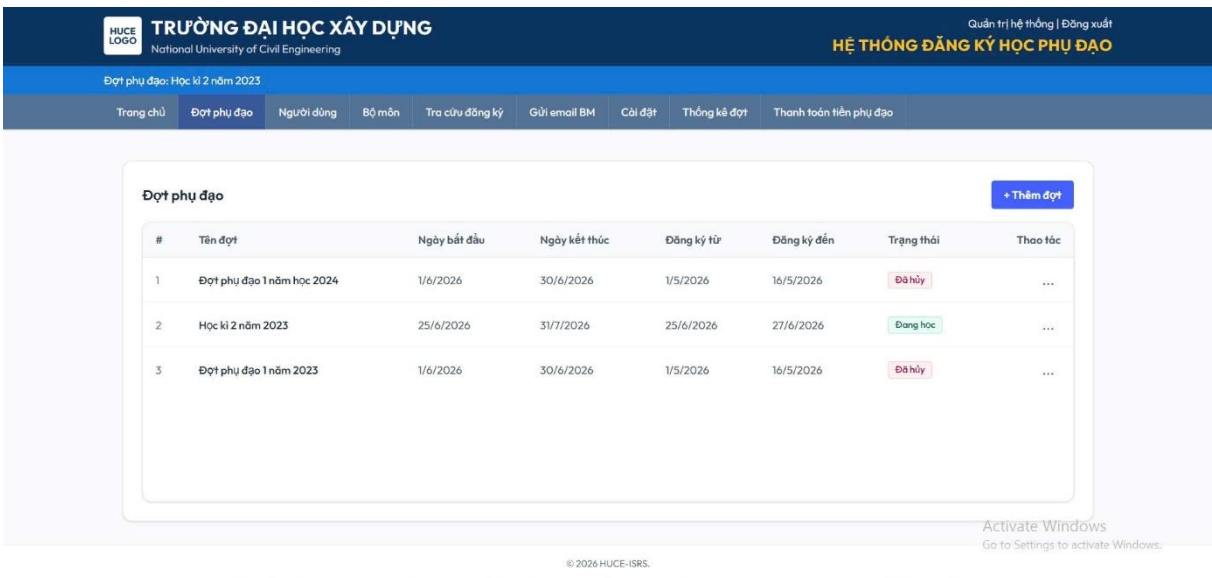
Giảng viên phụ đạo

Mã MH	Tên môn	Giảng viên	Liên hệ GV
461730	Phương pháp lập trình hướng đối tượng	Chưa phân công	—
471732	Công nghệ phần mềm	Đức Hoàng Văn	hduc18535@gmail.com 012345678

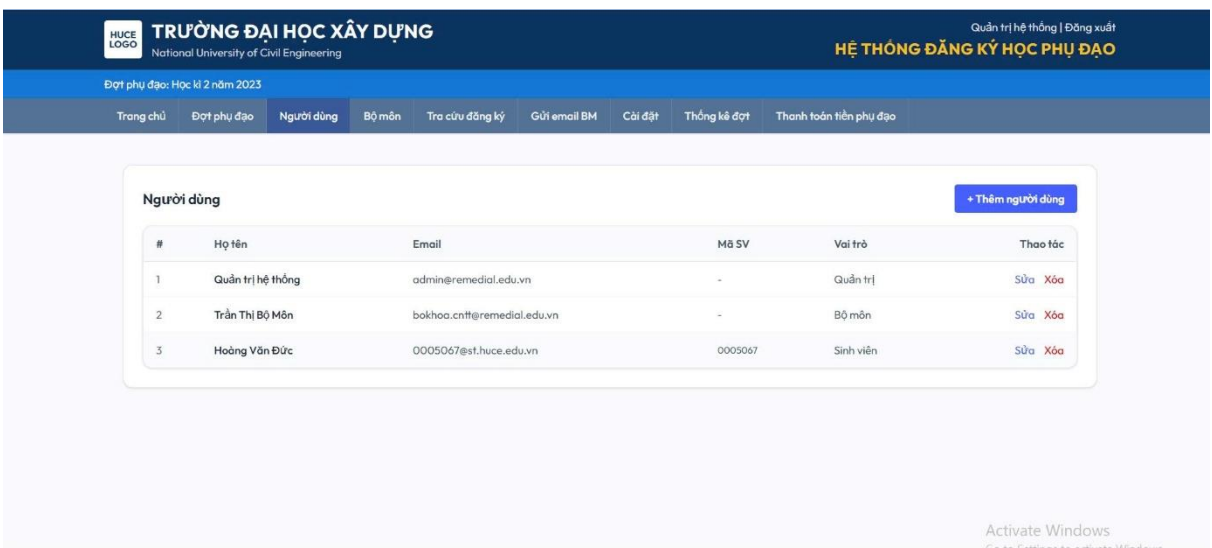
Hình 3.4.5. Giao diện xem giảng viên phụ đạo



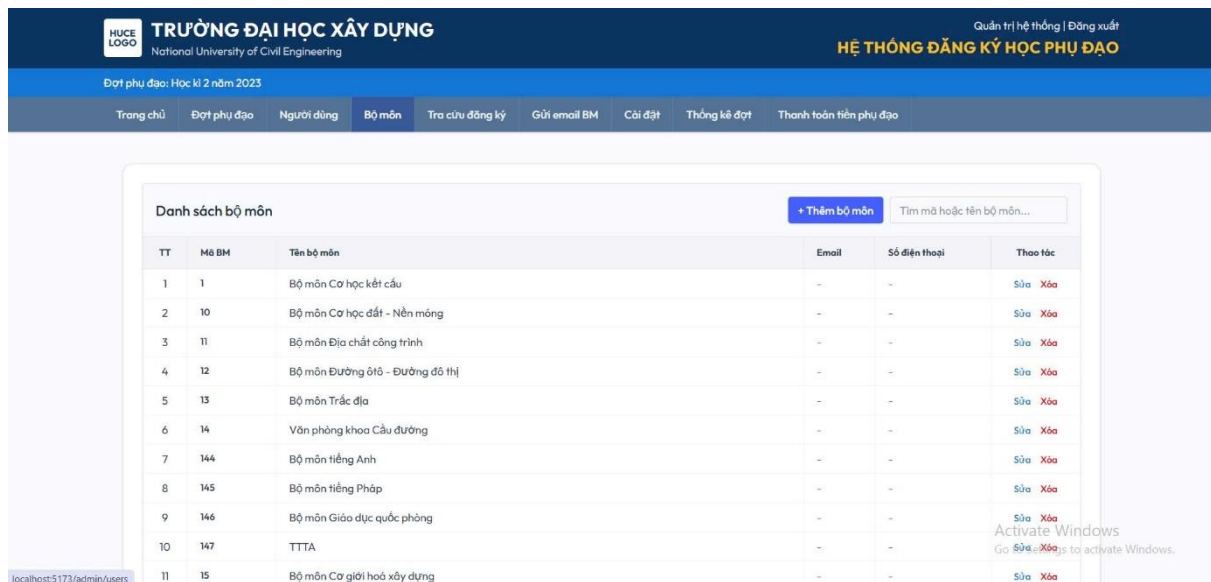
Hình 3.4.6. Giao diện trang chủ admin



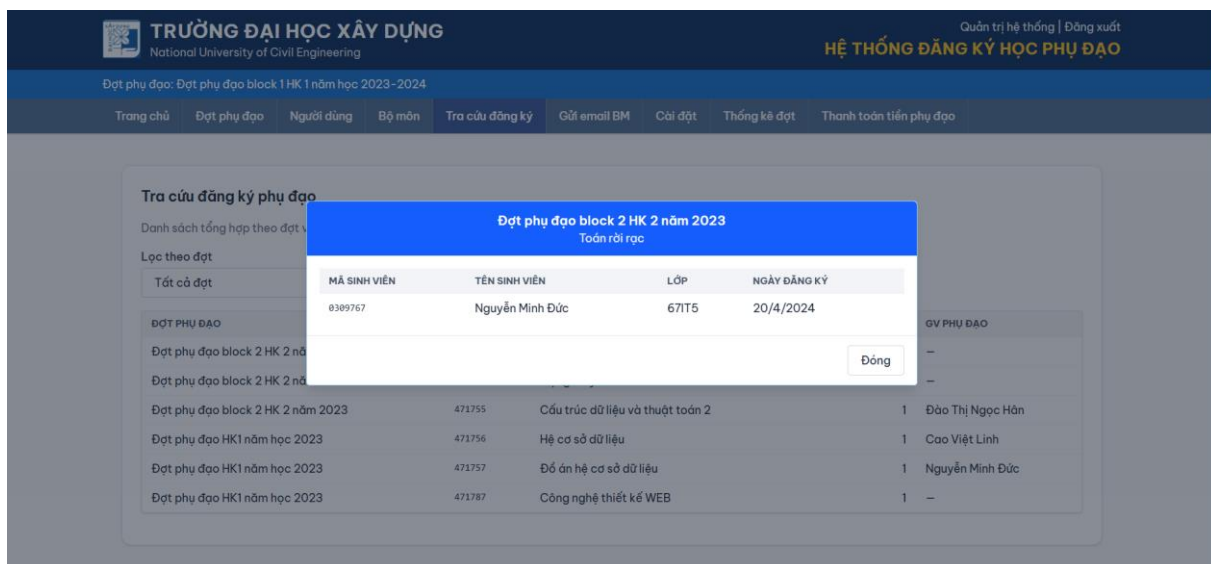
Hình 3.4.7. Giao diện đợt phụ đạo



Hình 3.4.8. Giao diện xem người dùng



Hình 3.4.9. Giao diện xem bộ môn



Hình 3.4.10. Giao diện tra cứu đăng ký

TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Quản trị hệ thống | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt phụ đạo: Học kì 2 năm 2023

Trang chủ | Đợt phụ đạo | Người dùng | Bộ môn | Tra cứu đăng ký | **Gửi email BM** | Cài đặt | Thống kê đợt | Thanh toán tiền phụ đạo

Gửi email về bộ môn

Bộ môn
Bộ môn Cơ học kết cấu (1)

Tiêu đề
Thông báo phụ đạo – danh sách môn/SV

Nội dung
Kính gửi Trưởng bộ môn,

Vui lòng xem danh sách môn phụ đạo và sinh viên đăng ký trong hệ thống.

Gửi email

Activate Windows
Go to Settings to activate Windows.

Hình 3.4.11. Giao diện gửi email BH

TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Quản trị hệ thống | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt phụ đạo: Học kì 2 năm 2023

Trang chủ | Đợt phụ đạo | Người dùng | Bộ môn | Tra cứu đăng ký | **Gửi email BM** | **Cài đặt** | Thống kê đợt | Thanh toán tiền phụ đạo

Cấu hình hệ thống

Key	Giá trị	Mô tả	Hành động
admin_email	phongdaotao@nuce.edu.vn	Email đơn vị quản lý	Lưu Xóa
default_periods	45	Số tiết học mặc định khi đăng ký pt	Lưu Xóa
mail_summary_body	Kính gửi Bộ môn, đây là danh sách	Nội dung email gửi tóm tắt cho bộ	Lưu Xóa
mail_summary_subject	Danh sách sinh viên đăng ký học pt	Tiêu đề email gửi tóm tắt cho bộ m	Lưu Xóa
sender_email	byebyevidu@gmail.com	Email dùng để gửi email hệ thống	Lưu Xóa
sender_password		Mật khẩu email gửi	Lưu Xóa
weeks_from_registration	10	Số tuần tính từ tuần đăng ký	Lưu Xóa
ws_host	http://127.0.0.1:8001	Host webservice học phụ đạo	Lưu Xóa

Activate Windows
Go to Settings to activate Windows.

Hình 3.4.12. Giao diện cài đặt

TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Quản trị hệ thống | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt phụ đạo: Học kì 2 năm 2023

Trang chủ | Đợt phụ đạo | Người dùng | Bộ môn | Tra cứu đăng ký | **Gửi email BM** | **Cài đặt** | **Thống kê đợt** | Thanh toán tiền phụ đạo

Thống kê đợt phụ đạo

TT	Tên đợt phụ đạo	Số sinh viên	Số môn học	Tổng tiền
1	Đợt phụ đạo 1 năm học 2024	0	0	0 đ
2	Học kì 2 năm 2023	1	2	13.500.000 đ
3	Đợt phụ đạo 1 năm 2023	1	5	33.750.000 đ

Hình 3.4.13. Giao diện thống kê đợt



TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Quản trị hệ thống | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Đợt phụ đạo: Học kì 2 năm 2023

[Trang chủ](#)
[Đợt phụ đạo](#)
[Người dùng](#)
[Bộ môn](#)
[Tra cứu đăng ký](#)
[Gửi email BM](#)
[Cài đặt](#)
[Thống kê đợt](#)
[Thanh toán tiền phụ đạo](#)

Đợt phụ đạo:
Đợt phụ đạo 1 năm học 2024

Tên giảng viên...

BẢNG TỔNG HỢP THANH TOÁN GIẢNG DẠY PHỤ ĐẠO
Đợt phụ đạo 1 năm học 2024

TT	GV phụ đạo	SBT GV	Mã MH	Tên MH	ST PB	Đơn giá/ 1 tiết	Số tiền (*hệ số 1)	Tổng tiền
Không có dữ liệu								

Xuất excel

Trang 1 / 1

Hình 3.4.14. Giao diện thanh toán tiền phụ đạo



TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Trần Thị Bộ Môn | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Hệ thống đăng ký học phụ đạo — Bộ môn

[Trang chủ](#)
[Thông tin bộ môn](#)
[Phản công GV](#)

Xin chào, **Trần Thị Bộ Môn**
Khu vực quản lý bộ môn — chỉ xem đăng ký thuộc bộ môn của bạn.

[Thông tin bộ môn](#)
[Đăng ký & phản công GV](#)

ĐĂNG KÝ PHỤ ĐẠO THUỘC BỘ MÔN
2

Hình 3.4.15. Trang chủ bộ môn



TRƯỜNG ĐẠI HỌC XÂY DỰNG
National University of Civil Engineering

Trần Thị Bộ Môn | Đăng xuất
HỆ THỐNG ĐĂNG KÝ HỌC PHỤ ĐẠO

Hệ thống đăng ký học phụ đạo — Bộ môn

[Trang chủ](#)
[Thông tin bộ môn](#)
[Phản công GV](#)

Thông tin bộ môn của tôi

Mã bộ môn: 54

Tên bộ môn
Bộ môn Công nghệ phần mềm

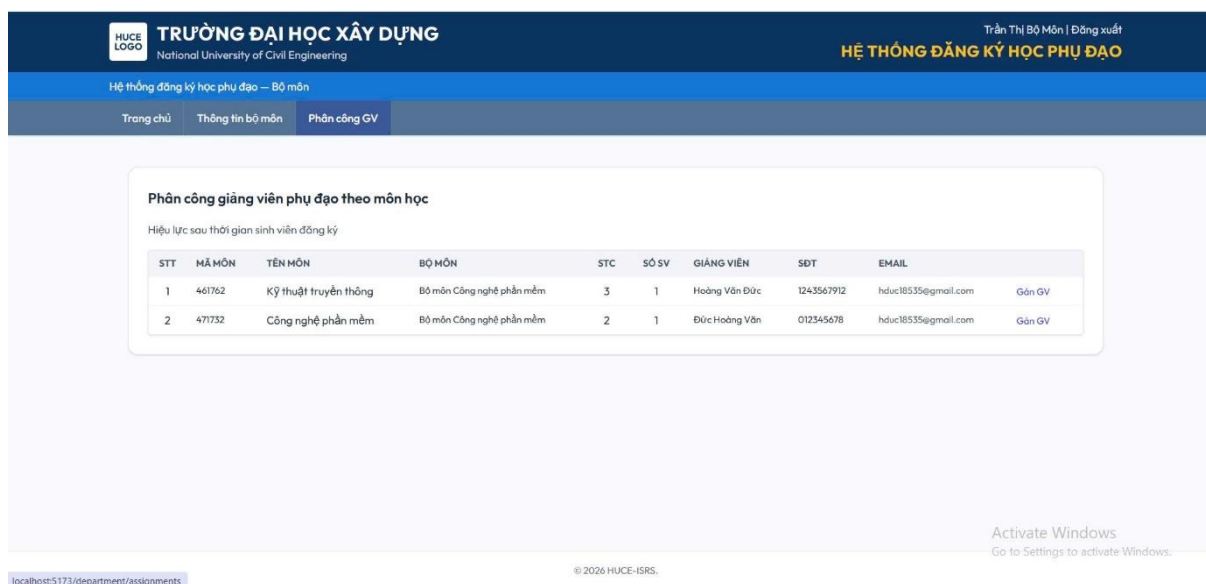
Email liên hệ bộ môn

Số điện thoại liên hệ

Tài khoản đăng nhập
Họ tên hiển thị
Trần Thị Bộ Môn

Email đăng nhập

Hình 3.4.16. Giao diện thông tin bộ môn



Hình 3.4.17. Giao diện xem phân công GV

Chương IV. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

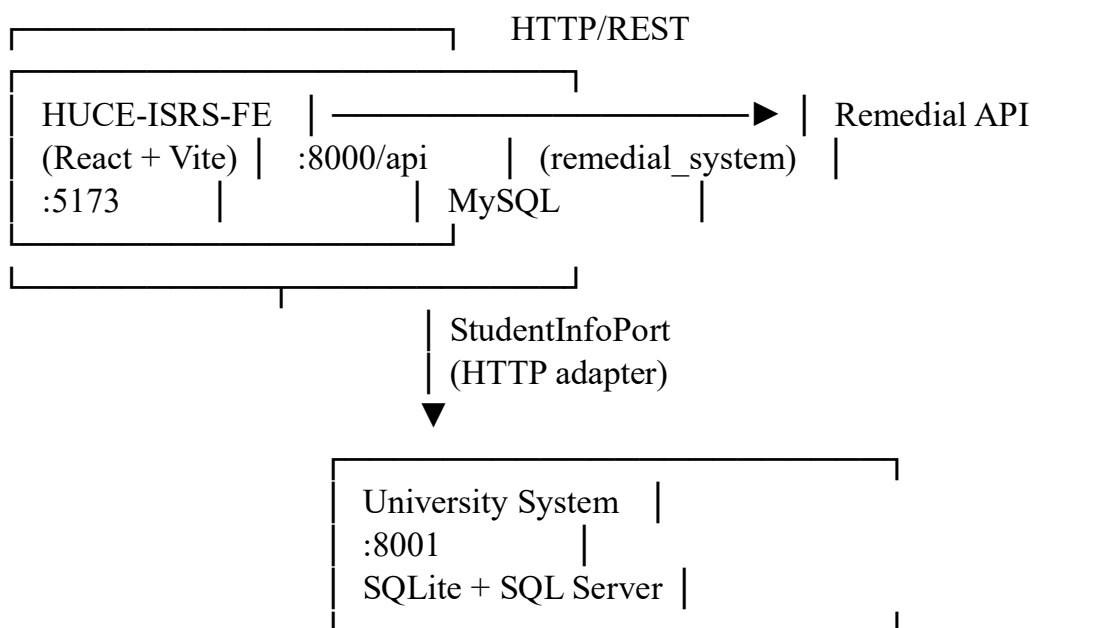
4.1. Môi trường triển khai

4.1.1. Kiến trúc tổng thể

HUCE-ISRS được tổ chức dưới dạng **monorepo** gồm ba thành phần độc lập, giao tiếp qua HTTP/REST API:

Thành phần	Thư mục	Công nghệ	Cổng mặc định	Vai trò
Remedial API	HUCE-ISRS-BE/remedial_system	Laravel 12, PHP 8.2+, MySQL	8000	API nghiệp vụ phụ đạo (kiến trúc Hexagonal)
University System	HUCE-ISRS-BE/university_system	Laravel 12, PHP 8.2+, SQLite/SQL Server	8001	Mock/gateway xác thực và dữ liệu sinh viên từ hệ thống trường
Frontend	HUCE-ISRS-FE	React 19, TypeScript, Vite, Tailwind CSS	5173 (dev)	Giao diện web cho Admin, Bộ môn, Sinh viên

Sơ đồ triển khai môi trường phát triển:



4.1.2. Môi trường phát triển (Development)

Trong quá trình phát triển và kiểm thử, hệ thống được triển khai trên máy trạm cục bộ (localhost):

- **Hệ điều hành:** Windows 10/11, macOS hoặc Linux (đã kiểm thử trên Windows 10)
- **Web server:** PHP built-in server (php artisan serve) cho hai backend; Vite dev server cho frontend
- **Cơ sở dữ liệu:**
 - Remedial System: MySQL 8.0+ trên 127.0.0.1:3306
 - University System: SQLite (metadata API client/token); tùy chọn SQL Server cho dữ liệu sinh viên thật (EDU_NUCE)
- **Trình duyệt:** Chrome, Edge, Firefox (phiên bản mới hỗ trợ ES modules)

4.1.3. Môi trường triển khai thực tế (Production — định hướng)

Để đưa vào vận hành tại Trường, kiến trúc có thể mở rộng như sau:

Thành phần	Đề xuất triển khai
Remedial API	Nginx/Apache + PHP-FPM 8.2+, MySQL dedicated
University System	Thay thế bằng API thật của trường; hoặc gateway nội bộ trên cổng riêng
Frontend	Build tĩnh (npm run build) phục vụ qua Nginx hoặc CDN

Thành phần	Đề xuất triển khai
Bảo mật	HTTPS, APP_DEBUG=false, Sanctum token, phân quyền theo role middleware
Email	Cấu hình SMTP thật thay cho MAIL_MAILER=log

Các endpoint chính sau khi triển khai:

- Giao diện người dùng: <http://localhost:5173> (dev) hoặc domain production
- API phụ đạo: <http://127.0.0.1:8000/api>
- Tài liệu Swagger API: <http://localhost:8000/api/documentation>
- University System: <http://localhost:8001/api>

4.2. Cài đặt hệ thống

4.2.1. Yêu cầu hệ thống

Phần cứng tối thiểu (môi trường phát triển)

Hạng mục	Yêu cầu
CPU	2 nhân trở lên
RAM	8 GB trở lên (khuyến nghị 16 GB khi chạy đồng thời MySQL + 3 service)
Ổ cứng	Tối thiểu 2 GB trống (bao gồm vendor/, node_modules/, database)
Mạng	Kết nối Internet khi cài đặt dependency lần đầu

Phần mềm

Thành phần	Phiên bản	Ghi chú
PHP	≥ 8.2	Extension bắt buộc: pdo_mysql, mbstring, openssl, tokenizer, xml,

Thành phần	Phiên bản	Ghi chú
		ctype, json, bcmath; gd cho xuất Excel
Composer	≥ 2.0	Quản lý dependency PHP
Node.js	≥ 18	Chạy frontend Vite
npm	≥ 9	Cài đặt package JavaScript
MySQL	≥ 8.0	Database cho Remedial System
SQLite	3.x	Mặc định cho University System (metadata)
SQL Server + PHP sqlsrv driver	Tùy chọn	Khi kết nối database trường EDU_NUCE
Git	2.x	Clone mã nguồn monorepo

Thư viện chính (theo composer.json / package.json)

Backend (Remedial System):

- Laravel Framework ^12.0
- Laravel Sanctum ^4.3 (xác thực API token)
- darkaonline/l5-swagger ^11.0 (tài liệu OpenAPI)
- maatwebsite/excel ^3.1 (xuất báo cáo Excel)

Frontend:

- React ^19.2, React Router DOM ^7.14
- TypeScript ~6.0, Vite ^8.0
- Tailwind CSS ^4.2

Tài khoản và quyền truy cập

- Quyền tạo database MySQL (CREATE DATABASE, GRANT)
- Quyền ghi thư mục storage/, bootstrap/cache/ (Laravel)
- (Tùy chọn) Quyền SELECT trên SQL Server trường cho University System

4.2.2. Các bước cài đặt chương trình

Cài đặt yêu cầu mở **ba terminal** (hoặc chạy song song ba service). Thứ tự khuyến nghị: University System → Remedial API → Frontend.

Bước 0: Chuẩn bị mã nguồn và database

```
# Clone hoặc giải nén monorepo, di chuyển vào thư mục gốc
cd HUCE-ISRS
```

```
# Tạo database MySQL cho Remedial System (ví dụ qua MySQL CLI hoặc
phpMyAdmin)
```

```
# CREATE DATABASE remedial_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

Bước 1: Cài đặt University System (cổng 8001)

University System phải chạy trước vì Remedial API gọi sang đây để xác thực sinh viên và đồng bộ dữ liệu.

```
cd HUCE-ISRS-BE\university_system
composer install
```

```
copy .env.example .env
```

```
# Chỉnh .env: DB_CONNECTION=sqllite (mặc định)
```

```
# Nếu dùng SQL Server trường: cấu hình DB_SQLSRV_HOST,
DB_SQLSRV_DATABASE, ...
```

```
# Tạo file SQLite nếu chưa có (Linux/macOS: touch database/database.sqlite)
New-Item -ItemType File -Path database\database.sqlite -Force
```

```
php artisan key:generate
```

```
php artisan migrate --seed
```

```
php artisan serve --port=8001
```

Kiểm tra: truy cập <http://localhost:8001/api/documentation>.

Bước 2: Cài đặt Remedial API (cổng 8000)

```
cd HUCE-ISRS-BE\remedial_system
```

```
composer install
```

```
# Nếu thiếu extension GD trên máy dev: composer install --ignore-platform-reqs
```

```
copy .env.example .env
```

Chỉnh file .env với các tham số quan trọng:

```
DB_CONNECTION=mysql
```

```
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
```

```
DB_DATABASE=remedial_db
DB_USERNAME=root
DB_PASSWORD=<mật_khẩu_mysql>
```

```
UNIVERSITY_BASE_URL=http://localhost:8001
UNIVERSITY_CLIENT_ID=remedial_system
UNIVERSITY_CLIENT_SECRET=remedial_secret_2024
```

```
APP_DEBUG=true
APP_URL=http://localhost:8000
```

Tiếp tục khởi tạo:

```
php artisan key:generate
php artisan migrate --seed
php artisan l5-swagger:generate # tùy chọn — tạo tài liệu API
php artisan serve
```

Kiểm tra: <http://127.0.0.1:8000/api> phản hồi; Swagger tại </api/documentation>.

Bước 3: Cài đặt Frontend (cổng 5173)

```
cd HUCE-ISRS-FE
npm install
npm run dev
```

Mở trình duyệt: <http://localhost:5173>.

Frontend gọi API qua `src/shared/utis/apiClient.ts` với base URL <http://127.0.0.1:8000/api>. Khi triển khai production, cần chỉnh biến môi trường hoặc cấu hình reverse proxy tương ứng.

Bước 4: Build production (tùy chọn)

```
# Frontend
cd HUCE-ISRS-FE
npm run build
npm run preview
```

```
# Backend — tối ưu Laravel
cd HUCE-ISRS-BE\remedial_system
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Bước 5: Xác minh cài đặt

Kiểm tra	Cách thực hiện	Kết quả mong đợi
API Remedial	GET http://127.0.0.1:8000	Phản hồi HTTP 200
Đăng nhập Admin	FE → email admin@remedial.edu.vn	Redirect /admin
Đăng nhập Bộ môn	bokhoa.cntt@remedial.edu.vn	Redirect /department
Đăng nhập Sinh viên	student_code + mật khẩu trường	Redirect /student
PHPUnit	php artisan test trong remedial_system	Tất cả test pass

Tài khoản mẫu sau seed

Vai trò	Đăng nhập	Mật khẩu
Admin	admin@remedial.edu.vn	Admin@2024!
Bộ môn	bokhoa.cntt@remedial.edu.vn	BoMon@2024!
Sinh viên	Mã SV từ University seed	nuce_backdoor_2026

Lệnh bảo trì thường dùng

Làm mới database và dữ liệu mẫu (Remedial)
php artisan migrate:fresh --seed

Xóa cache (gồm token University)
php artisan cache:clear

Liệt kê route API
php artisan route:list --path=api

Chương V. Kết quả và đánh giá

5.1. Kết quả thử nghiệm hệ thống

Hệ thống được kiểm thử theo hai hướng: **kiểm thử tự động (PHPUnit)** trên backend và **kiểm thử hồi quy thủ công (regression)** trên toàn bộ luồng ba vai trò qua giao diện và API.

5.1.1. Kiểm thử tự động — PHPUnit (Remedial System)

Lệnh chạy:

```
cd HUCE-ISRS-BE\remedial_system  
php artisan test
```

Kết quả thực thi (môi trường phát triển):

Chỉ số	Giá trị
Tổng số test case	22
Số assertion	76
Kết quả	22 passed, 0 failed
Thời gian	~42 giây

Chi tiết theo nhóm test:

Nhóm	Số test	Nội dung kiểm thử chính
Tests\Unit\RemedialTermRegistrationWindowTest	2	Cửa sổ đăng ký mở/đóng theo registration_end
Tests\Feature\AuthTest	4	Đăng nhập Admin, sai mật khẩu, /auth/me, đăng nhập SV qua University
Tests\Feature\RemedialTermTest	2	CRUD đợt phụ đạo (Admin); từ chối truy cập với role Bộ môn
Tests\Feature\StudentRegistrationTest	6	Đăng ký/hủy môn, môn đủ điều kiện, môn theo kỳ, lọc theo cửa sổ thi

Nhóm	Số test	Nội dung kiểm thử chính
Tests\Feature\DepartmentRegistrationTest	4	Phân công GV theo môn, chỉ sau khi hết hạn đăng ký, lọc theo đợt hiện tại
Tests\Feature\AdminRegistrationTest	1	Tra cứu đăng ký nhóm theo đợt và môn
Tests\Feature\FakeTodayMiddlewareTest	1	Định dạng hằng số ngày giả lập cho kiểm thử
Tests\Feature\ExampleTest + Unit\ExampleTest	2	Smoke test ứng dụng

Các test Feature sử dụng FakeStudentInfoPort để mô phỏng University System, đảm bảo kiểm thử độc lập không phụ thuộc service bên ngoài đang chạy.

5.1.2. Kiểm thử hồi quy thủ công — ba vai trò

Checklist regression (theo REGRESSION.md) được thực hiện với Remedial API (php artisan serve) và Frontend (npm run dev).

Admin (/admin):

#	Luồng	Kết quả
1	Đăng nhập email + password → redirect /admin	Đạt
2	CRUD đợt phụ đạo (/admin/remedial-terms)	Đạt
3	CRUD người dùng (/admin/users)	Đạt
4	CRUD bộ môn (/admin/departments)	Đạt
5	Cài đặt hệ thống (/admin/system-configurations)	Đạt
6	Tra cứu đăng ký theo đợt	Đạt
7	Gửi email tổng hợp cho Bộ môn	Đạt
8	Thống kê đợt (/admin/statistics/terms)	Đạt

Bộ môn (/department):

#	Luồng	Kết quả
1	Đăng nhập → redirect /department	Đạt
2	Xem/sửa hồ sơ (GET/PATCH /department/me)	Đạt
3	Xem đăng ký thuộc bộ môn	Đạt
4	Cập nhật giảng viên phụ đạo	Đạt
5	Không truy cập được /admin/* (HTTP 403)	Đạt

Sinh viên (/student):

#	Luồng	Kết quả
1	Đăng nhập bằng student_code	Đạt
2	Đăng ký / hủy môn phụ đạo	Đạt
3	Xem môn đủ điều kiện (eligible-subjects)	Đạt
4	Danh sách đăng ký và GV — dữ liệu từ API thật	Đạt
5	Banner đợt hiện tại (/student/remedial-terms/current)	Đạt

5.1.3. Kiểm thử tích hợp

Kịch bản	Mô tả	Kết quả
Xác thực sinh viên	Remedial gọi University System qua StudentInfoPort	Sinh viên đăng nhập thành công, tài khoản được provision lần đầu

Kịch bản	Mô tả	Kết quả
Đồng bộ môn/điểm	Lấy kết quả học tập từ trường, áp dụng SubjectEligibleForRemedial	Chỉ môn nợ/hổng trong cửa sổ thi được hiển thị
Phân quyền API	Middleware auth:sanctum + EnsureUserHasRole	Mỗi role chỉ gọi được namespace API tương ứng
Envelope JSON	{ success, message, data, errors }, snake_case	Frontend map đúng, không cần fallback PascalCase

5.1.4. Hạn chế đã ghi nhận trong kiểm thử

- Một số trang Admin cũ (phân công GV /admin/tutoring-classes, thanh toán /admin/payments) đã gỡ khỏi menu — không nằm trong phạm vi regression hiện tại.
- Kiểm thử tự động tập trung ở Remedial System; University System chỉ có test mẫu (ExampleTest).
- Frontend chưa có bộ test E2E tự động (Cypress/Playwright); regression FE thực hiện thủ công.

5.2. Đánh giá hiệu quả hệ thống

5.2.1. Mức độ đáp ứng yêu cầu nghiệp vụ

Yêu cầu	Đánh giá	Minh chứng
Quản lý đợt phụ đạo (tạo, sửa, trạng thái, hệ số học phí)	Đạt	CRUD đợt, test RemedialTermTest, trang CohortListPage
Sinh viên đăng ký/hủy môn trong cửa sổ thời gian	Đạt	StudentRegistrationTest, rule RemedialTermRegistration Window
Lọc môn đủ điều kiện theo điểm và kỳ thi	Đạt	Specification SubjectEligibleForRemedial, test eligible subjects

Yêu cầu	Đánh giá	Minh chứng
Bộ môn phân công GV sau khi hết hạn đăng ký	Đạt	DepartmentRegistrationTest — chặn assign trước registration_end
Admin tra cứu, thống kê, gửi email BM	Đạt	API statistics, send-email, regression Admin
Tích hợp hệ thống trường (xác thực + dữ liệu SV)	Đạt	University System adapter, login SV qua student_code
Phân quyền ba vai trò	Đạt	Test 403 cho Bộ môn truy cập Admin; routing FE theo home_url

5.2.2. Đánh giá kỹ thuật

Ưu điểm:

1. **Kiến trúc Hexagonal (Ports & Adapters):** Tách Domain, Application, Infrastructure giúp logic nghiệp vụ (điều kiện đăng ký, cửa sổ thời gian) được kiểm thử độc lập ở tầng Unit; thay đổi nguồn dữ liệu trường chỉ cần đổi adapter StudentInfoPort.
2. **API thống nhất:** Envelope JSON và snake_case nhất quán giữa BE-FE, giảm lỗi mapping.
3. **Bảo mật:** Laravel Sanctum + role middleware; mật khẩu Admin/Bộ môn lưu bcrypt; sinh viên xác thực qua hệ thống trường.
4. **Khả năng mở rộng:** Cấu hình động qua system_configurations; export Excel; Swagger mô tả contract API.
5. **Trải nghiệm người dùng:** Giao diện React theo từng vai trò, redirect sau login, banner đợt hiện tại cho sinh viên.

Hạn chế và hướng cải thiện:

Hạn chế	Hướng xử lý
Chưa có Docker/CI pipeline chính thức	Bổ sung docker-compose và GitHub Actions chạy php artisan test
FE hard-code API_BASE_URL	Dùng biến VITE_API_BASE_URL theo môi trường

Hạn chế	Hướng xử lý
University System là mock	Khi go-live, trở UNIVERSITY_BASE_URL sang API production của trường
Thiếu test E2E frontend	Thêm Playwright cho luồng đăng ký SV end-to-end
Email đang ghi log (MAIL_MAILER=log)	Cấu hình SMTP thật trước triển khai

5.2.3. Đánh giá hiệu quả vận hành

So với quy trình đăng ký phụ đạo thủ công (biểu mẫu giấy, tổng hợp Excel rời):

- **Giảm sai sót:** Quy tắc đủ điều kiện môn được máy tính hóa từ điểm đồng bộ trường, hạn chế đăng ký nhầm môn không đủ điều kiện.
- **Rút ngắn thời gian:** Sinh viên tự đăng ký online; Bộ môn xem danh sách tập trung; Admin thống kê theo đợt không cần tổng hợp thủ công.
- **Minh bạch:** Lịch sử đăng ký lưu database; tra cứu theo đợt/môn/bộ môn real-time.
- **Khả năng tích hợp:** Kiến trúc port cho phép thay University mock bằng hệ thống EDU_NUCE thật mà không đụng core nghiệp vụ.

5.2.4. Kết luận chương 5

Hệ thống HUCE-ISRS đã được cài đặt thành công trên môi trường phát triển với đầy đủ ba thành phần (Remedial API, University System, Frontend). **22/22** test PHPUnit đều pass; checklist regression cho Admin, Bộ môn và Sinh viên đáp ứng các luồng nghiệp vụ cốt lõi. Về mặt kiến trúc và chức năng, hệ thống đạt mục tiêu số hóa quy trình đăng ký học phần phụ đạo cho sinh viên quốc tế, sẵn sàng cho bước triển khai pilot tại Trường sau khi hoàn thiện cấu hình production (HTTPS, SMTP, kết nối database trường thật).